

城市绿色物流中混合车队路径优化与动态重调度研究

摘要

随着电子商务与城市物流需求的快速增长，以及“双碳”目标下绿色低碳约束的不断强化，城市配送调度问题日趋复杂。基于此，本文以某省会城市物流企业的混合车队配送问题为研究对象，构建了面向**成本控制与碳排放协同优化**的绿色配送调度模型，为城市物流企业运输调度决策提供支持。

针对问题一，本文构建了考虑**时变速度与载重修正能耗**的异构车队软时间窗路径优化模型（TDVRPTW-H），以最小化车辆固定成本、能源消耗成本、碳排放成本及时间窗惩罚成本为目标，并综合考虑载重、容量及路径连续性等约束。针对该模型，本文设计了**自适应大邻域搜索算法（ALNS）**进行求解，通过破坏-修复算子、自适应权重更新机制及模拟退火接受准则优化配送路径。数值结果表明，在静态情形下，方案共调动123辆车，总成本为105212.62元，碳排放量为11257.25 kg，其中**车辆固定成本与能源消耗成本**占比较高，是静态配送成本的主要来源。

针对问题二，在基准模型中引入“8:00-16:00 燃油车禁入绿色配送区”的政策约束，设计**新能源车优先、时空错位调度与路径柔性绕行**三类响应策略，并比较该政策对**总成本、车辆使用结构与碳排放总量**的影响。结果表明，绿色限行政策使系统总成本由基准方案的105212.62元上升至132199.33元及以上，成本增加主要来自车辆替换、服务时刻调整和路径重构；车辆使用结构由燃油车主导转向燃油车与新能源车协同，其中新能源车优先策略下新能源车使用率提高至15.54%；碳排放总量受策略选择影响明显，新能源车优先策略碳排放量最低，为12947.05 kg。因此，新能源车优先策略可作为绿色限行情景下兼顾成本与低碳目标的推荐方案。

针对问题三，本文构建了基于**局部冻结与最小边际成本插入**的事件驱动动态重调度模型，以动态总成本最小为目标，将运营成本、路径扰动成本和未响应任务惩罚成本纳入统一目标函数，并综合考虑车辆容量、时间窗、绿色限行和任务服务状态等约束。算法上，引入局部冻结机制锁定近端执行计划，建立动态任务池统一刻画订单取消、新增订单、地址变更和时间窗调整等突发事件，并采用**最小边际成本插入算法（MCI）**实现局部路径快速修正。结果表明，动态调度方案实现突发任务100%响应，总成本为134851.13元，总碳排放量为12920.25 kg，路径扰动比例仅为1.26%，说明该策略能够在保持原方案稳定性的同时实现快速动态响应。

针对灵敏度分析，为验证基准方案在复杂交通环境下的稳定性，本文对分时段车速施加 $\pm 20\%$ 扰动，模拟不同交通状况并进行灵敏度分析。结果表明，总成本波动控制在1%以内，说明调度方案具有良好的**鲁棒性**；其中，时间窗惩罚成本最为敏感，表明交通扰动主要通过影响运输时间进而作用于时效成本。

综上，本文构建的城市混合车队绿色配送调度模型能够有效刻画**静态配送优化、绿色限行政策影响和动态突发事件响应**过程，在保证服务可行性的同时兼顾运营成本和低碳目标，为城市绿色物流配送中的车辆调度与政策响应提供了可行的建模方法和决策参考。

关键词：异构车队路径优化；时变车辆路径问题；软时间窗；绿色限行政策；自适应大邻域搜索；动态重调度

1 问题重述

1.1 问题背景

随着电子商务快速发展与物流信息化水平提升,城市配送系统面临需求动态化、规模扩大化与环保约束强化的多重挑战。在“双碳”目标引导下,绿色物流已成为城市可持续发展的重要抓手。近年来,多地推进绿色配送示范区建设,并对燃油车在核心区域实施分时段限行,这对企业车队调度提出了更高要求:不仅要控制运营成本,还要兼顾服务时效与碳排放表现 [1, 4]。

本文研究对象为某省会城市物流企业的日常配送任务。企业拥有燃油车与新能源车组成的混合车队,需要在时变交通环境下完成多客户、多约束配送。调度决策同时受到车辆容量、客户时间窗、能耗成本、碳排放成本与政策准入规则影响,属于具有现实约束的复杂组合优化问题。

1.2 问题重述

已知数据与成本规则: 配送任务覆盖 98 个客户点,需求由 2169 条订单汇总形成;每个客户给定需求重量、体积与软时间窗。早到产生等待成本 (20 元/小时),晚到产生延误惩罚 (50 元/小时),单客户服务时间取 20 分钟。车辆由燃油车和新能源车构成,统一启动成本为 400 元/辆。考虑时变交通后,车速随时段变化;能耗与速度满足 U 型关系,并进一步换算能源费用与碳排放成本。

问题一 (静态基准调度): 在无政策限制条件下,建立考虑车型差异、载重容积约束、软时间窗与时变速度的调度模型,以总成本最小为目标,输出车辆使用方案、配送路径、到达时刻与成本结构,形成后续对比基准。

问题二 (限行政策影响分析): 在问题一基础上加入绿色配送区限行规则:8:00–16:00 禁止燃油车进入以市中心为圆心、半径 10 km 的绿色配送区。在该政策下重新规划路径,比较不同策略对总成本、车辆结构与碳排放总量的影响。

问题三 (动态事件实时调度): 在配送执行阶段考虑订单取消、新增订单、地址变更、时间窗调整等突发事件,设计可实时响应的重调度策略,并给出典型事件下的路径调整方案与效果评估。

因此,本文总体目标是构建“静态优化–政策响应–动态重调度”的统一方法框架,在满足服务可行性的前提下,实现运营成本、客户时效与低碳目标的协同优化。

2 问题分析

2.1 问题一分析

问题一属于静态环境下的异构车队车辆路径优化问题。其核心难点在于:同一方案需同时满足重量/体积双容量约束、软时间窗约束与时变交通条件;同时总目标函数由固定成本、能源成本、碳排放成本和时间窗惩罚共同构成,存在显著多因素耦合 [5, 3]。若仅追求短距离,可能增加迟到惩罚;若过度压缩时间惩罚,又可能增加车辆启用数与固定成本。因此,问题一的本质是多约束下的综合成本最优平衡。

2.2 问题二分析

问题二在问题一基础上增加绿色配送区限行政策 (8:00–16:00 燃油车禁入),导致可行域收缩并改变路径组织方式。政策作用不仅体现在“是否可达”,还会通过等待、换车或绕行等机制影响时间窗满足率、总里程和能耗结构 [4, 5]。不同策略对应不同代价

转移路径：新能源优先更偏向车辆结构调整，时空错位更依赖服务时序调整，路径绕行则主要影响里程与能耗。因此，问题二的关键是识别并比较三类策略在“成本-低碳-可行性”上的权衡差异。

2.3 问题三分析

问题三由离线静态优化转向在线动态决策。突发事件会打破原始最优路径，若每次全局重算虽可能得到更优目标值，但在实务上会造成指令频繁变化、执行不稳定。故需采用“实时感知 + 局部重调度”思路，在保证响应时效和任务可执行性的前提下，控制新增成本与路径扰动 [6]。该问题的重点不再是绝对全局最优，而是动态环境下的快速可行与稳定优化。

3 模型假设

为简化问题，我们提出以下假设：

假设一 可拆分配送：允许同一客户由多辆车分批服务，以处理单客户需求超过单车容量的情形。

假设二 闭环运营：所有车辆均由同一配送中心出发并最终返回，不考虑中途补货与多仓协同。

假设三 单车单次出车：在单日计划内每辆物理车辆最多执行一条主路径（不重复出车），用于约束调度复杂度并与企业管理规则一致。

假设四 动态局部重调度：动态事件发生后仅调整未冻结路径段，已执行与临近执行任务不回溯修改，以保证现场可执行性。

4 符号说明

表 1: 主要符号说明

符号	含义
$V = \{0\} \cup N$	配送节点集合，其中 0 表示配送中心， N 表示客户集合
R	车辆集合
H	车型集合
d_j	客户 j 的需求重量
g_j	客户 j 的需求体积
Q_r	车辆 r 的最大载重
V_r	车辆 r 的最大容积
x_{ijr}	若车辆 r 从节点 i 行驶至节点 j ，则取 1，否则取 0
y_r	若车辆 r 被启用，则取 1，否则取 0
q_{jr}	车辆 r 向客户 j 配送的重量
v_{jr}	车辆 r 向客户 j 配送的体积
A_{jr}	车辆 r 到达客户 j 的时刻
B_{jr}	车辆 r 在客户 j 开始服务的时刻
w_{jr}	车辆 r 在客户 j 处的等待时间

续表 1

符号	含义
l_{jr}	车辆 r 在客户 j 处的延误时间
t_{ijr}	车辆 r 在弧段 (i, j) 上的行驶时间
e_{ijr}	车辆 r 在弧段 (i, j) 上的能耗量
f_r	车辆 r 的固定启动成本
p_r	车辆 r 的单位能源价格
β_r	车辆 r 单位能耗对应的碳排放系数
c^{carb}	单位碳排放成本
c^{early}	单位提前等待惩罚成本
c^{late}	单位延误惩罚成本
N_g	绿色配送区内客户集合
K_e	新能源车辆集合
K_f	燃油车辆集合
T_{ban}	燃油车在绿色配送区的禁行时段
R_k^{fix}	车辆 k 的冻结路径段
R_k^{free}	车辆 k 的可调整路径段
B	动态任务池
Φ_k	车辆 k 调整前后路径的扰动比例

5 数据预处理和分析

5.1 缺失值处理

首先,对原始数据中的缺失值进行系统处理。统计结果表明,订单信息中存在少量重量与体积缺失的情况,其中共有 14 条订单缺失重量信息,16 条订单缺失体积信息(如第 61 号订单缺失体积、第 143 号订单缺失重量)。为避免直接删除缺失样本对订单结构造成破坏,本文采用分组中位数填补策略:以客户为分组单位,利用同一客户历史订单的中位数对缺失值进行填补。该方法在保证数据完整性的同时,有效降低了异常值对统计结果的干扰,提高了数据的稳健性。

5.2 数据分析

在完成数据预处理后,本文对整理后的数据集进行了整体统计与分析。统计结果表明订单信息共包含订单 2169 条,涉及客户点 88 个。

从需求规模来看,所有客户的总配送重量为 285554.99 kg,总体积为 774.11 m³,其中单个订单最大的重量达到 5987kg,远超单个货车运送极限,必须允许多个车辆服务一个客户,对此我们已经在模型假设中做了说明。

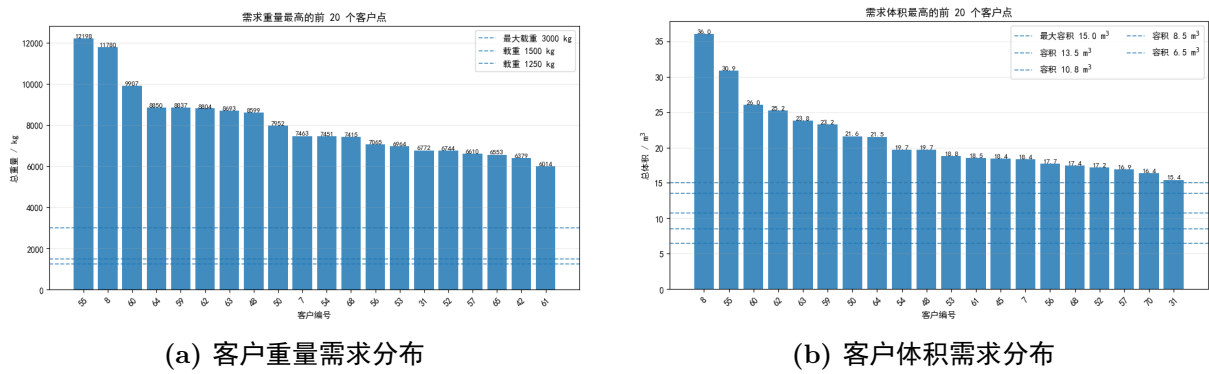


图 1: 客户需求特征对比分析（重量与体积）

在绿色配送约束方面，根据以配送中心为圆心、半径 10 km 划定的绿色配送区域，共识别出 15 个客户点位于该区域内。这部分客户在后续建模中需优先满足绿色配送要求，例如限制燃油车辆进入或优先采用新能源车辆进行配送。

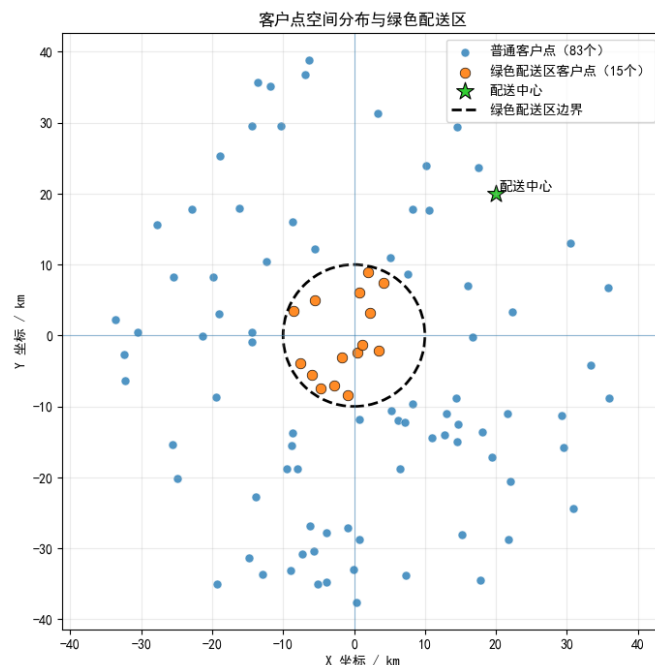


图 2: 客户点空间分布及绿色配送区域示意图

在时间特性方面，如图3所示，每一条水平线表示一个客户的可服务时间区间，由最早服务时间（ET）与最晚服务时间（LT）共同决定。客户时间窗整体分布于 9.15 至 20.97 小时之间，时间跨度较大，表明配送任务具有明显的时间约束特征，需要在路径规划中重点考虑时间窗可行性。

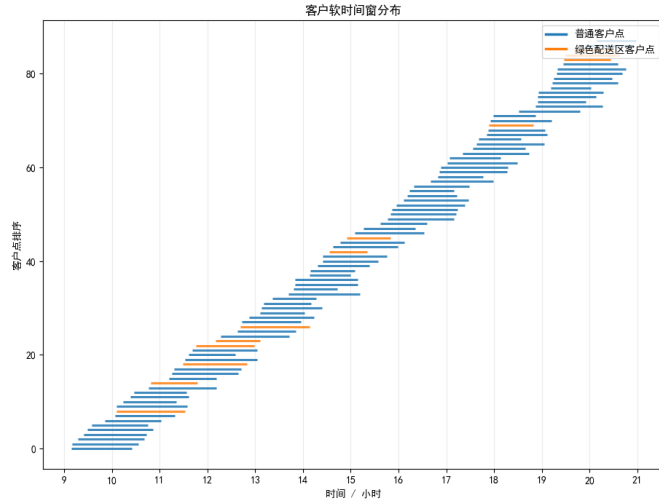


图 3: 客户点空间分布及绿色配送区域示意图

6 问题一模型建立与求解

6.1 模型建立

本题要求在无政策限制下，为 98 个客户点设计最优配送方案。目标是在满足车辆载荷、容积、软时间窗及时变速度约束的前提下，实现总成本最低。公司拥有由 5 类车型组成的异构车队，涵盖燃油与新能源两类动力系统。由于行驶速度随时间段动态波动，且能效受车速与载重率双重影响，该问题属于带软时间窗的异构车队时变车辆路径问题。基于上面的认识，我们从决策变量，优化目标，约束条件，问题求解四个方面对问题一进行了建模。



图 4: 问题一建模流程

6.1.1 决策变量

为了刻画城市物流配送路径优化问题，本文定义如下决策变量：

表 2: 决策变量说明

符号	类型	含义
x_{ijr}	0-1 变量	若车辆 r 从节点 i 行驶至节点 j , 则取 1, 否则为 0
y_r	0-1 变量	若车辆 r 被启用执行配送任务, 则取 1, 否则为 0
q_{jr}	连续变量	车辆 r 在完成客户 j 服务后车内剩余货物重量 (kg)
v_{jr}	连续变量	车辆 r 在完成客户 j 服务后车内剩余货物体积 (m^3)
t_{jr}	连续变量	车辆 r 到达客户节点 j 的时刻
w_{jr}	连续变量	车辆 r 在客户 j 处因提前到达产生的等待时间 (h)
l_{jr}	连续变量	车辆 r 在客户 j 处因延迟服务产生的延误时间 (h)

6.1.2 目标函数（成本构成）

本文以城市物流配送系统的总运营成本最小为优化目标。综合考虑车辆使用成本、能源消耗、碳排放以及时间窗偏差成本等因素，构建如下目标函数：

$$\min Z = Z_1 + Z_2 + Z_3 + Z_4 \quad (1)$$

其中，各部分成本定义如下：

(1) 车辆固定使用成本 车辆一旦被启用，即产生固定运营成本，表示为：

$$Z_1 = \sum_{r \in R} f_r y_r \quad (2)$$

其中， f_r 表示车辆 r 的固定使用成本 400 元， y_r 为车辆启用决策变量。

(2) 能源消耗成本 车辆在运输过程中产生的能源消耗受行驶速度与车辆载重的共同影响。根据相关研究，单位里程能耗与车速之间呈典型的“U 型关系”，即在低速拥堵或高速行驶状态下能耗较高，而在中等速度区间能耗较低 [2, 5]。燃油车与新能源车的百公里能耗函数分别表示为：

$$FPK(v) = 0.0025v^2 - 0.2554v + 31.75 \quad (3)$$

$$EPK(v) = 0.0014v^2 - 0.12v + 36.19 \quad (4)$$

其中， v 表示车辆行驶速度 (km/h)， $FPK(v)$ 与 $EPK(v)$ 分别表示燃油车与新能源车在该速度下的百公里油耗 (L/100km) 与电耗 (kWh/100km)。为统一表示，定义能耗函数：

$$\phi_r(v) = \begin{cases} FPK(v), & r \text{ 为燃油车} \\ EPK(v), & r \text{ 为新能源车} \end{cases} \quad (5)$$

进一步考虑车辆载重对能耗的影响。实际运行中，车辆能耗随载重增加而上升。在保证模型可解性的前提下刻画该影响，本文采用线性插值方法进行近似描述。设车辆 r 在弧段 (i, j) 上的实际载重为 L_{ijr} ，最大载重为 Q_r ，定义载重比例为：

$$\lambda_{ijr} = \frac{L_{ijr}}{Q_r} \quad (6)$$

根据经验数据, 车辆满载时能耗相较空载有所增加, 其中燃油车约增加 40%, 新能源车约增加 35%。因此引入载重修正系数 α_r (燃油车取 0.4, 新能源车取 0.35), 得到考虑载重影响的单位里程能耗函数:

$$\phi_r^*(v, L_{ijr}) = \phi_r(v) (1 + \alpha_r \lambda_{ijr}) \quad (7)$$

在城市交通环境下, 车辆行驶速度具有明显的时变特性 [3]。设一天被划分为若干时间区间 (顺畅——一般——拥堵), 在每个时间段里, 速度呈现正态分布, 根据假设我们直接取均值。可表示为如下分段函数:

$$\bar{v}(t) = \begin{cases} 9.8, & t \in [8:00, 9:00) \cup [11:30, 13:00) \\ 55.3, & t \in [9:00, 10:00) \cup [13:00, 15:00) \\ 35.4, & t \in [10:00, 11:30) \cup [15:00, 17:00) \end{cases} \quad (8)$$

若车辆在段 (i, j) 上的行驶过程跨越多个时间段, 则需分段计算。设车辆在第 m 个时间段内行驶时间为 $\tau_{ijr}^{(m)}$, 对应平均速度为 $\bar{v}_m$, 则有:

$$d_{ij} = \sum_m \bar{v}_m \tau_{ijr}^{(m)} \quad (9)$$

据此, 车辆 r 在弧段 (i, j) 上的总能耗可表示为:

$$e_{ijr} = \sum_m \left[\frac{\bar{v}_m \tau_{ijr}^{(m)}}{100} \cdot \phi_r^*(\bar{v}_m, L_{ijr}) \right] \quad (10)$$

最终, 能源消耗成本定义为:

$$Z_2 = \sum_{r \in R} \sum_{i \in V} \sum_{\substack{j \in V \\ j \neq i}} x_{ijr} e_{ijr} p_r \quad (11)$$

其中, p_r 表示单位能耗价格。具体地, 燃油车燃料价格取 7.61 元/L, 新能源车电价取 1.64 元/度。

(3) 碳排放成本 每百公里碳排放量与车辆能耗之间存在显著的线性关系。具体而言, 燃油消耗与电能消耗可分别通过固定的碳排放转换系数转化为碳排放量, 其中燃油与电力的碳排放系数分别为 $\eta = 2.547 \text{ kg/L}$ 和 $\gamma = 0.501 \text{ kg/(kWh)}$ 。在此基础上, 进一步考虑碳排放所带来的环境成本。车辆在运输过程中产生的碳排放量与其能耗成正比, 因此碳排放成本可表示为:

$$Z_3 = \sum_{r \in R} \sum_{i \in V} \sum_{\substack{j \in V \\ j \neq i}} x_{ijr} e_{ijr} \beta_r c_c \quad (12)$$

其中, e_{ijr} 表示车辆 r 在弧段 (i, j) 上的能耗量, β_r 为单位能耗对应的碳排放系数 (燃油车取 η , 新能源车取 γ), c_c 为单位碳排放成本。

(4) 时间窗惩罚成本 为刻画配送服务对时间窗的满足程度, 引入软时间窗惩罚机制:

$$Z_4 = \sum_{j \in N} \sum_{r \in R} (c_e w_{jr} + c_l l_{jr}) \quad (13)$$

其中, w_{jr} 和 l_{jr} 分别表示车辆 r 在客户 j 处的提前等待时间与延迟服务时间, c_e 与 c_l 分别为单位提前与延误惩罚系数。

6.1.3 约束条件

为保证模型解的可行性与实际意义，需对车辆路径、配送量及时间等方面施加如下约束：

(1) 客户需求满足约束 每个客户节点的重量与体积需求必须被完全满足：

$$\sum_{r \in R} q_{jr} = d_j, \quad \forall j \in N$$

$$\sum_{r \in R} v_{jr} = g_j, \quad \forall j \in N$$

(2) 配送量与访问一致性约束 仅当车辆访问客户节点时，方可向其配送货物：

$$q_{jr} \leq d_j \sum_{i \in V} x_{ijr}, \quad \forall j \in N, r \in R$$

$$v_{jr} \leq g_j \sum_{i \in V} x_{ijr}, \quad \forall j \in N, r \in R$$

(3) 车辆出发与返回约束 被启用的车辆必须从配送中心出发，并最终返回配送中心：

$$\sum_{j \in N} x_{0jr} = y_r, \quad \forall r \in R \quad (14)$$

$$\sum_{i \in N} x_{i0r} = y_r, \quad \forall r \in R \quad (15)$$

(4) 路径连续性约束 车辆在访问任一客户节点后必须离开该节点，以保证路径的连通性：

$$\sum_{\substack{i \in V \\ i \neq j}} x_{ijr} = \sum_{\substack{l \in V \\ l \neq j}} x_{jlr}, \quad \forall j \in N, r \in R \quad (16)$$

(5) 车辆容量约束 每辆车在配送过程中所承担的货物总量不得超过其最大载重与最大容积：

$$\sum_{j \in N} q_{jr} \leq Q_r y_r, \quad \forall r \in R \quad (17)$$

$$\sum_{j \in N} v_{jr} \leq C_r y_r, \quad \forall r \in R \quad (18)$$

(6) 时间窗约束 车辆到达客户节点后，其服务开始时刻应满足时间窗限制。设车辆 r 到达客户 j 的时刻为 A_{jr} ，则：

$$B_{jr} = \max(A_{jr}, ET_j) \quad (19)$$

提前等待时间与延迟服务时间分别为：

$$w_{jr} = \max(0, ET_j - A_{jr}) \quad (20)$$

$$l_{jr} = \max(0, A_{jr} - LT_j) \quad (21)$$

(7) 时间递推约束 若车辆 r 从节点 i 行驶至节点 j , 则有:

$$A_{jr} \geq B_{ir} + s + t_{ijr} - M(1 - x_{ijr}), \quad \forall i, j \in V, r \in R \quad (22)$$

其中, s 表示服务时间, t_{ijr} 为基于时变速度计算得到的行驶时间, M 为足够大的正数。

(8) 车型数量约束 各类型车辆的使用数量不得超过公司现有数量:

$$\sum_{r \in R_h} y_r \leq Num_h, \quad \forall h \in H \quad (23)$$

其中, R_h 表示车型 h 对应的车辆集合。

6.2 问题求解

综上所述, 本文构建的城市绿色物流配送问题可形式化表示为如下优化问题:

$$\begin{aligned} \min \quad & Z = Z_1 + Z_2 + Z_3 + Z_4 \\ \text{s.t.} \quad & \begin{cases} \sum_{r \in R} q_{jr} = d_j, & \forall j \in N \\ \sum_{r \in R} v_{jr} = g_j, & \forall j \in N \\ q_{jr} \leq d_j \sum_{i \in V} x_{ijr}, & \forall j \in N, r \in R \\ v_{jr} \leq g_j \sum_{i \in V} x_{ijr}, & \forall j \in N, r \in R \\ \sum_{j \in N} x_{0jr} = y_r, & \forall r \in R \\ \sum_{i \in N} x_{i0r} = y_r, & \forall r \in R \\ \sum_{i \in V, i \neq j} x_{ijr} = \sum_{l \in V, l \neq j} x_{jlr}, & \forall j \in N, r \in R \\ \sum_{j \in N} q_{jr} \leq Q_r y_r, & \forall r \in R \\ \sum_{j \in N} v_{jr} \leq C_r y_r, & \forall r \in R \\ B_{jr} = \max(A_{jr}, ET_j), & \forall j \in N, r \in R \\ w_{jr} = \max(0, ET_j - A_{jr}), & \forall j \in N, r \in R \\ l_{jr} = \max(0, A_{jr} - LT_j), & \forall j \in N, r \in R \\ A_{jr} \geq B_{ir} + s + t_{ijr} - M(1 - x_{ijr}), & \forall i, j \in V, r \in R \\ \sum_{r \in R_h} y_r \leq Num_h, & \forall h \in H \end{cases} \end{aligned} \quad (24)$$

$$x_{ijr} \in \{0, 1\}, \quad y_r \in \{0, 1\}, \quad q_{jr}, v_{jr}, A_{jr}, w_{jr}, l_{jr} \geq 0 \quad (25)$$

由于该模型同时包含异构车队、软时间窗及非线性能耗等复杂约束，属于典型 NP 难组合优化问题，传统精确算法难以在合理时间内求解。已有研究表明，大邻域搜索及其改进算法适合求解带时间窗的城市配送路径优化问题 [7]。为此，本文采用自适应大邻域搜索（ALNS）算法，通过“破坏—修复”机制与模拟退火策略相结合，实现对解空间的高效全局搜索，从而获得高质量近似最优解。其伪代码如下：

Algorithm 1: 基于 ALNS 的城市物流配送优化算法

Input: 最大迭代次数 $Iter_{max}$, 初始温度 T , 冷却系数 α 等

Output: 最优解 S^*

```

1 初始化参数，构造初始解  $S$ ;
2 计算目标函数值  $Z(S)$ ，设置当前最优解  $S^* \leftarrow S$ ;
3 for  $k = 1$  to  $Iter_{max}$  do
4   执行破坏算子，得到中间解  $S'$ ;
5   执行修复算子，得到新解  $S''$ ;
6   计算新解目标函数值  $Z(S'')$ ;
7   if  $Z(S'') < Z(S)$  then
8     | 接受新解:  $S \leftarrow S''$ ;
9   end
10  else
11    | 按模拟退火准则以概率  $p = \exp\left(-\frac{Z(S'')-Z(S)}{T}\right)$  接受新解;
12  end
13  if  $Z(S) < Z(S^*)$  then
14    | 更新最优解:  $S^* \leftarrow S$ ;
15  end
16  更新温度:  $T \leftarrow \alpha T$ ;
17 end
18 return  $S^*$ 

```

6.3 结果可视化与分析

基于上述模型与 ALNS 启发式算法，本文对无政策限制条件下的城市物流配送调度问题进行求解。在约束下，模型综合考虑车辆容量、客户时间窗、时变速度、能源消耗与碳排放成本，得到问题一的配送方案。为直观展示调度结果，本文分别从车辆使用方案、车辆行驶路径、客户到达时间以及成本构成四个方面进行可视化分析。

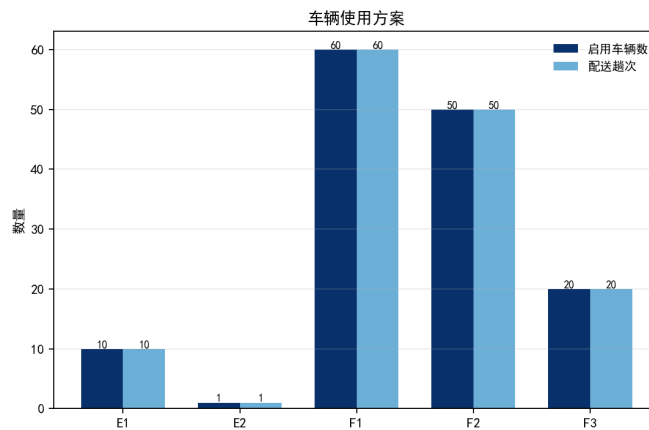


图 5: 问题一车辆使用方案

由图 5 可知，在无政策约束条件下，共启用 123 辆车辆完成配送任务。其中，燃油车 98 辆，占比 79.67%，新能源车 25 辆，占比 20.33%。整体来看，燃油车仍占据主要地位，但新能源车也承担了一定比例的配送任务，说明在当前成本结构与容量约束下，混合车队能够通过车型互补完成配送需求。从车型构成看，F1、F2 分别投入 60 辆和 38 辆，F3 未被启用；新能源车中 E1 与 E2 分别投入 10 辆和 15 辆。该结果表明，在未施加绿色配送约束时，车辆调度主要由运输成本、载重能力及路径效率共同决定。由于燃油车数量充足且车型覆盖不同载重需求，其在任务分配中仍具有优势；新能源车则主要在部分需求规模和时间窗相匹配的路径中发挥补充作用。

从车型结构来看，F1 与 F2 使用频率较高，说明中大型车辆在提升单次装载效率、减少配送趟次方面具有优势；F3 则主要用于匹配小规模或分散需求订单，以提高容量利用率，体现了异构车队在运力层面的互补性。

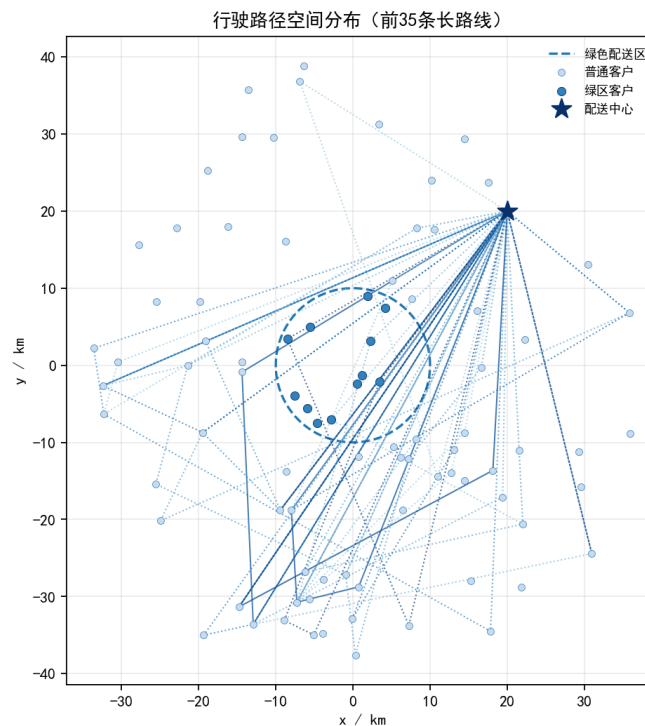


图 6: 问题一车辆行驶路径空间分布

图 6 展示了问题一调度方案中的车辆行驶路径空间分布。图中星形点表示配送中心，圆形虚线表示半径为 10 km 的绿色配送区域，客户点按照是否位于绿色配送区内进行区分。由于问题一不考虑绿色配送区限行政策，燃油车和新能源车均可自由进入绿色配送区，因此路径组织主要受客户空间分布、车辆容量以及时间窗约束影响。

从整体路径形态来看，车辆配送路径以配送中心为起点，向城市不同方向辐射展开。中心区域客户较为密集，车辆路径在该区域存在一定重叠；外围客户分布相对分散，车辆需要承担较长的往返距离。该结果说明，客户空间分布的不均衡性是影响配送总里程和能源消耗的重要因素。对于距离配送中心较远的客户，单次服务往往会带来较高的行驶成本，因此模型倾向于将空间位置相近、时间窗相容的客户合并到同一路径中，以降低总行驶距离。

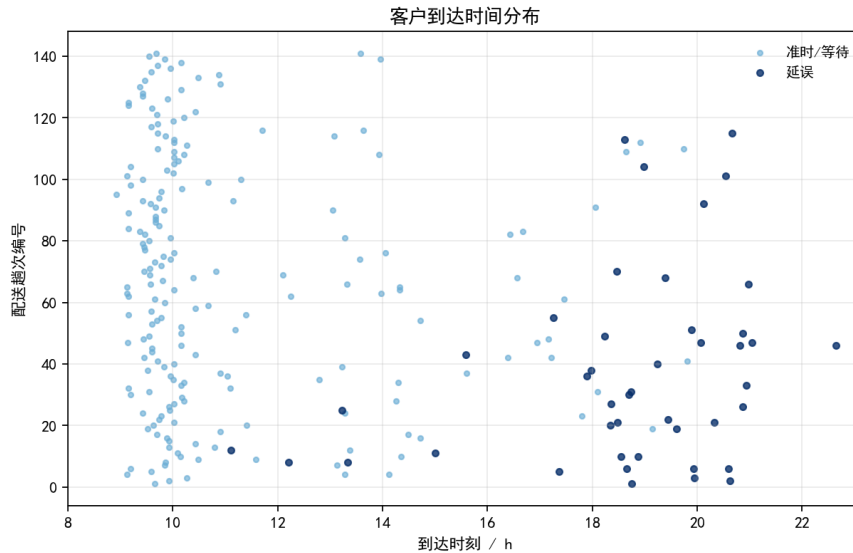


图 7: 问题一客户到达时间分布

图 7 给出了各车辆路径中客户节点的到达时间分布。可以看出，大部分客户到达时间集中在 9:00 至 18:00 之间，与客户时间窗分布基本一致。由于题目设置了软时间窗约束，车辆早到会产生等待成本，晚到会产生惩罚成本，因此模型在路径排序时需要兼顾空间距离与服务时序。

从图中可以观察到，部分客户存在一定程度的延误。这主要由两方面原因造成：第一，部分客户时间窗较短，且空间位置相对分散，难以在较短时间内完成连续服务；第二，车辆行驶速度具有明显时变特征，在拥堵时段车辆行驶时间显著增加。



图 8: 问题一总成本构成

图 8 展示了问题一最优调度方案的成本构成。结果表明，系统总成本为 105212.62 元，其中车辆固定启动成本为 49200.00 元，能源消耗成本为 33829.14 元，碳排放成本为 7317.21 元，时间窗惩罚成本为 14866.26 元。

从成本占比来看，固定启动成本仍是总成本中占比最高的部分，占总成本的 46.76%。这是因为本文严格限定每辆车最多执行一条路径，且客户总需求量较大，部分客户需求还需要拆分配送，因此需要启用较多车辆完成全部配送任务。能源消耗成本占总成本的 32.15%，是第二大成本来源，说明车辆行驶里程、时变速度和载重率对运营成本具有显著影响。

碳排放成本为 7317.21 元，占总成本的 6.96%。虽然其占比低于固定成本和能源成本，但该部分成本已经体现出碳排放因素对调度方案的约束作用。本文根据题目给定的

碳排放转换系数和碳排放成本进行核算，总碳排放量为 11257.25 kg，其中燃油车辆仍是主要碳排放来源。这也说明，在无政策限制条件下，模型更倾向于使用数量充足、容量适配性较强的燃油车辆，从而导致碳排放总量仍处于较高水平。

表 3: 问题一调度结果汇总

指标	数值
总成本 / 元	105212.62
车辆固定启动成本 / 元	49200.00
能源消耗成本 / 元	33829.14
碳排放成本 / 元	7317.21
时间窗惩罚成本 / 元	14866.26
总行驶里程 / km	16814.13
总碳排放量 / kg	11257.25
启用车辆数 / 辆	123
燃油车辆数 / 辆	98
新能源车辆数 / 辆	25
新能源车辆使用率	20.33%

综合分析可知，在无政策约束条件下，模型能够在容量、时间窗及时变速度等约束下完成全部配送任务，并通过路径优化与车型匹配有效控制车辆使用数量及固定成本；燃油车占比为 79.67%，新能源车占比为 20.33%，系统总碳排放量为 11257.25 kg，对应碳成本为 7317.21 元。该结果可作为基准方案，用于后续评估绿色配送政策对成本结构、车辆配置及碳排放的影响。

7 问题二模型建立与求解

7.1 问题分析

在问题一中，本文针对无政策限制情形，构建了考虑异构车队、软时间窗及时变速度的城市配送优化模型。然而在实际城市管理中，绿色配送区往往对燃油车辆实施通行限制，要求物流企业在满足客户服务需求的同时，兼顾低碳排放与政策合规性。混合车队和区域准入约束会共同改变车辆可服务集合与路径组织方式 [5, 4]，因此，问题二需要在问题一模型基础上，进一步引入绿色配送区准入约束，研究政策限制下的混合车队调度优化方案。

绿色配送区政策的实施，改变了车辆的可达性与路径。一方面，新能源车辆没有限行约束，可直接承担绿色配送区内客户的配送任务；另一方面，燃油车辆受到时段限制，可能需要等待限行结束后进入，或通过绕行方式规避绿色配送区。这使得问题二不仅需要传统的成本、时间窗与容量约束，还需同时刻画政策作用机制。

为系统分析绿色配送区政策对配送方案的影响，本文在问题一统一模型的基础上，构建三种不同的政策响应策略，分别为新能源车辆绝对优先策略、时空错位调度策略和路径绕行策略。具体建模思路如下：

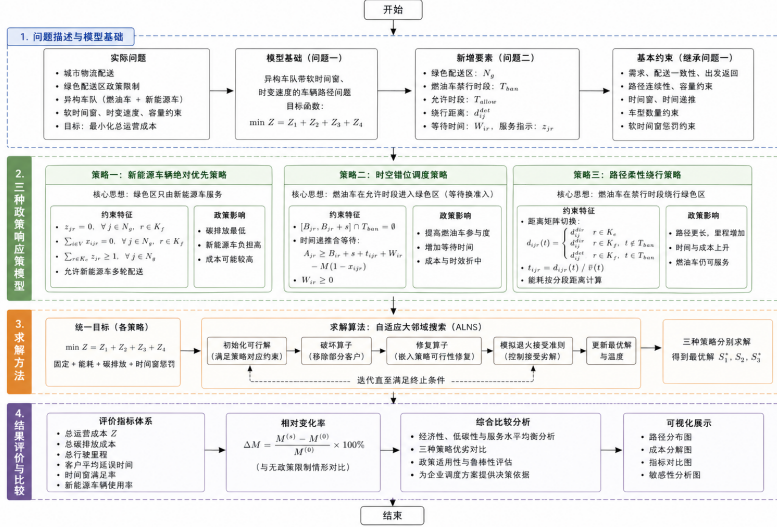


图 9: 问题二建模流程

7.2 模型建立

为便于后续约束表达，引入客户服务指示变量：

$$z_{jr} = \sum_{i \in V} x_{ijr}, \quad \forall j \in N, r \in R \quad (26)$$

即当车辆 r 到达客户节点 j 时，认为其承担了该客户的配送任务。
即当车辆 r 到达客户节点 j 时，认为其承担了该客户的配送任务。

7.2.1 目标函数与基本约束条件

问题二仍以系统总运营成本最小为目标，包括车辆固定使用成本、能源消耗成本、碳排放成本与时间窗惩罚成本，即：

$$\min Z = Z_1 + Z_2 + Z_3 + Z_4 \quad (27)$$

与问题一不同之处在于，问题二中车辆的可达路径、到达时刻及行驶距离将受到绿色配送区准入政策的影响，因此不同策略下的 e_{ijr} 与 t_{ijr} 将随政策规则发生变化。

7.2.2 基本约束条件

问题二继承问题一中的客户需求满足约束、配送一致性约束、车辆出发与返回约束、路径连续性约束、容量约束、时间窗约束、时间递推约束以及车型数量约束。在此基础上，进一步叠加绿色配送区政策约束，分别构建三类策略模型。

7.2.3 策略一：新能源车辆绝对优先策略

该策略将绿色配送区视为新能源车辆优先服务区域，燃油车辆原则上不得承担绿色配送区内客户的配送任务。其核心思想是优先利用新能源车辆完成绿色配送区内需求，以实现低碳效益最大化。

由于问题一中已允许客户需求拆分配送，因此在问题二中不再限制“某客户只能被一辆车访问”，而是限制绿色配送区客户的配送量只能由新能源车辆承担。于是有：

$$\sum_{r \in K_f} q_{jr} = 0, \quad \forall j \in N_g \quad (28)$$

$$\sum_{r \in K_f} v_{jr} = 0, \quad \forall j \in N_g \quad (29)$$

即绿色配送区内客户的重量需求与体积需求均不能由燃油车辆完成，只能由新能源车辆承担。与此同时，问题一中的客户需求满足约束仍然成立，因此绿色配送区客户需求最终由新能源车辆完全覆盖。

考虑到新能源车辆数量有限，若单次装载无法完成全部配送任务，则允许新能源车辆返回配送中心补货后继续执行后续任务，即采用多轮配送机制。该策略在约束层面最为严格，其求解结果可作为绿色配送政策下的环境效益标杆。

7.2.4 策略二：时空错位调度策略

该策略考虑燃油车辆并非完全禁止进入绿色配送区，而是在特定限行时段内禁止通行。通过调整客户服务顺序与车辆到达时刻，可使燃油车辆在允许时段进入绿色配送区，从而提高车队整体调度灵活性。

设燃油车辆允许进入绿色配送区的时间集合为：

$$T_{allow} = [0, 24] \setminus T_{ban} \quad (30)$$

若燃油车辆 $r \in K_f$ 服务绿色配送区客户 $j \in N_g$ ，则其服务开始时刻必须位于允许进入区间内，即满足：

$$[B_{jr}, B_{jr} + s] \cap T_{ban} = \emptyset, \quad \forall j \in N_g, r \in K_f \quad (31)$$

为实现燃油车辆在绿色配送区边界等待后再进入，引入等待变量 W_{ir} ，并将时间递推关系修正为：

$$A_{jr} \geq B_{ir} + s + t_{ijr} + W_{ir} - M(1 - x_{ijr}), \quad \forall i, j \in V, r \in R \quad (32)$$

$$W_{ir} \geq 0, \quad \forall i \in V, r \in R \quad (33)$$

对于新能源车辆，由于不受绿色配送区限行影响，其时间递推方式与问题一相同。

在实际求解中，为便于算法判定可行性，可将燃油车辆进入绿色配送区的允许时段离散为若干可服务区间；若燃油车辆到达时刻落入禁行区间，则通过等待修复机制将其顺延至最近的允许进入时刻。该策略的本质是以等待时间换取准入可行性，从而在政策约束下兼顾运营成本与服务时效。

7.2.5 策略三：路径柔性绕行策略

该策略认为绿色配送政策不仅影响车辆是否可以进入绿色配送区，也可能改变燃油车辆的路径组织方式。对于燃油车辆，若某一弧段的直接最短路径在禁行时段穿越绿色配送区，则需改为绕行路径，从而避免违规通行。

为此，本文预先构建两类距离矩阵：其一为所有节点间的直接最短距离矩阵 d_{ij}^{dir} ；其二为燃油车辆在限行时段绕开绿色配送区后的绕行距离矩阵 d_{ij}^{det} 。进一步定义弧集合 A_g 表示“直接路径穿越绿色配送区的弧段集合”。于是，车辆 r 在弧段 (i, j) 上的实际行驶距离可表示为：

$$d_{ijr}(t) = \begin{cases} d_{ij}^{dir}, & r \in K_e \\ d_{ij}^{dir}, & r \in K_f, t \notin T_{ban} \\ d_{ij}^{det}, & r \in K_f, t \in T_{ban}, (i, j) \in A_g \\ d_{ij}^{dir}, & r \in K_f, t \in T_{ban}, (i, j) \notin A_g \end{cases} \quad (34)$$

相应地，车辆在弧段 (i, j) 上的行驶时间表示为：

$$t_{ijr} = \frac{d_{ijr}(t)}{\bar{v}(t)} \quad (35)$$

若弧段行驶过程跨越多个时段，则仍采用问题一中的分段速度计算方法。于是，在该策略下，车辆能耗修正为：

$$e_{ijr} = \sum_m \left[\frac{d_{ijr}^{(m)}}{100} \cdot \phi_r^*(\bar{v}_m, L_{ijr}) \right] \quad (36)$$

其中， $d_{ijr}^{(m)}$ 表示车辆 r 在第 m 个时间段内于弧段 (i, j) 上的实际行驶距离。

该策略并不直接限制燃油车辆服务哪些客户，而是通过改变禁行时段下的可通行路径来间接反映绿色配送区政策的影响，因此更能体现现实路网环境中的柔性交通组织特征。

7.3 问题求解

综上所述，问题二可视为在问题一统一优化框架下，叠加不同绿色配送政策约束后的多场景异构车队路径优化问题。三种策略的求解目标均为：

$$\min Z = Z_1 + Z_2 + Z_3 + Z_4 \quad (37)$$

但其差异主要体现在绿色配送区客户的服务准入规则、燃油车辆服务时序安排以及路径选择机制上。

和第一问类似，第二问仍采用自适应大邻域搜索（ALNS）算法进行求解，并针对不同策略设计相应的可行性修复机制：

对于策略一，在修复阶段优先将绿色配送区客户插入新能源车辆路径，若燃油车辆被分配到绿区客户，则判为不可行并重新分配；

对于策略二，若燃油车辆到达绿色配送区客户时刻处于禁行区间，则触发等待修复，将服务开始时刻顺延至最近的允许进入时刻；

对于策略三，在路径评估阶段根据车辆类型、出发时刻及弧段属性动态调用直接距离矩阵或绕行距离矩阵，并据此重算行驶时间、能耗及目标函数值。

通过上述改进，ALNS 算法能够在统一求解框架下有效处理三类绿色配送策略，并实现不同政策场景下配送方案的比较分析。

7.4 结果可视化与分析

为分析绿色配送区限行政策对城市物流配送系统的影响，本文以问题一无政策限制下的调度结果作为基准方案，并分别求解新能源优先、时空错位调度和路径柔性绕行三种政策响应策略。三种策略均满足燃油车在 8:00–16:00 禁止进入绿色配送区的政策要求，并在此基础上比较政策对总成本、车辆使用结构、碳排放总量及配送路径组织方式的影响。

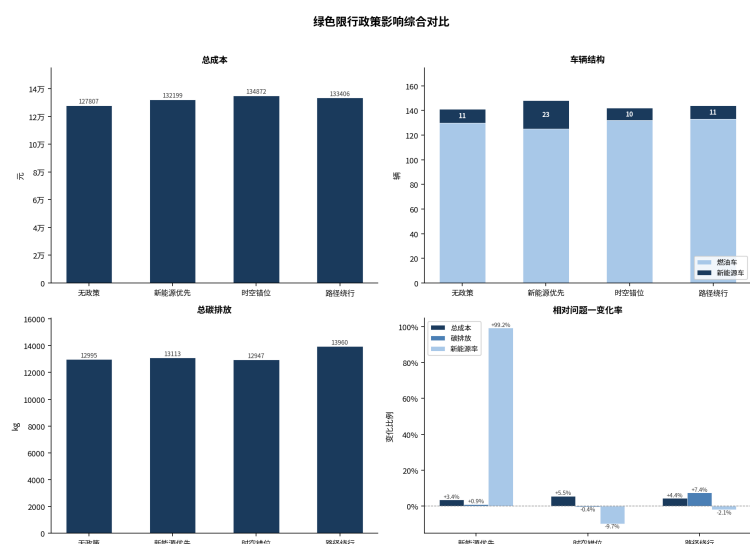


图 10: 绿色限行政策影响综合对比

由图 10 可见，限行政策实施后三种策略的总成本均高于问题一基准方案，说明政策约束压缩了原有路径组织自由度，系统需通过车辆替代、时序调整或空间绕行来换取可行性。分策略比较，新能源优先策略成本增幅最小（+25.65%），路径绕行次之（+26.80%），时空错位最大（+28.19%），表明其成本控制能力依次递减。对车辆结构与碳排放信息而言，新能源优先策略在三类政策方案中新能源车辆使用率最高，为 15.54%；时空错位策略虽然新能源车辆使用率较低，但通过服务时序调整降低了部分高能耗行驶影响，碳排放量最低，为 12947.05 kg；路径绕行策略因额外里程增加，碳排放量最高。

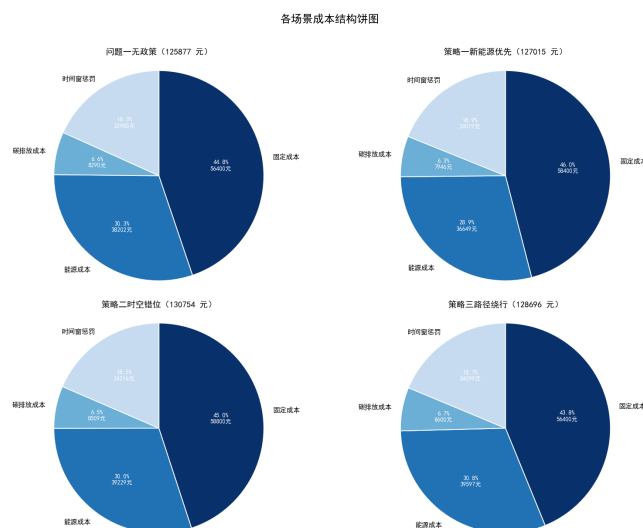


图 11: 绿色限行政策下成本构成对比

图 11 进一步揭示了“成本增加从何而来”。四类成本中，基准方案仍以固定成本为主；政策介入后，不同策略对应不同成本变化方式：新能源优先主要增加车辆启用相关成本，时空错位主要抬升时间窗惩罚，路径绕行主要推高能源与碳排放成本。换言之，三种策略均可满足限行约束，但其代价来源存在差异。

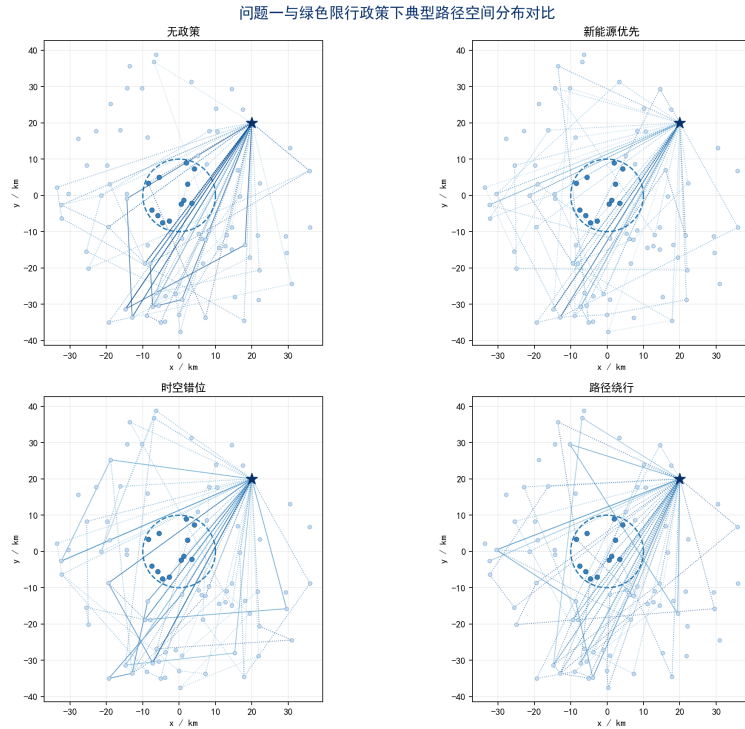


图 12: 问题一与绿色限行政策下典型路径空间分布对比

图 12 从空间层面解释了上述差异：新能源优先策略通过车辆替代完成绿区服务重分配；时空错位策略路径形态与基准较接近，但必须调整时间来满足准入；路径绕行策略则出现明显绕行与路径增加。

表 4: 绿色限行政策影响对比

方案	总成本/元	启用车辆数/辆	新能源车辆使用率	总碳排放量/kg
问题一基准	105212.62	123	20.33%	11257.25
新能源优先	132199.33	148	15.54%	13113.30
时空错位	134872.27	142	7.04%	12947.05
路径绕行	133406.09	144	7.64%	13959.70

表 4 给出了最终量化结果。综合成本、车辆结构与碳排放三方面指标，新能源优先策略的总成本最低，且在三类政策方案中新能源车辆使用率最高；时空错位策略虽然成本代价最高，但碳排放量最低；路径绕行策略虽满足约束，但因额外里程导致能耗与碳排放上升，综合表现相对较弱。

据此可直接回答题目关于绿色限行政策影响的问题：**(1) 对总成本的影响：**限行后总成本整体上升，三种策略较基准方案分别增加 25.65%、28.19%、26.80%；**(2) 对车辆使用结构的影响：**政策响应改变了不同动力车辆的配置结构，新能源优先策略在三类政策方案中新能源车辆使用率最高，为 15.54%；**(3) 对碳排放总量的影响：**不同策略差异显著，其中时空错位策略碳排放最低（12947.05 kg），路径绕行因额外里程导致碳排放最高（13959.70 kg）。

8 问题三模型建立与求解

8.1 问题分析

前两问研究的是静态环境下的车辆路径优化问题，即订单需求、客户地址、时间窗和车辆状态在配送开始前均已确定。然而在实际城市配送过程中，订单取消、新增订单、客户地址变更以及时间窗调整等突发事件会不断发生，这会直接打乱原有配送计划，使得静态优化得到的路径方案不再完全适用。因此，问题三的核心不再是一次性求得全局最优路径，而是在配送过程中面对突发扰动时，如何快速形成一套可执行、稳定且成本较低的动态调度方案。

结合前两问的研究基础，本文将问题三建模为事件驱动下的动态车辆路径调整问题。考虑到动态调度需要兼顾实时性与稳定性，本文不采用突发事件一发生就全局重算的方式，而是采用“实时感知-局部冻结-任务更新-局部插入”的动态调度策略 [6]。该策略的基本思想是：当突发事件出现时，保留已经执行或短时间内难以改变的配送任务，仅对受事件影响的任务以及尚未冻结的路径进行局部重优化，从而在保证响应速度的同时，尽量减少对原有方案的扰动。

8.2 系统状态描述

设突发事件发生时刻为 t 。此时，每辆车的运行状态不再仅仅由一条既定路径决定，而是由当前位置、当前时间、已锁定任务和可调整任务共同决定。为此，记车辆 k 在时刻 t 的状态为

$$S_k(t) = (p_k, \tau_k, R_k^{fix}, R_k^{free}, a_k),$$

其中， p_k 表示车辆当前位置， τ_k 表示车辆当前可再次参与调度决策的时间， R_k^{fix} 表示冻结路径段， R_k^{free} 表示可调整路径段， a_k 表示车辆是否仍具备承接新增任务的能力。

与此同时，配送任务也会随着事件发生而改变状态。本文将任务状态划分为六类：

$$\{\text{served, frozen, open, cancelled, conflict, new}\}.$$

其中，served 表示任务已完成服务，frozen 表示任务已进入短期内不可调整范围，open 表示任务尚未服务且可重新安排，cancelled 表示订单已取消，conflict 表示原路径因地址变更或时间窗调整而失去可行性，new 表示新增订单。

需要说明的是，本文仍采用单车单次配送假设。也就是说，车辆一旦离开配送中心执行任务，就不再返回补货后重新派出。因此，对于配送过程中出现的新增订单，已经在途且无法补货的车辆不能直接承接，该类任务应优先交由尚未出发车辆或备用车辆处理。这样处理既符合实际配送流程，也能保证动态调度方案具有现实可执行性。

8.3 突发事件处理机制

设动态事件集合为 $E(t)$ 。为便于统一处理不同类型的扰动，本文将每个事件简记为

$$e = (\text{type}, o, \Delta, t, \rho),$$

其中， type 表示事件类型， o 表示受影响任务， Δ 表示事件带来的参数变化， ρ 表示事件优先级。

本文主要考虑以下四类典型突发事件：

订单取消：若订单对应任务尚未完成服务，则将其从原路径中删除，并释放相应车辆的载重和容积资源；

新增订单：将新增任务作为新的动态任务加入任务池，等待重新分配；

地址变更：将原地址任务从当前路径中剔除，并将变更后的新地址视为新的动态任务重新加入任务池；

时间窗调整：更新任务时间窗，若原路径不再满足新的时间要求，则将该任务移入任务池重新分配。

为方便程序统一处理，本文将新增订单 D1 记为动态虚拟节点 99，将地址变更后的新位置记为动态虚拟节点 100。需要强调的是，虚拟节点仅用于第三问动态重优化阶段，并不改变原始客户集合和编号。

8.4 局部冻结机制

动态调度并不意味着所有车辆路径都可以任意修改。实际上，对于已经完成服务的客户点，以及车辆短时间内即将到达的客户，应当保持原调度方案不变。为此，本文设置冻结时间窗 Δ_f 。若某客户点的预计到达时间满足

$$A_j \leq t + \Delta_f,$$

则认为该客户点已经进入执行锁定区间，不再参与后续调整。

在此基础上，车辆路径可以划分为两部分：

$$R_k = R_k^{fix} \cup R_k^{free},$$

其中， R_k^{fix} 表示冻结路径段，必须保持不变； R_k^{free} 表示可调整路径段，可根据突发事件重新排序或重新分配。

引入局部冻结机制的目的在于保证动态调度的可执行性。若对所有路径段都允许自由调整，虽然理论上可能获得更低的成本，但在现实中往往会造成车辆频繁变更目标客户、调度指令难以及时下达等问题。通过冻结即将执行的任务，可以在动态调整能力与实际操作稳定性之间取得较好的平衡。

8.5 动态任务池构建

当突发事件发生后，所有需要重新决策的任务被统一放入动态任务池。本文定义动态任务池为

$$B = O^{new} \cup O^{conflict},$$

其中， O^{new} 表示新增任务集合， $O^{conflict}$ 表示由于地址变更或时间窗调整而与原路径发生冲突的任务集合。

对于任务池中的每个任务 o ，只有满足以下条件时，才允许将其插入某辆车的可调整路径：

1. 插入后车辆总载重与总体积不超过容量上限；
2. 插入后能够满足时间窗约束，或在允许的软时间窗惩罚范围内完成服务；
3. 若任务位于绿色配送区，则插入车辆及其服务时刻必须满足限行要求；
4. 对新增订单而言，插入车辆必须具有承接新增任务的资格。

为方便描述，记任务 o 对车辆 k 的可行性判定为

$$Fea(o, k) = \begin{cases} 1, & \text{若任务 } o \text{ 可由车辆 } k \text{ 服务,} \\ 0, & \text{否则.} \end{cases}$$

8.6 动态优化目标函数

动态调度不仅要求调整后的运营成本尽可能低，还需要尽量保持原方案稳定，并保证突发任务能够被及时响应。因此，本文构建如下动态总目标函数：

$$\min Z_{dyn} = Z_{op} + Z_{dev} + Z_{unserved}.$$

其中， Z_{op} 表示调整后方案的运营成本，包括固定成本、能源成本、碳排放成本和时间窗惩罚成本，即

$$Z_{op} = Z_{fixed} + Z_{energy} + Z_{carbon} + Z_{time}.$$

在本文中，车辆固定成本直接由动态调整后实际启用车辆数决定，可表示为

$$Z_{fixed} = 400N_{use},$$

其中 N_{use} 为启用车辆数。

能源成本和碳排放成本分别沿用前两问的建模方式：

$$Z_{energy} = \sum_k \sum_{(i,j) \in R_k} e_{ijk} p_k,$$

$$Z_{carbon} = 0.65 \sum_k \sum_{(i,j) \in R_k} e_{ijk} \beta_k.$$

其中， e_{ijk} 表示车辆 k 在弧段 (i, j) 上的能耗， p_k 表示单位能源价格， β_k 表示对应能源的碳排放系数。

时间窗惩罚成本仍按题目给定标准计算：

$$Z_{time} = \sum_j [20 \max(0, ET_j - A_j) + 50 \max(0, A_j - LT_j)].$$

为了衡量新方案相对于原方案的改动程度，本文进一步引入路径扰动惩罚。设车辆 k 调整前后的客户序列分别为 R_k^{old} 和 R_k^{new} ，则其路径扰动比例定义为

$$\Phi_k = 1 - \frac{|R_k^{old} \cap R_k^{new}|}{\max\{|R_k^{old}|, |R_k^{new}|, 1\}}.$$

于是，总扰动惩罚可写为

$$Z_{dev} = \eta \sum_k \Phi_k,$$

其中 η 为扰动惩罚系数。 η 越大，说明模型越倾向于保持原路径结构不变。

若动态任务无法被成功分配，则系统需要承担较大的服务损失，因此设置未响应任务惩罚：

$$Z_{unserved} = \sum_{o \in B} M_o (1 - z_o),$$

其中， $z_o = 1$ 表示任务 o 被成功响应， $z_o = 0$ 表示任务未被分配， M_o 为未响应惩罚系数。

综上，动态调度问题最终转化为：在保证突发任务能够有效处理的前提下，尽量降低运营成本、路径扰动和未服务损失。

8.7 约束条件

在动态调整过程中，除了第一问的基本约束，还需要满足额外约束：

(1) **冻结路径保持约束** 已经完成服务或进入冻结时间窗的路径段保持不变，即冻结路径在调整前后完全一致。

(2) **新增订单装载约束** 在单车单次配送假设下，新增订单不能由已经离开配送中心的车辆直接承接，应优先分配给尚未出发车辆或备用车辆。

8.8 局部插入优化算法

对于任务池中的每个动态任务，本文采用最小边际成本插入策略。设车辆当前路径中存在相邻节点 (i, j) ，若将任务 o 插入二者之间，则路径长度变化为

$$\Delta d = d_{io} + d_{oj} - d_{ij}.$$

进一步考虑能耗、碳排放和时间窗惩罚后，该插入操作的边际成本可表示为

$$\Delta C = \Delta C_{energy} + \Delta C_{carbon} + \Delta C_{time} + \Delta C_{dev}.$$

其中， ΔC_{energy} 表示额外能耗成本， ΔC_{carbon} 表示额外碳排放成本， ΔC_{time} 表示时间窗惩罚变化， ΔC_{dev} 表示对原路径结构造成的扰动成本。

对所有可行车辆及其插入位置进行枚举后，选择边际成本最小的方案执行插入。若所有现有路径均不可行，则尝试启用备用车辆。这样既能够保证突发任务被及时响应，又可以尽量减少对原有调度方案的破坏。

Algorithm 2: 事件驱动局部插入动态调度算法

Input: 当前配送方案 $S(t)$ ，突发事件集合 $E(t)$

Output: 调整后配送方案 $S^*(t)$

```

1 实时感知突发事件  $E(t)$ ;
2 更新车辆状态和任务状态;
3 根据冻结时间窗确定冻结路径与可调整路径;
4 根据事件类型构建动态任务池  $B$ ;
5 for 每个动态任务  $o \in B$  do
6   for 每辆可用车辆  $k$  do
7     枚举车辆  $k$  可调整路径中的插入位置;
8     计算插入后的边际成本  $\Delta C$ ;
9     检查容量、时间窗、装载与绿色限行约束;
10  end
11 选择可行边际成本最小的位置;
12 if 存在可行插入位置 then
13   将任务  $o$  插入对应车辆路径中;
14 end
15 else
16   尝试启用备用车辆服务任务  $o$ ;
17 end
18 end
19 重新计算受影响路径的到达时间、能耗、碳排放与成本;
20 输出动态调整方案  $S^*(t)$ ;
21 return  $S^*(t)$ 

```

8.9 突发事件下的调度策略

为验证模型对不同突发事件的响应能力，本文在 $t = 12:00$ 时刻设置四类突发事件：客户 49 订单取消、新增订单 D1、客户 55 地址变更以及客户 27 时间窗提前。事件感知过程如图 13 所示。

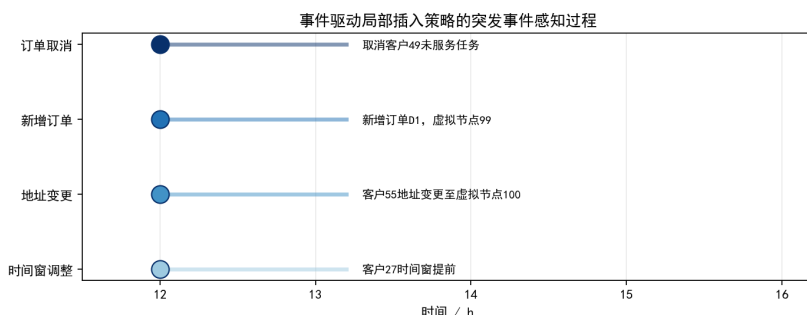


图 13: 突发事件实时感知时间线

针对上述事件，本文的处理策略如下：

- 对客户 49 订单取消事件，若该任务尚未服务，则直接从原路径中删除，并释放车辆资源；
- 对新增订单 D1，将其设置为虚拟节点 99，并优先分配给尚未出发车辆或备用车辆；
- 对客户 55 地址变更事件，删除原地址任务，将新地址设置为虚拟节点 100，并重新进行可行性检查；
- 对客户 27 时间窗提前事件，若原路径无法准时服务，则将其移入动态任务池，重新插入到可行路径中。

突发事件下的车辆调度策略

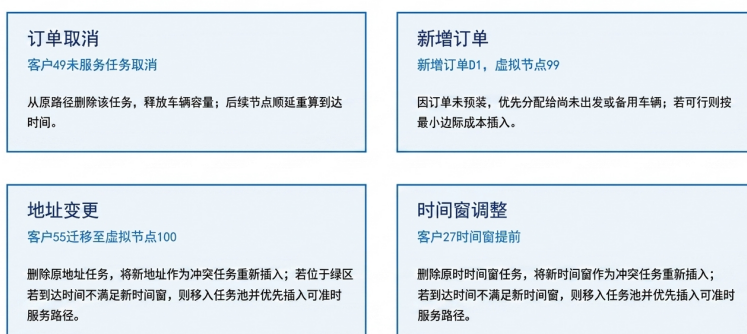


图 14: 突发事件下的车辆调度策略

8.10 动态调度结果分析

采用事件驱动局部插入算法后，系统对四类突发事件进行了局部路径调整。为突出路径变化，图 15 仅展示发生调整的重点车辆路径，其中灰蓝虚线表示调整前路径，深

蓝实线表示调整后路径，橙色节点表示新增或重新插入节点，红色叉号表示被删除或迁出的节点。

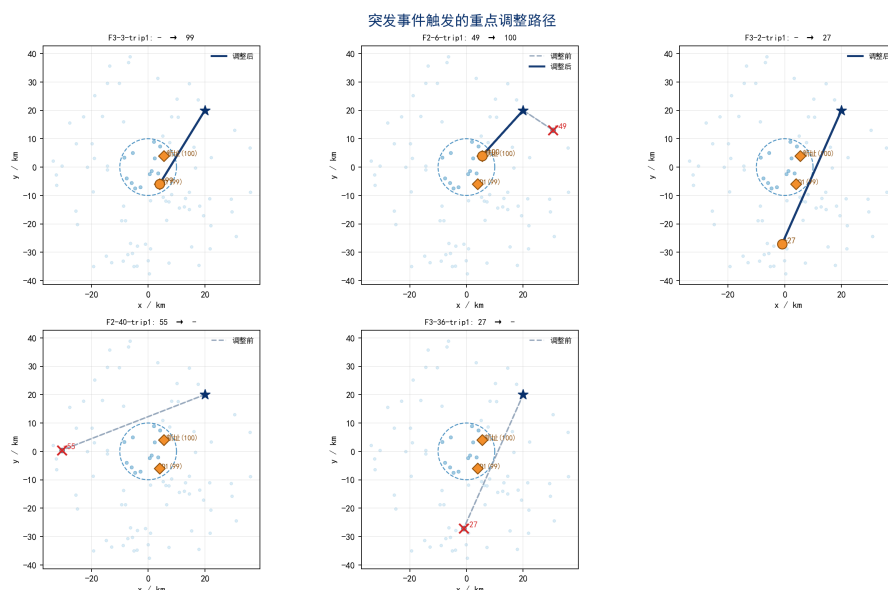


图 15: 突发事件触发的重点调整路径

由图 15 可以看出，客户 49 订单取消后，其原服务路径被删除并释放车辆资源；客户 55 地址变更后，原路径中的客户 55 被移除，变更后的新地址虚拟节点 100 被插入到车辆 F2-6 的路径中；客户 27 因时间窗提前，从原车辆 F3-36 的路径中移出，并重新分配给车辆 F3-2；新增订单 D1 作为虚拟节点 99，由车辆 F3-3 承接。该结果说明，本文模型能够根据不同事件类型触发有针对性的局部调整，而无需对全部路径进行全局重构。

调整后的关键路径变化如表 5 所示。

表 5: 突发事件触发的重点路径调整

车辆	调整前路径	调整后路径	调整原因
F2-40	55	—	客户 55 地址变更，原地址任务删除
F2-6	49	100	客户 49 取消，车辆改为服务变更地址 100
F3-2	—	27	客户 27 时间窗提前后重新分配
F3-3	—	99	新增订单 D1 由备用车辆承接
F3-36	27	—	客户 27 从原路径中移出

进一步地，本文对动态调度方案的整体指标进行汇总，结果如表 6 所示。

表 6: 问题三动态调度结果

指标	数值
动态总成本 / 元	134851.13
未响应任务数	0
任务响应率	100%
总碳排放量 / kg	12920.25
启用车辆数 / 辆	142
路径扰动比例	1.26%

由表 6 可知, 动态调整后所有突发任务均得到响应, 未响应任务数为 0, 任务响应率达到 100%。动态总成本为 134851.13 元, 总碳排放量为 12920.25 kg, 启用车辆数为 142 辆。路径扰动比例仅为 1.26%, 说明局部插入策略能够在较小调整范围内完成动态响应。

综合来看, 本文构建的事件驱动局部插入动态调度策略能够有效处理订单取消、新增订单、地址变更和时间窗调整等多类突发事件。该策略通过局部冻结机制保持原方案主体稳定, 并通过边际成本插入机制快速完成受影响任务的重新分配, 能够在响应速度、配送成本和执行稳定性之间取得较好平衡。

9 灵敏度分析

在问题一的静态调度模型中, 车辆行驶速度是影响行驶时间、能源消耗和时间窗惩罚的重要参数。题目给出的顺畅、一般和拥堵三类交通状态下的车速分别服从如下正态分布:

$$v_{\text{顺畅}} \sim N(55.3, 0.1^2), \quad v_{\text{一般}} \sim N(35.4, 5.2^2), \quad v_{\text{拥堵}} \sim N(9.8, 4.7^2).$$

在前文模型求解过程中, 为降低随机因素带来的求解复杂度, 本文采用各时段车速分布的均值作为确定性输入。该处理方式能够简化模型计算, 但也可能忽略实际交通环境中车速波动对调度结果的影响。因此, 有必要对车速参数进行灵敏度分析, 以检验基准调度方案在随机交通扰动下的稳定性。

本文以问题一得到的基准配送方案为分析对象, 在保持车辆分配、客户访问顺序和配送量不变的条件下, 对所有时段的车速均值进行等比例扰动。扰动范围设定为 -20% 至 20% , 步长为 5% 。对于每一种扰动情形, 重新计算配送过程中的行驶时间、能源消耗、碳排放量、时间窗惩罚成本以及总成本。该方法能够在不改变路径结构的前提下, 单独考察车速变化对方案执行效果的影响。

图 16 给出了不同车速扰动水平下总成本相对于基准方案的变化率。由图可知, 总成本对车速扰动整体不敏感。当车速整体降低 20% 时, 总成本为 105118.21 元, 相比基准方案下降 0.09% ; 当车速整体提高 20% 时, 总成本为 105951.26 元, 相比基准方案上升 0.70% 。在 $-20\% \sim 20\%$ 的扰动范围内, 总成本变化率始终控制在 1% 以内, 说明问题一基于均值车速得到的调度方案具有较好的稳健性。

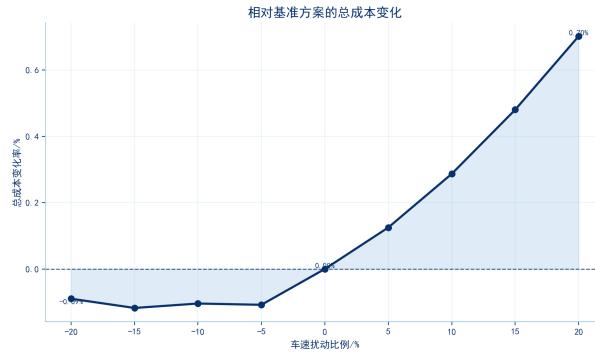


图 16: 相对基准方案的总成本变化率

从变化趋势来看，当车速由较低水平逐渐提高时，总成本整体呈缓慢上升趋势。这一现象与车辆能耗函数和时间窗惩罚机制共同作用有关。一方面，车速提高会缩短部分路段的行驶时间，但由于本问题中存在早到等待机制，较早到达并不一定能够直接降低服务成本，反而可能增加等待时间；另一方面，燃油车和新能源车的能耗均与车速呈 U 型关系，当车速偏离较优能耗区间时，单位里程能耗可能上升，从而推高能源成本和碳排放成本。因此，总成本并不随车速提高而单调下降。

为进一步比较不同指标对车速扰动的敏感程度，本文统计了总成本、能源成本、碳排放量和时间窗惩罚成本在 -20% 至 20% 扰动区间内的变化幅度，结果如图 17 所示。可以看出，从车速扰动 -20% 变化到 20% 时，总成本变化幅度为 0.79% ，能源成本和碳排放量的变化幅度分别为 -0.64% 和 -0.64% ，而时间窗惩罚成本的变化幅度达到 7.37% 。这说明车速扰动对总成本的直接影响并不显著，但对时间相关指标的影响较为明显。

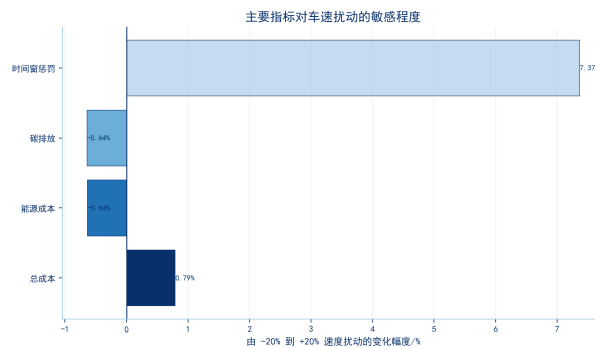


图 17: 主要指标对车速扰动的敏感程度

上述结果表明，车速变化主要通过两方面影响配送方案：一是改变行驶时间，从而影响到达时刻及时间窗惩罚成本；二是通过 U 型能耗函数影响单位里程能耗，进而作用于能源与碳排放成本。然而，在本次灵敏度分析中，车辆数量、客户分配及路径结构保持不变，使得固定成本与总里程基本稳定，而固定成本占比较高，因此总成本波动不明显。

总体来看，方案在车速均值附近具有较强稳定性，基于分时段均值车速进行调度是合理的。实际应用中，当交通波动出现时，无需整体重构路径，可针对时间窗较紧或拥堵时段客户进行局部调整，以提升方案鲁棒性。

10 模型评价

10.1 模型优点

建模完整性较强：模型将异构车队、软时间窗、时变交通、能耗与碳排放成本统一纳入同一优化框架，能够较全面刻画城市绿色配送的关键约束与决策因素。

政策适配能力较好：针对绿色配送区限行政策，模型可通过新能源优先、时空错位、路径绕行等多策略进行响应，并支持多场景对比，便于政策效果评估与管理决策。

动态响应可执行性高：在问题三中采用“事件驱动 + 局部冻结 + 边际插入”的实时重调度机制，在保证 100% 任务响应率的同时将路径扰动控制在较低水平，具有较好的落地性。

10.2 模型缺点

随机性刻画不足：当前模型主要基于确定性距离与时段速度参数，未显式考虑交通事故、极端天气、临时道路管制等随机扰动，复杂场景下鲁棒性仍有提升空间。

计算规模扩展受限：随着客户规模、策略维度与动态事件数量增加，启发式搜索与实时重调度的计算开销会明显上升，大规模实时应用下仍需进一步优化算法效率。

参考文献

- [1] 物流配送中的绿色车辆路径模型与求解算法研究综述 [J]. 系统工程理论与实践, 2021, 41(1).
- [2] 周鲜成, 蒋涛营, 贺彩虹, 等. 冷链物流配送的绿色车辆路径模型及其求解算法 [J]. 中国管理科学, 2023, 31(12): 203–214.
- [3] 张歆悦, 靳鹏, 胡笑旋, 等. 时间依赖型多配送中心带时间窗的开放式车辆路径问题研究 [J]. 中国管理科学, 2024, 32(1): 146–157.
- [4] Sahin M K, Yaman H. The Vehicle Routing Problem with Access Restrictions[R]. 2023.
- [5] 刘长石, 陈昭璞, 吴张. 城市物流配送的混合车辆路径规划模型与优化算法 [J]. 控制与决策, 2023, 38(3): 759–768.
- [6] Research on Dynamic Distribution Vehicle Route Optimization under the Influence of Carbon Emission[R].
- [7] 马振鹏, 焦晗暘, 张喆, 等. 面向城市物流配送的车辆路径优化算法研究 [J]. 系统仿真学报, 2025, 37(11): 2768–2777.
- [8] OpenAI. ChatGPT[EB/OL]. OpenAI, 2026-04-25.
- [9] Google. Gemini[EB/OL]. Google, 2026-04-25.
- [10] DeepSeek. DeepSeek-R1-0528[EB/OL]. 深度求索（DeepSeek）, 2026-04-25.

A 代码附录

本附录列出本文数据预处理、建模求解、政策场景分析、动态调度、可视化与灵敏度分析所使用的主要程序文件。

A.1 数据预处理

```
1 import os
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5
6
7
8 plt.rcParams["font.sans-serif"] = ["SimHei", "Microsoft YaHei"]
9 plt.rcParams["axes.unicode_minus"] = False
10
11 base_dir = r"D:\数学建模\2026华中杯\A题：城市绿色物流配送调度\附件"
12
13 order_path = os.path.join(base_dir, "订单信息.xlsx")
14 coord_path = os.path.join(base_dir, "客户坐标信息.xlsx")
15 time_path = os.path.join(base_dir, "时间窗.xlsx")
16 dist_path = os.path.join(base_dir, "距离矩阵.xlsx")
17
18 output_dir = os.path.join(base_dir, "预处理结果")
19 os.makedirs(output_dir, exist_ok=True)
20
21
22 # 1. 读取数据
23
24 orders = pd.read_excel(order_path)
25 coords = pd.read_excel(coord_path)
26 time_windows = pd.read_excel(time_path)
27 distance_matrix = pd.read_excel(dist_path, index_col=0)
28
29 orders.columns = orders.columns.str.strip()
30 coords.columns = coords.columns.str.strip()
31 time_windows.columns = time_windows.columns.str.strip()
32
33 print("订单信息列名: ", orders.columns.tolist())
34 print("坐标信息列名: ", coords.columns.tolist())
35 print("时间窗列名: ", time_windows.columns.tolist())
36 print("距离矩阵规模: ", distance_matrix.shape)
37
38
39 # 2. 按真实列名重命名
40
41 orders = orders.rename(columns={
42     "订单编号": "order_id",
43     "重量": "weight",
44     "体积": "volume",
45     "目标客户编号": "customer_id"
46 })
47
48 coords = coords.rename(columns={
49     "类型": "type",
50     "ID": "customer_id",
51     "X (km)": "x",
52     "Y (km)": "y"
53 })
54
55 time_windows = time_windows.rename(columns={
56     "客户编号": "customer_id",
57     "开始时间": "start_time",
58     "结束时间": "end_time"
59 })
60
61
62 # 3. 数据类型转换
63
64 orders["order_id"] = pd.to_numeric(orders["order_id"], errors="coerce").astype("Int64")
65 orders["customer_id"] = pd.to_numeric(orders["customer_id"], errors="coerce").astype("Int64")
66 orders["weight"] = pd.to_numeric(orders["weight"], errors="coerce")
67 orders["volume"] = pd.to_numeric(orders["volume"], errors="coerce")
68
69 coords["customer_id"] = pd.to_numeric(coords["customer_id"], errors="coerce").astype("Int64")
70 coords["x"] = pd.to_numeric(coords["x"], errors="coerce")
71 coords["y"] = pd.to_numeric(coords["y"], errors="coerce")
```

```

72 time_windows["customer_id"] = pd.to_numeric(time_windows["customer_id"], errors="coerce").astype("Int64")
73
74
75
76 # 4. 缺失值处理
77 # 重量、体积缺失值填为该客户点的中位数
78 # 如果某客户点对应字段全部缺失，则用全局中位数兜底
79
80 orders["weight_missing"] = orders["weight"].isna()
81 orders["volume_missing"] = orders["volume"].isna()
82
83 weight_missing_before = orders["weight_missing"].sum()
84 volume_missing_before = orders["volume_missing"].sum()
85
86 orders["weight"] = orders.groupby("customer_id")["weight"].transform(
87     lambda s: s.fillna(s.median())
88 )
89 orders["volume"] = orders.groupby("customer_id")["volume"].transform(
90     lambda s: s.fillna(s.median())
91 )
92
93 orders["weight"] = orders["weight"].fillna(orders["weight"].median())
94 orders["volume"] = orders["volume"].fillna(orders["volume"].median())
95
96 print("重量缺失填补前: ", weight_missing_before)
97 print("体积缺失填补前: ", volume_missing_before)
98 print("重量缺失填补后: ", orders["weight"].isna().sum())
99 print("体积缺失填补后: ", orders["volume"].isna().sum())
100
101
102 # 5. 按客户点汇总订单需求
103
104 customer_demand = orders.groupby("customer_id").agg(
105     total_weight=("weight", "sum"),
106     total_volume=("volume", "sum"),
107     order_count=("order_id", "count"),
108     weight_missing_count=("weight_missing", "sum"),
109     volume_missing_count=("volume_missing", "sum")
110 ).reset_index()
111
112
113 # 6. 时间窗处理
114
115 def time_to_hour(t):
116     """
117     将 Excel 时间、字符串时间、datetime 时间统一转成小时数。
118     例如 8:30 -> 8.5
119     """
120     if pd.isna(t):
121         return np.nan
122
123     if isinstance(t, (int, float)):
124         if 0 <= t <= 1:
125             return t * 24
126         return float(t)
127
128     t = pd.to_datetime(str(t))
129     return t.hour + t.minute / 60 + t.second / 3600
130
131
132 time_windows["ET"] = time_windows["start_time"].apply(time_to_hour)
133 time_windows["LT"] = time_windows["end_time"].apply(time_to_hour)
134 time_windows["time_window_length"] = time_windows["LT"] - time_windows["ET"]
135
136
137 # 7. 坐标处理与绿色配送区判断
138 # 绿色配送区：以市中心 (0,0) 为圆心，半径 10 km
139
140 coords["distance_to_center"] = np.sqrt(coords["x"] ** 2 + coords["y"] ** 2)
141 coords["in_green_zone"] = coords["distance_to_center"] <= 10
142
143 green_customers = coords[
144     (coords["type"] == "客户") &

```

```

145     (coords["in_green_zone"]))
146 ]
147
148 print("绿色配送区客户数量: ", len(green_customers))
149
150
151 # 8. 合并客户数据
152
153 customer_data = (
154     customer_demand
155     .merge(coords, on="customer_id", how="left")
156     .merge(
157         time_windows[["customer_id", "start_time", "end_time", "ET", "LT", "time_window_length"]],
158         on="customer_id",
159         how="left"
160     )
161 )
162
163 customer_data = customer_data.sort_values("customer_id").reset_index(drop=True)
164
165
166 # 9. 距离矩阵处理
167
168 distance_matrix.index = distance_matrix.index.astype(int)
169 distance_matrix.columns = distance_matrix.columns.astype(int)
170
171 def get_distance(i, j):
172     return float(distance_matrix.loc[int(i), int(j)])
173
174
175 # 10. 车型参数
176
177 vehicle_types = pd.DataFrame({
178     "vehicle_type": ["F1", "F2", "F3", "E1", "E2"],
179     "energy_type": ["fuel", "fuel", "fuel", "electric", "electric"],
180     "capacity_weight": [3000, 1500, 1250, 3000, 1250],
181     "capacity_volume": [13.5, 10.8, 6.5, 15.0, 8.5],
182     "max_num": [60, 50, 50, 10, 15],
183     "fixed_cost": [400, 400, 400, 400, 400],
184     "load_energy_alpha": [0.40, 0.40, 0.40, 0.35, 0.35],
185     "energy_price": [7.61, 7.61, 7.61, 1.64, 1.64],
186     "carbon_factor": [2.547, 2.547, 2.547, 0.501, 0.501]
187 })
188
189
190 # 11. 速度与能耗函数
191
192 def get_speed_by_departure_time(t):
193     """
194     根据离开时刻返回对应时段的期望速度。
195     t 单位: 小时。
196     """
197     if 8 <= t < 9:
198         return 9.8
199     elif 9 <= t < 10:
200         return 55.3
201     elif 10 <= t < 11.5:
202         return 35.4
203     elif 11.5 <= t < 13:
204         return 9.8
205     elif 13 <= t < 15:
206         return 55.3
207     elif 15 <= t < 17:
208         return 35.4
209     else:
210         return 35.4
211
212
213 def fpk(v):
214     """
215     燃油车百公里油耗, 单位 L/100km。
216     """
217     return 0.0025 * v ** 2 - 0.2554 * v + 31.75

```

```

218
219
220 def epk(v):
221     """
222     新能源车百公里电耗, 单位 kWh/100km。
223     """
224     return 0.0014 * v ** 2 - 0.12 * v + 36.19
225
226
227 def energy_consumption(distance_km, speed, load, capacity, energy_type):
228     """
229     计算某段路的油耗或电耗。
230     """
231     if energy_type == "fuel":
232         base_energy = fpk(speed)
233         alpha = 0.40
234     else:
235         base_energy = epk(speed)
236         alpha = 0.35
237
238     load_rate = load / capacity
239     return distance_km / 100 * base_energy * (1 + alpha * load_rate)
240
241
242 def carbon_emission(energy, energy_type):
243     """
244     计算碳排放量。
245     燃油: kg CO2 = L * 2.547
246     电能: kg CO2 = kWh * 0.501
247     """
248     if energy_type == "fuel":
249         return energy * 2.547
250     else:
251         return energy * 0.501
252
253
254 def carbon_cost(emission):
255     """
256     碳排放成本。
257     """
258     return emission * 0.65
259
260
261 # 12. 数据一致性检查
262
263 print("\n数据一致性检查")
264 print("订单总数: ", len(orders))
265 print("客户点数量: ", customer_data["customer_id"].nunique())
266 print("坐标客户数量: ", coords[coords["type"] == "客户"]["customer_id"].nunique())
267 print("时间窗客户数量: ", time_windows["customer_id"].nunique())
268 print("绿色配送区客户数量: ", len(green_customers))
269 print("总重量: ", customer_data["total_weight"].sum())
270 print("总体积: ", customer_data["total_volume"].sum())
271 print("最早时间窗: ", customer_data["ET"].min())
272 print("最晚时间窗: ", customer_data["LT"].max())
273
274 missing_coord = customer_data[customer_data["x"].isna() | customer_data["y"].isna()]
275 missing_tw = customer_data[customer_data["ET"].isna() | customer_data["LT"].isna()]
276
277 print("缺失坐标客户数: ", len(missing_coord))
278 print("缺失时间窗客户数: ", len(missing_tw))
279
280 if len(missing_coord) > 0:
281     print("缺失坐标客户: ")
282     print(missing_coord["customer_id"].tolist())
283
284 if len(missing_tw) > 0:
285     print("缺失时间窗客户: ")
286     print(missing_tw["customer_id"].tolist())
287
288
289 # 13. 可视化 1: 客户点空间分布与绿色配送区
290

```

```

291 fig, ax = plt.subplots(figsize=(8, 8))
292
293 normal = coords[(coords["type"] == "客户") & (~coords["in_green_zone"])]
294 green = coords[(coords["type"] == "客户") & (coords["in_green_zone"])]
295 depot = coords[coords["type"] == "配送中心"]
296
297 p1 = ax.scatter(
298     normal["x"], normal["y"],
299     label=f"普通客户点 ({len(normal)}个)",
300     s=42,
301     alpha=0.78,
302     edgecolors="white",
303     linewidths=0.5
304 )
305
306 p2 = ax.scatter(
307     green["x"], green["y"],
308     label=f"绿色配送区客户点 ({len(green)}个)",
309     s=58,
310     alpha=0.9,
311     edgecolors="black",
312     linewidths=0.5
313 )
314
315 p3 = ax.scatter(
316     depot["x"], depot["y"],
317     label="配送中心",
318     s=180,
319     marker="*",
320     color="limegreen",
321     edgecolors="black",
322     linewidths=0.8
323 )
324
325 circle = plt.Circle((0, 0), 10, fill=False, linestyle="--", linewidth=2, color="black")
326 ax.add_patch(circle)
327
328 circle_legend = plt.Line2D(
329     [0], [0],
330     color="black",
331     linestyle="--",
332     linewidth=2,
333     label="绿色配送区边界"
334 )
335
336 ax.axhline(0, linewidth=0.8, alpha=0.45)
337 ax.axvline(0, linewidth=0.8, alpha=0.45)
338
339 for _, row in depot.iterrows():
340     ax.text(row["x"] + 0.4, row["y"] + 0.4, "配送中心", fontsize=10)
341
342 ax.set_xlabel("X 坐标 / km")
343 ax.set_ylabel("Y 坐标 / km")
344 ax.set_title("客户点空间分布与绿色配送区")
345 ax.legend(handles=[p1, p2, p3, circle_legend], loc="best", frameon=True)
346 ax.axis("equal")
347 ax.grid(alpha=0.25)
348
349 plt.savefig(os.path.join(output_dir, "客户点空间分布与绿色配送区.png"), dpi=300, bbox_inches="tight")
350 plt.show()
351
352
353 # 14. 可视化 2: 重量需求最高客户
354
355 top_weight = customer_data.sort_values("total_weight", ascending=False).head(20)
356
357 fig, ax = plt.subplots(figsize=(11, 5.5))
358
359 bars = ax.bar(top_weight["customer_id"].astype(str), top_weight["total_weight"], alpha=0.85)
360
361 weight_limits = [
362     (3000, "最大载重 3000 kg"),
363     (1500, "载重 1500 kg"),

```

```

364     (1250, "载重 1250 kg")
365 ]
366
367 for value, label in weight_limits:
368     ax.axhline(value, linestyle="--", linewidth=1.2, alpha=0.8, label=label)
369
370 ax.set_xlabel("客户编号")
371 ax.set_ylabel("总重量 / kg")
372 ax.set_title("需求重量最高的前 20 个客户点")
373 ax.tick_params(axis="x", rotation=45)
374 ax.grid(axis="y", alpha=0.25)
375 ax.legend(frameon=True)
376
377 for bar in bars:
378     h = bar.get_height()
379     ax.text(bar.get_x() + bar.get_width() / 2, h, f"{h:.0f}", ha="center", va="bottom", fontsize=8)
380
381 plt.savefig(os.path.join(output_dir, "需求重量最高的前20个客户点.png"), dpi=300, bbox_inches="tight")
382 plt.show()
383
384
385 # 15. 可视化 3: 体积需求最高客户
386
387 top_volume = customer_data.sort_values("total_volume", ascending=False).head(20)
388
389 fig, ax = plt.subplots(figsize=(11, 5.5))
390
391 bars = ax.bar(top_volume["customer_id"].astype(str), top_volume["total_volume"], alpha=0.85)
392
393 volume_limits = [
394     (15.0, "最大容积 15.0 m3"),
395     (13.5, "容积 13.5 m3"),
396     (10.8, "容积 10.8 m3"),
397     (8.5, "容积 8.5 m3"),
398     (6.5, "容积 6.5 m3")
399 ]
400
401 for value, label in volume_limits:
402     ax.axhline(value, linestyle="--", linewidth=1.2, alpha=0.8, label=label)
403
404 ax.set_xlabel("客户编号")
405 ax.set_ylabel("总体积 / m3")
406 ax.set_title("需求体积最高的前 20 个客户点")
407 ax.tick_params(axis="x", rotation=45)
408 ax.grid(axis="y", alpha=0.25)
409 ax.legend(frameon=True, ncol=2)
410
411 for bar in bars:
412     h = bar.get_height()
413     ax.text(bar.get_x() + bar.get_width() / 2, h, f"{h:.1f}", ha="center", va="bottom", fontsize=8)
414
415 plt.savefig(os.path.join(output_dir, "需求体积最高的前20个客户点.png"), dpi=300, bbox_inches="tight")
416 plt.show()
417
418
419 # 16. 可视化 4: 重量-体积分布
420
421 fig, ax = plt.subplots(figsize=(9, 6.5))
422
423 normal_data = customer_data[~customer_data["in_green_zone"]]
424 green_data = customer_data[customer_data["in_green_zone"]]
425
426 s1 = ax.scatter(
427     normal_data["total_weight"],
428     normal_data["total_volume"],
429     s=45 + normal_data["order_count"] * 0.8,
430     alpha=0.75,
431     edgecolors="white",
432     linewidths=0.5,
433     label="普通客户点"
434 )
435
436 s2 = ax.scatter(

```

```

437     green_data["total_weight"],
438     green_data["total_volume"],
439     s=45 + green_data["order_count"] * 0.8,
440     alpha=0.9,
441     edgecolors="black",
442     linewidths=0.5,
443     label="绿色配送区客户点"
444 )
445
446 capacity_points = vehicle_types[
447     ["vehicle_type", "capacity_weight", "capacity_volume"]
448 ].drop_duplicates()
449
450 for _, row in capacity_points.iterrows():
451     w = row["capacity_weight"]
452     v = row["capacity_volume"]
453     name = row["vehicle_type"]
454
455     ax.axvline(w, linestyle="--", linewidth=1.0, alpha=0.55, color="gray")
456     ax.axhline(v, linestyle="--", linewidth=1.0, alpha=0.55, color="gray")
457     ax.scatter(
458         w, v,
459         marker="X",
460         s=95,
461         color="white",
462         edgecolors="black",
463         linewidths=1.2,
464         zorder=5
465     )
466     ax.text(w + 80, v + 0.25, name, fontsize=9)
467
468 capacity_legend = plt.Line2D(
469     [0], [0],
470     marker="X",
471     color="white",
472     markeredgecolor="black",
473     markerfacecolor="white",
474     markersize=9,
475     linestyle="None",
476     label="车型容量点"
477 )
478
479 line_legend = plt.Line2D(
480     [0], [0],
481     color="gray",
482     linestyle="--",
483     linewidth=1.2,
484     label="容量边界"
485 )
486
487 ax.set_xlabel("客户总重量 / kg")
488 ax.set_ylabel("客户总体积 / m³")
489 ax.set_title("客户需求重量-体积分布与车辆容量边界")
490 ax.grid(alpha=0.25)
491 ax.legend(handles=[s1, s2, capacity_legend, line_legend], loc="lower right", frameon=True)
492
493 plt.savefig(os.path.join(output_dir, "客户需求重量体积分布.png"), dpi=300, bbox_inches="tight")
494 plt.show()
495
496 # 17. 可视化 5: 软时间窗分布
497
498 tw = customer_data.sort_values(["ET", "LT"]).copy()
499 y_pos = np.arange(len(tw))
500
501 fig, ax = plt.subplots(figsize=(11, 8))
502
503 for y, (_, row) in zip(y_pos, tw.iterrows()):
504     color = "C1" if row["in_green_zone"] else "C0"
505     ax.hlines(
506         y=y,
507         xmin=row["ET"],
508         xmax=row["LT"],

```

```

510         linewidth=2.2,
511         color=color,
512         alpha=0.85
513     )
514
515 ax.set_xlabel("时间 / 小时")
516 ax.set_ylabel("客户点排序")
517 ax.set_title("客户软时间窗分布")
518 ax.grid(axis="x", alpha=0.25)
519 ax.set_xticks(np.arange(np.floor(tw["ET"].min()), np.ceil(tw["LT"].max()) + 1, 1))
520
521 ax.legend(
522     handles=[
523         plt.Line2D([0], [0], color="C0", linewidth=2.5, label="普通客户点"),
524         plt.Line2D([0], [0], color="C1", linewidth=2.5, label="绿色配送区客户点")
525     ],
526     loc="best",
527     frameon=True
528 )
529
530 plt.savefig(os.path.join(output_dir, "客户软时间窗分布.png"), dpi=300, bbox_inches="tight")
531 plt.show()
532
533
534 # 18. 可视化 6: 缺失值情况
535
536 missing_summary = pd.DataFrame({
537     "字段": ["重量", "体积"],
538     "缺失订单数": [
539         int(orders["weight_missing"].sum()),
540         int(orders["volume_missing"].sum())
541     ]
542 })
543
544 fig, ax = plt.subplots(figsize=(6, 4.5))
545
546 bars = ax.bar(missing_summary["字段"], missing_summary["缺失订单数"], alpha=0.85)
547
548 ax.set_ylabel("缺失订单数")
549 ax.set_title("订单重量与体积缺失情况")
550 ax.grid(axis="y", alpha=0.25)
551
552 for bar in bars:
553     h = bar.get_height()
554     ax.text(bar.get_x() + bar.get_width() / 2, h, str(int(h)), ha="center", va="bottom", fontsize=10)
555
556 plt.savefig(os.path.join(output_dir, "订单重量与体积缺失情况.png"), dpi=300, bbox_inches="tight")
557 plt.show()
558
559
560 # 19. 可视化 7: 客户订单数量分布
561
562 fig, ax = plt.subplots(figsize=(7.5, 5))
563
564 ax.hist(customer_data["order_count"], bins=20, alpha=0.85, edgecolor="white")
565 ax.axvline(
566     customer_data["order_count"].mean(),
567     linestyle="--",
568     linewidth=1.5,
569     label=f"均值: {customer_data['order_count'].mean():.1f}"
570 )
571 ax.axvline(
572     customer_data["order_count"].median(),
573     linestyle=":",
574     linewidth=1.8,
575     label=f"中位数: {customer_data['order_count'].median():.1f}"
576 )
577
578 ax.set_xlabel("订单数量")
579 ax.set_ylabel("客户点数量")
580 ax.set_title("客户点订单数量分布")
581 ax.grid(axis="y", alpha=0.25)
582 ax.legend(frameon=True)

```

```

583
584 plt.savefig(os.path.join(output_dir, "客户点订单数量分布.png"), dpi=300, bbox_inches="tight")
585 plt.show()
586
587
588 # 20. 可视化 8: 距离矩阵热力图
589
590 fig, ax = plt.subplots(figsize=(8, 6.5))
591
592 im = ax.imshow(distance_matrix.values, aspect="auto")
593 cbar = plt.colorbar(im, ax=ax)
594 cbar.set_label("距离 / km")
595
596 ax.set_title("距离矩阵热力图")
597 ax.set_xlabel("客户点编号")
598 ax.set_ylabel("客户点编号")
599
600 plt.savefig(os.path.join(output_dir, "距离矩阵热力图.png"), dpi=300, bbox_inches="tight")
601 plt.show()
602
603
604 # 21. 可视化 9: 速度-能耗函数曲线
605
606 speed_points = np.linspace(5, 70, 200)
607 speed_levels = [9.8, 35.4, 55.3]
608
609 fig, ax = plt.subplots(figsize=(8.5, 5.5))
610
611 ax.plot(speed_points, fpk(speed_points), label="燃油车 FPK")
612 ax.plot(speed_points, epk(speed_points), label="新能源车 EPK")
613
614 ax.scatter(speed_levels, [fpk(v) for v in speed_levels], marker="o", s=60, label="燃油车典型速度点")
615 ax.scatter(speed_levels, [epk(v) for v in speed_levels], marker="s", s=60, label="新能源车典型速度点")
616
617 for v in speed_levels:
618     ax.axvline(v, linestyle="--", linewidth=0.9, alpha=0.45)
619     ax.text(v + 0.5, ax.get_ylim()[0] + 0.5, f"{v}km/h", fontsize=9, rotation=90, va="bottom")
620
621 ax.set_xlabel("速度 / km/h")
622 ax.set_ylabel("百公里能耗")
623 ax.set_title("速度-能耗函数曲线")
624 ax.legend(frameon=True)
625 ax.grid(alpha=0.25)
626
627 plt.savefig(os.path.join(output_dir, "速度能耗函数曲线.png"), dpi=300, bbox_inches="tight")
628 plt.show()
629
630
631 # 22. 保存预处理结果
632
633 orders.to_excel(os.path.join(output_dir, "订单清洗结果.xlsx"), index=False)
634 customer_data.to_excel(os.path.join(output_dir, "客户需求汇总结果.xlsx"), index=False)
635 vehicle_types.to_excel(os.path.join(output_dir, "车型参数.xlsx"), index=False)
636 distance_matrix.to_excel(os.path.join(output_dir, "距离矩阵处理结果.xlsx"))
637
638 with pd.ExcelWriter(os.path.join(output_dir, "第一问数据预处理汇总.xlsx")) as writer:
639     orders.to_excel(writer, sheet_name="订单清洗结果", index=False)
640     customer_data.to_excel(writer, sheet_name="客户需求汇总", index=False)
641     vehicle_types.to_excel(writer, sheet_name="车型参数", index=False)
642     distance_matrix.to_excel(writer, sheet_name="距离矩阵")
643
644
645 # 23. 最终输出
646
647 print("\n数据预处理完成")
648 print("结果文件夹: ", output_dir)
649 print("订单清洗结果: ", os.path.join(output_dir, "订单清洗结果.xlsx"))
650 print("客户需求汇总结果: ", os.path.join(output_dir, "客户需求汇总结果.xlsx"))
651 print("第一问数据预处理汇总: ", os.path.join(output_dir, "第一问数据预处理汇总.xlsx"))
652
653 print("\n关键结果")
654 print("订单总数: ", len(orders))
655 print("客户点数量: ", customer_data["customer_id"].nunique())

```

```

656 print("绿色配送区客户数量: ", len(green_customers))
657 print("总重量: ", round(customer_data["total_weight"].sum(), 4))
658 print("总体积: ", round(customer_data["total_volume"].sum(), 4))
659 print("重量缺失订单数: ", int(weight_missing_before))
660 print("体积缺失订单数: ", int(volume_missing_before))
661 print("时间窗范围: ", customer_data["ET"].min(), "到", customer_data["LT"].max())
662
663 print("\n车型参数: ")
664 print(vehicle_types)

```

Listing 1: 数据预处理.py

A.2 问题一求解主程序

```

1  from __future__ import annotations
2
3  import argparse
4  import json
5  import math
6  import random
7  from dataclasses import dataclass, field
8  from pathlib import Path
9  from typing import Dict, List, Optional, Tuple
10
11 import matplotlib.pyplot as plt
12 import numpy as np
13 import pandas as pd
14
15
16
17 def fuel_consumption_100km(v: float) -> float:
18     return 0.0025 * v * v - 0.2554 * v + 31.75
19
20
21 def electric_consumption_100km(v: float) -> float:
22     return 0.0014 * v * v - 0.12 * v + 36.19
23
24
25 def parse_time_to_hour(value) -> float:
26     if pd.isna(value):
27         return 0.0
28     if isinstance(value, (int, float, np.integer, np.floating)):
29         return float(value)
30     text = str(value).strip()
31     if ":" not in text:
32         return float(text)
33     h, m = text.split(":")[:2]
34     return int(h) + int(m) / 60.0
35
36
37 def hour_to_text(x: float) -> str:
38     total_minutes = int(round(x * 60))
39     h = total_minutes // 60
40     m = total_minutes % 60
41     return f"{h:02d}:{m:02d}"
42
43
44 def normalize_node_id(x) -> str:
45     if isinstance(x, (int, np.integer)):
46         return str(int(x))
47     if isinstance(x, (float, np.floating)):
48         return str(int(x)) if float(x).is_integer() else str(float(x))
49     return str(x).strip()
50
51
52 @dataclass
53 class Customer:
54     customer_id: str
55     demand_weight: float
56     demand_volume: float

```

```

57     earliest: float
58     latest: float
59     x: float
60     y: float
61     green_zone: int
62
63
64 @dataclass
65 class VehicleType:
66     type_id: str
67     powertrain: str
68     max_weight: float
69     max_volume: float
70     count: int
71     energy_price: float
72     fixed_cost: float
73     load_factor: float
74     carbon_factor: float
75
76
77 @dataclass
78 class Vehicle:
79     vehicle_id: str
80     vehicle_type: VehicleType
81
82
83 @dataclass
84 class Visit:
85     customer_id: str
86     deliver_weight: float
87     deliver_volume: float
88
89
90 @dataclass
91 class RouteMetrics:
92     distance: float = 0.0
93     travel_time: float = 0.0
94     waiting_time: float = 0.0
95     late_time: float = 0.0
96     energy_amount: float = 0.0
97     energy_cost: float = 0.0
98     carbon_cost: float = 0.0
99     time_penalty_cost: float = 0.0
100     fixed_cost: float = 0.0
101     total_cost: float = 0.0
102     return_time: float = 0.0
103     arrival_times: List[float] = field(default_factory=list)
104     service_start_times: List[float] = field(default_factory=list)
105     leave_times: List[float] = field(default_factory=list)
106     leg_distances: List[float] = field(default_factory=list)
107     leg_energy: List[float] = field(default_factory=list)
108
109
110 @dataclass
111 class Route:
112     vehicle: Vehicle
113     visits: List[Visit] = field(default_factory=list)
114     metrics: Optional[RouteMetrics] = None
115
116     def total_weight(self) -> float:
117         return sum(v.deliver_weight for v in self.visits)
118
119     def total_volume(self) -> float:
120         return sum(v.deliver_volume for v in self.visits)
121
122     def remaining_weight_capacity(self) -> float:
123         return self.vehicle.vehicle_type.max_weight - self.total_weight()
124
125     def remaining_volume_capacity(self) -> float:
126         return self.vehicle.vehicle_type.max_volume - self.total_volume()
127
128     def copy(self) -> "Route":
129         return Route(

```

```

130         vehicle=self.vehicle,
131         visits=[Visit(v.customer_id, v.deliver_weight, v.deliver_volume) for v in self.visits],
132         metrics=self.metrics,
133     )
134
135
136 @dataclass
137 class ProblemData:
138     depot_id: str
139     depot_x: float
140     depot_y: float
141     customers: Dict[str, Customer]
142     distances: Dict[Tuple[str, str], float]
143     vehicle_types: List[VehicleType]
144     vehicles: List[Vehicle]
145     start_time: float
146     service_time: float
147     early_cost_per_hour: float
148     late_cost_per_hour: float
149     carbon_cost_per_unit: float
150     time_segments: List[Dict[str, float]]
151     random_seed: int
152
153
154 @dataclass
155 class Solution:
156     routes: List[Route]
157     unserved_weight: Dict[str, float]
158     unserved_volume: Dict[str, float]
159     total_cost: float = math.inf
160
161     def copy(self) -> "Solution":
162         return Solution(
163             routes=[r.copy() for r in self.routes],
164             unserved_weight=dict(self.unserved_weight),
165             unserved_volume=dict(self.unserved_volume),
166             total_cost=self.total_cost,
167         )
168
169
170
171
172 class GreenLogisticsSolver:
173     def __init__(self, data: ProblemData):
174         self.data = data
175         self.rng = random.Random(data.random_seed)
176         self.vehicle_lookup = {v.vehicle_id: v for v in data.vehicles}
177         self.available_vehicle_ids = [v.vehicle_id for v in data.vehicles]
178
179     def solve(
180         self,
181         iterations: int = 600,
182         restarts: int = 6,
183         destroy_min: int = 3,
184         destroy_max: int = 8,
185     ) -> Solution:
186         best = None
187
188         for _ in range(restarts):
189             current = self._build_initial_solution()
190             self._repair_solution(current)
191             self._post_optimize(current)
192             self._recompute_solution_cost(current)
193
194             if best is None or current.total_cost < best.total_cost:
195                 best = current.copy()
196
197         T = max(100.0, current.total_cost * 0.04)
198         for _ in range(iterations):
199             candidate = current.copy()
200             self._destroy_solution(candidate, destroy_min, destroy_max)
201             self._repair_solution(candidate)
202             self._post_optimize(candidate)

```

```

203         self._recompute_solution_cost(candidate)
204
205         if self._accept(candidate.total_cost, current.total_cost, T):
206             current = candidate
207             if current.total_cost < best.total_cost:
208                 best = current.copy()
209             T *= 0.995
210
211         return best
212
213     def _build_initial_solution(self) -> Solution:
214         remaining_w = {cid: c.demand_weight for cid, c in self.data.customers.items()}
215         remaining_v = {cid: c.demand_volume for cid, c in self.data.customers.items()}
216         routes: List[Route] = []
217
218         priority = sorted(
219             self.data.customers.keys(),
220             key=lambda cid: (self.data.customers[cid].latest, -self.data.customers[cid].demand_weight)
221         )
222
223         tmp = Solution(routes, remaining_w, remaining_v)
224         for cid in priority:
225             while tmp.unserved_weight[cid] > 1e-6:
226                 if self._best_insert_existing(tmp, cid, commit=True):
227                     continue
228                 route = self._open_new_route(tmp, cid)
229                 if route is None:
230                     break
231                 tmp.routes.append(route)
232
233         return tmp
234
235     def _repair_solution(self, solution: Solution) -> None:
236         attempts = 0
237         while any(v > 1e-6 for v in solution.unserved_weight.values()) and attempts < 10000:
238             attempts += 1
239             cid = self._pick_next_unserved(solution.unserved_weight)
240             ok = self._best_insert_existing(solution, cid, commit=True)
241             if not ok:
242                 route = self._open_new_route(solution, cid)
243                 if route is None:
244                     break
245                 solution.routes.append(route)
246             self._compact(solution)
247
248     def _post_optimize(self, solution: Solution) -> None:
249         self._compact(solution)
250         for _ in range(2):
251             self._improve_order(solution)
252             self._relocate(solution)
253             self._compact(solution)
254
255     def _improve_order(self, solution: Solution) -> None:
256         ranked = sorted(
257             range(len(solution.routes)),
258             key=lambda i: solution.routes[i].metrics.total_cost if solution.routes[i].metrics else 0,
259             reverse=True
260         )[: min(15, len(solution.routes))]
261
262         for idx in ranked:
263             route = solution.routes[idx]
264             if len(route.visits) < 3:
265                 continue
266
267             best_route = route
268             best_metrics = route.metrics if route.metrics else self._evaluate_route(route)
269             best_cost = best_metrics.total_cost
270
271             limit = min(len(route.visits), 8)
272             for i in range(limit - 1):
273                 for j in range(i + 1, limit):
274                     cand = route.copy()
275                     cand.visits[i:j + 1] = list(reversed(cand.visits[i:j + 1]))

```

```

276         metrics = self._evaluate_route(cand)
277         if metrics is None:
278             continue
279         if metrics.total_cost < best_cost - 1e-6:
280             cand.metrics = metrics
281             best_route = cand
282             best_cost = metrics.total_cost
283
284         solution.routes[idx] = best_route
285
286     def _relocate(self, solution: Solution) -> None:
287         changed = True
288         while changed:
289             changed = False
290             for src_idx, src_route in enumerate(solution.routes):
291                 if not src_route.visits:
292                     continue
293
294                 src_positions = [0, len(src_route.visits) - 1]
295                 if len(src_route.visits) > 2:
296                     src_positions.append(len(src_route.visits) // 2)
297                 src_positions = sorted(set(p for p in src_positions if 0 <= p < len(src_route.visits)))
298
299                 for visit_pos in src_positions:
300                     best_move = None
301                     src_cost = src_route.metrics.total_cost if src_route.metrics else self._evaluate_route(
src_route).total_cost
302
303                     for dst_idx, dst_route in enumerate(solution.routes):
304                         if src_idx == dst_idx:
305                             continue
306
307                         dst_cost = dst_route.metrics.total_cost if dst_route.metrics else self.
_evaluate_route(dst_route).total_cost
308                         src_trial = src_route.copy()
309                         moved = src_trial.visits.pop(visit_pos)
310
311                         src_trial_metrics = self._evaluate_route(src_trial) if src_trial.visits else
RouteMetrics(total_cost=0.0)
312                         candidate_positions = [0, len(dst_route.visits)]
313                         if len(dst_route.visits) > 1:
314                             candidate_positions.append(len(dst_route.visits) // 2)
315
316                         for insert_pos in sorted(set(candidate_positions)):
317                             dst_trial = dst_route.copy()
318                             dst_trial.visits.insert(insert_pos, moved)
319                             dst_trial_metrics = self._evaluate_route(dst_trial)
320                             if dst_trial_metrics is None:
321                                 continue
322
323                             delta = (src_trial_metrics.total_cost + dst_trial_metrics.total_cost) - (
src_cost + dst_cost)
324                             if best_move is None or delta < best_move[0]:
325                                 best_move = (
326                                     delta,
327                                     dst_idx,
328                                     src_trial,
329                                     src_trial_metrics,
330                                     dst_trial,
331                                     dst_trial_metrics,
332                                 )
333
334                             if best_move and best_move[0] < -1e-6:
335                                 _, dst_idx, src_trial, src_m, dst_trial, dst_m = best_move
336                                 solution.routes[src_idx] = src_trial
337                                 solution.routes[src_idx].metrics = src_m if src_trial.visits else None
338                                 solution.routes[dst_idx] = dst_trial
339                                 solution.routes[dst_idx].metrics = dst_m
340                                 self._compact(solution)
341                                 changed = True
342                                 break
343         if changed:
344             break

```

```

345
346 def _pick_next_unserved(self, rem: Dict[str, float]) -> str:
347     pos = [(cid, w) for cid, w in rem.items() if w > 1e-6]
348     pos.sort(key=lambda x: x[1], reverse=True)
349     top = pos[: min(8, len(pos))]
350     return self.rng.choice(top)[0]
351
352 def _best_insert_existing(self, solution: Solution, customer_id: str, commit: bool = True) -> bool:
353     rem_w = solution.unserved_weight[customer_id]
354     rem_v = solution.unserved_volume[customer_id]
355     if rem_w <= 1e-6:
356         return True
357
358     customer = self.data.customers[customer_id]
359     vpw = customer.demand_volume / customer.demand_weight if customer.demand_weight > 1e-9 else 0.0
360     best = None
361
362     for ridx, route in enumerate(solution.routes):
363         cap_w = route.remaining_weight_capacity()
364         cap_v = route.remaining_volume_capacity()
365         if cap_w <= 1e-6 or cap_v <= 1e-6:
366             continue
367
368         deliver_w = min(rem_w, cap_w, cap_v / vpw if vpw > 1e-9 else cap_w)
369         if deliver_w <= 1e-6:
370             continue
371         deliver_v = rem_v if customer.demand_weight <= 1e-9 else deliver_w * vpw
372         deliver_v = min(deliver_v, rem_v, cap_v)
373
374         base_cost = route.metrics.total_cost if route.metrics else self._evaluate_route(route).
total_cost
375         for pos in range(len(route.visits) + 1):
376             cand = route.copy()
377             cand.visits.insert(pos, Visit(customer_id, deliver_w, deliver_v))
378             metrics = self._evaluate_route(cand)
379             if metrics is None:
380                 continue
381
382             penalty = 0.0
383             if metrics.late_time > 3.0:
384                 penalty += 2000.0
385
386             delta = metrics.total_cost - base_cost + penalty
387             if best is None or delta < best[0]:
388                 best = (delta, ridx, pos, deliver_w, deliver_v, metrics)
389
390         if best is None:
391             return False
392         if not commit:
393             return True
394
395         _, ridx, pos, deliver_w, deliver_v, metrics = best
396         solution.routes[ridx].visits.insert(pos, Visit(customer_id, deliver_w, deliver_v))
397         solution.routes[ridx].metrics = metrics
398         solution.unserved_weight[customer_id] = max(0.0, solution.unserved_weight[customer_id] - deliver_w)
399         solution.unserved_volume[customer_id] = max(0.0, solution.unserved_volume[customer_id] - deliver_v)
400         return True
401
402 def _open_new_route(self, solution: Solution, customer_id: str) -> Optional[Route]:
403     used_ids = {r.vehicle.vehicle_id for r in solution.routes}
404     available = [vid for vid in self.available_vehicle_ids if vid not in used_ids]
405     if not available:
406         return None
407
408     customer = self.data.customers[customer_id]
409     rem_w = solution.unserved_weight[customer_id]
410     rem_v = solution.unserved_volume[customer_id]
411     vpw = customer.demand_volume / customer.demand_weight if customer.demand_weight > 1e-9 else 0.0
412
413     best = None
414     for vid in available:
415         vehicle = self.vehicle_lookup[vid]
416         vt = vehicle.vehicle_type

```

```

417         deliver_w = min(rem_w, vt.max_weight, vt.max_volume / vpw if vpw > 1e-9 else vt.max_weight)
418         if deliver_w <= 1e-6:
419             continue
420         deliver_v = rem_v if customer.demand_weight <= 1e-9 else deliver_w * vpw
421         deliver_v = min(deliver_v, rem_v, vt.max_volume)
422
423         route = Route(vehicle=vehicle, visits=[Visit(customer_id, deliver_w, deliver_v)])
424         metrics = self._evaluate_route(route)
425         if metrics is None:
426             continue
427         route.metrics = metrics
428
429         score = metrics.total_cost
430         if best is None or score < best[0]:
431             best = (score, route)
432
433     if best is None:
434         return None
435
436     route = best[1]
437     v = route.visits[0]
438     solution.unserved_weight[customer_id] = max(0.0, solution.unserved_weight[customer_id] - v.
439 deliver_weight)
440     solution.unserved_volume[customer_id] = max(0.0, solution.unserved_volume[customer_id] - v.
441 deliver_volume)
442     return route
443
444 def _destroy_solution(self, solution: Solution, destroy_min: int, destroy_max: int) -> None:
445     all_pos = []
446     for ridx, route in enumerate(solution.routes):
447         for pos, visit in enumerate(route.visits):
448             all_pos.append((ridx, pos, visit))
449     if not all_pos:
450         return
451
452     k = self.rng.randint(destroy_min, min(destroy_max, len(all_pos)))
453     selected = self.rng.sample(all_pos, k)
454     selected.sort(key=lambda x: (x[0], x[1]), reverse=True)
455
456     for ridx, pos, _ in selected:
457         if ridx >= len(solution.routes) or pos >= len(solution.routes[ridx].visits):
458             continue
459         removed = solution.routes[ridx].visits.pop(pos)
460         solution.unserved_weight[removed.customer_id] += removed.deliver_weight
461         solution.unserved_volume[removed.customer_id] += removed.deliver_volume
462         if solution.routes[ridx].visits:
463             solution.routes[ridx].metrics = self._evaluate_route(solution.routes[ridx])
464
465     self._compact(solution)
466
467 def _compact(self, solution: Solution) -> None:
468     solution.routes = [r for r in solution.routes if r.visits]
469
470 def _recompute_solution_cost(self, solution: Solution) -> None:
471     total = 0.0
472     penalty = 0.0
473     for route in solution.routes:
474         route.metrics = self._evaluate_route(route)
475         total += route.metrics.total_cost
476
477     for cid, rem in solution.unserved_weight.items():
478         if rem > 1e-6:
479             customer = self.data.customers[cid]
480             penalty += 100000.0 * rem / max(customer.demand_weight, 1e-6)
481
482     solution.total_cost = total + penalty
483
484 def _accept(self, new_cost: float, old_cost: float, T: float) -> bool:
485     if new_cost <= old_cost:
486         return True
487     if T <= 1e-9:
488         return False
489     return self.rng.random() < math.exp(-(new_cost - old_cost) / T)

```

```

488
489 def _evaluate_route(self, route: Route) -> Optional[RouteMetrics]:
490     vt = route.vehicle.vehicle_type
491     if route.total_weight() - vt.max_weight > 1e-6:
492         return None
493     if route.total_volume() - vt.max_volume > 1e-6:
494         return None
495
496     metrics = RouteMetrics(fixed_cost=vt.fixed_cost)
497     current_node = self.data.depot_id
498     current_time = self.data.start_time
499     remaining_load = route.total_weight()
500
501     for visit in route.visits:
502         d = self._distance(current_node, visit.customer_id)
503         t, e = self._travel_time_and_energy(d, current_time, remaining_load, route.vehicle)
504
505         arrival = current_time + t
506         customer = self.data.customers[visit.customer_id]
507         service_start = max(arrival, customer.earliest)
508         leave_time = service_start + self.data.service_time
509         wait = max(0.0, customer.earliest - arrival)
510         late = max(0.0, arrival - customer.latest)
511
512         metrics.distance += d
513         metrics.travel_time += t
514         metrics.waiting_time += wait
515         metrics.late_time += late
516         metrics.energy_amount += e
517         metrics.arrival_times.append(arrival)
518         metrics.service_start_times.append(service_start)
519         metrics.leave_times.append(leave_time)
520         metrics.leg_distances.append(d)
521         metrics.leg_energy.append(e)
522
523         current_node = visit.customer_id
524         current_time = leave_time
525         remaining_load -= visit.deliver_weight
526         if remaining_load < -1e-6:
527             return None
528
529     if route.visits:
530         d = self._distance(current_node, self.data.depot_id)
531         t, e = self._travel_time_and_energy(d, current_time, max(0.0, remaining_load), route.vehicle)
532         metrics.distance += d
533         metrics.travel_time += t
534         metrics.energy_amount += e
535         metrics.return_time = current_time + t
536
537     metrics.energy_cost = metrics.energy_amount * vt.energy_price
538     metrics.carbon_cost = metrics.energy_amount * vt.carbon_factor * self.data.carbon_cost_per_unit
539     metrics.time_penalty_cost = (
540         metrics.waiting_time * self.data.early_cost_per_hour
541         + metrics.late_time * self.data.late_cost_per_hour
542     )
543     metrics.total_cost = (
544         metrics.fixed_cost
545         + metrics.energy_cost
546         + metrics.carbon_cost
547         + metrics.time_penalty_cost
548     )
549     return metrics
550
551 def _travel_time_and_energy(
552     self,
553     distance: float,
554     depart_time: float,
555     load_weight: float,
556     vehicle: Vehicle,
557 ) -> Tuple[float, float]:
558     remaining_distance = distance
559     current_time = depart_time
560     total_time = 0.0

```

```

561         total_energy = 0.0
562
563         vt = vehicle.vehicle_type
564         load_ratio = min(1.0, max(0.0, load_weight / max(vt.max_weight, 1e-6)))
565
566         while remaining_distance > 1e-9:
567             speed, seg_end = self._speed_at(current_time)
568             max_duration = max(seg_end - current_time, 1e-6)
569             can_go = speed * max_duration
570             leg_distance = min(remaining_distance, can_go)
571             leg_time = leg_distance / max(speed, 1e-6)
572
573             base = fuel_consumption_100km(speed) if vt.powertrain == "fuel" else electric_consumption_100km
574             (speed)
575
576             # 空载-满载差异就在这里
577             leg_energy = (leg_distance / 100.0) * base * (1.0 + vt.load_factor * load_ratio)
578
579             total_time += leg_time
580             total_energy += leg_energy
581             current_time += leg_time
582             remaining_distance -= leg_distance
583
584         return total_time, total_energy
585
586     def _speed_at(self, current_time: float) -> Tuple[float, float]:
587         for seg in self.data.time_segments:
588             if seg["start"] <= current_time < seg["end"]:
589                 return seg["speed"], seg["end"]
590         last = self.data.time_segments[-1]
591         if current_time >= last["end"]:
592             return last["speed"], current_time + 24.0
593         first = self.data.time_segments[0]
594         return first["speed"], first["end"]
595
596     def _distance(self, i: str, j: str) -> float:
597         key = (str(i), str(j))
598         if key not in self.data.distances:
599             raise KeyError(f"缺少距离: {i} -> {j}")
600         return self.data.distances[key]
601
602
603
604     def read_customers_xlsx(path: Path, sheet_name: Optional[str]) -> Dict[str, Customer]:
605         df = pd.read_excel(path, sheet_name=sheet_name, engine="openpyxl")
606         customers = {}
607         for _, row in df.iterrows():
608             cid = normalize_node_id(row["customer_id"])
609             customers[cid] = Customer(
610                 customer_id=cid,
611                 demand_weight=float(row["total_weight"]),
612                 demand_volume=float(row["total_volume"]),
613                 earliest=parse_time_to_hour(row["ET"]),
614                 latest=parse_time_to_hour(row["LT"]),
615                 x=float(row.get("x", 0.0)),
616                 y=float(row.get("y", 0.0)),
617                 green_zone=int(bool(row.get("in_green_zone", 0))),
618             )
619         return customers
620
621
622     def read_vehicle_types_xlsx(path: Path, sheet_name: Optional[str]) -> List[VehicleType]:
623         df = pd.read_excel(path, sheet_name=sheet_name, engine="openpyxl")
624         ans = []
625         for _, row in df.iterrows():
626             ans.append(
627                 VehicleType(
628                     type_id=str(row["vehicle_type"]),
629                     powertrain=str(row["energy_type"]).strip().lower(),
630                     max_weight=float(row["capacity_weight"]),
631                     max_volume=float(row["capacity_volume"]),
632                     count=int(row["max_num"]),

```

```

633         energy_price=float(row["energy_price"]),
634         fixed_cost=float(row["fixed_cost"]),
635         load_factor=float(row["load_energy_alpha"]),
636         carbon_factor=float(row["carbon_factor"]),
637     )
638 )
639 return ans
640
641
642 def read_distance_matrix_xlsx(path: Path, sheet_name: Optional[str]) -> Dict[Tuple[str, str], float]:
643     df = pd.read_excel(path, sheet_name=sheet_name, engine="openpyxl")
644     header = [normalize_node_id(x) for x in df.columns[1:]]
645     distances = {}
646     for _, row in df.iterrows():
647         from_id = normalize_node_id(row.iloc[0])
648         for to_id, value in zip(header, row.iloc[1:]):
649             distances[(from_id, to_id)] = float(value)
650     return distances
651
652
653 def build_vehicles(vehicle_types: List[VehicleType]) -> List[Vehicle]:
654     vehicles = []
655     for vt in vehicle_types:
656         for i in range(1, vt.count + 1):
657             vehicles.append(Vehicle(vehicle_id=f"{vt.type_id}_{i}", vehicle_type=vt))
658     vehicles.sort(
659         key=lambda x: (
660             x.vehicle_type.fixed_cost,
661             x.vehicle_type.energy_price,
662             -x.vehicle_type.max_weight,
663             -x.vehicle_type.max_volume,
664         )
665     )
666     return vehicles
667
668
669 def load_problem(config_path: Path) -> ProblemData:
670     with open(config_path, "r", encoding="utf-8") as f:
671         cfg = json.load(f)
672
673     base_dir = config_path.parent
674
675     def resolve(p: str) -> Path:
676         path = Path(p)
677         return path if path.is_absolute() else base_dir / path
678
679     customers = read_customers_xlsx(resolve(cfg["customers_file"]), cfg.get("customers_sheet"))
680     vehicle_types = read_vehicle_types_xlsx(resolve(cfg["vehicle_types_file"]), cfg.get("vehicle_types_sheet"))
681     distances = read_distance_matrix_xlsx(resolve(cfg["distance_matrix_file"]), cfg.get("distance_matrix_sheet"))
682     vehicles = build_vehicles(vehicle_types)
683
684     return ProblemData(
685         depot_id=str(cfg.get("depot_id", "0")),
686         depot_x=float(cfg.get("depot_x", 0.0)),
687         depot_y=float(cfg.get("depot_y", 0.0)),
688         customers=customers,
689         distances=distances,
690         vehicle_types=vehicle_types,
691         vehicles=vehicles,
692         start_time=parse_time_to_hour(cfg.get("start_time", "08:00")),
693         service_time=float(cfg.get("service_time_hours", 1 / 3)),
694         early_cost_per_hour=float(cfg.get("early_cost_per_hour", 20.0)),
695         late_cost_per_hour=float(cfg.get("late_cost_per_hour", 50.0)),
696         carbon_cost_per_unit=float(cfg.get("carbon_cost_per_unit", 0.65)),
697         time_segments=cfg["time_segments"],
698         random_seed=int(cfg.get("random_seed", 42)),
699     )
700
701
702
703

```

```

704 def summarize_solution(solution: Solution) -> Dict:
705     total_fixed = sum(r.metrics.fixed_cost for r in solution.routes)
706     total_energy_cost = sum(r.metrics.energy_cost for r in solution.routes)
707     total_carbon_cost = sum(r.metrics.carbon_cost for r in solution.routes)
708     total_time_penalty = sum(r.metrics.time_penalty_cost for r in solution.routes)
709     total_distance = sum(r.metrics.distance for r in solution.routes)
710
711     type_summary = {}
712     for route in solution.routes:
713         vt = route.vehicle.vehicle_type
714         d = type_summary.setdefault(
715             vt.type_id,
716             {
717                 "powertrain": vt.powertrain,
718                 "vehicle_count": 0,
719                 "distance": 0.0,
720                 "weight": 0.0,
721                 "volume": 0.0,
722                 "fixed_cost": 0.0,
723                 "energy_cost": 0.0,
724                 "carbon_cost": 0.0,
725                 "time_penalty_cost": 0.0,
726                 "total_cost": 0.0,
727             },
728         )
729         d["vehicle_count"] += 1
730         d["distance"] += route.metrics.distance
731         d["weight"] += route.total_weight()
732         d["volume"] += route.total_volume()
733         d["fixed_cost"] += route.metrics.fixed_cost
734         d["energy_cost"] += route.metrics.energy_cost
735         d["carbon_cost"] += route.metrics.carbon_cost
736         d["time_penalty_cost"] += route.metrics.time_penalty_cost
737         d["total_cost"] += route.metrics.total_cost
738
739     return {
740         "total_cost": solution.total_cost,
741         "route_count": len(solution.routes),
742         "total_distance": total_distance,
743         "fixed_cost": total_fixed,
744         "energy_cost": total_energy_cost,
745         "carbon_cost": total_carbon_cost,
746         "time_penalty_cost": total_time_penalty,
747         "unserved_customer_count": sum(1 for v in solution.unserved_weight.values() if v > 1e-6),
748         "type_summary": type_summary,
749     }
750
751
752 def write_outputs(solution: Solution, data: ProblemData, out_dir: Path) -> None:
753     out_dir.mkdir(parents=True, exist_ok=True)
754     summary = summarize_solution(solution)
755
756     pd.DataFrame([
757         {
758             "total_cost": summary["total_cost"],
759             "route_count": summary["route_count"],
760             "total_distance": summary["total_distance"],
761             "fixed_cost": summary["fixed_cost"],
762             "energy_cost": summary["energy_cost"],
763             "carbon_cost": summary["carbon_cost"],
764             "time_penalty_cost": summary["time_penalty_cost"],
765             "unserved_customer_count": summary["unserved_customer_count"],
766         }
767     ]).to_csv(out_dir / "solution_summary.csv", index=False, encoding="utf-8-sig")
768
769     type_rows = []
770     for tid, x in sorted(summary["type_summary"].items()):
771         type_rows.append({
772             "type_id": tid,
773             "powertrain": x["powertrain"],
774             "vehicles_used": x["vehicle_count"],
775             "distance": x["distance"],
776             "delivered_weight": x["weight"],
777             "delivered_volume": x["volume"],
778             "fixed_cost": x["fixed_cost"],

```

```

777         "energy_cost": x["energy_cost"],
778         "carbon_cost": x["carbon_cost"],
779         "time_penalty_cost": x["time_penalty_cost"],
780         "total_cost": x["total_cost"],
781     })
782 pd.DataFrame(type_rows).to_csv(out_dir / "vehicle_type_summary.csv", index=False, encoding="utf-8-sig")
783
784 route_rows = []
785 detail_rows = []
786 for rid, route in enumerate(solution.routes, start=1):
787     vt = route.vehicle.vehicle_type
788     path_text = "0 -> " + " -> ".join(v.customer_id for v in route.visits) + " -> 0"
789     route_rows.append({
790         "route_id": rid,
791         "vehicle_id": route.vehicle.vehicle_id,
792         "type_id": vt.type_id,
793         "powertrain": vt.powertrain,
794         "customer_count": len(route.visits),
795         "total_weight": route.total_weight(),
796         "total_volume": route.total_volume(),
797         "weight_load_ratio": route.total_weight() / vt.max_weight,
798         "volume_load_ratio": route.total_volume() / vt.max_volume,
799         "distance": route.metrics.distance,
800         "travel_time": route.metrics.travel_time,
801         "waiting_time": route.metrics.waiting_time,
802         "late_time": route.metrics.late_time,
803         "fixed_cost": route.metrics.fixed_cost,
804         "energy_cost": route.metrics.energy_cost,
805         "carbon_cost": route.metrics.carbon_cost,
806         "time_penalty_cost": route.metrics.time_penalty_cost,
807         "route_total_cost": route.metrics.total_cost,
808         "return_time": hour_to_text(route.metrics.return_time),
809         "path": path_text,
810     })
811
812     for idx, visit in enumerate(route.visits, start=1):
813         detail_rows.append({
814             "route_id": rid,
815             "stop_order": idx,
816             "vehicle_id": route.vehicle.vehicle_id,
817             "type_id": vt.type_id,
818             "customer_id": visit.customer_id,
819             "deliver_weight": visit.deliver_weight,
820             "deliver_volume": visit.deliver_volume,
821             "arrival_time": hour_to_text(route.metrics.arrival_times[idx - 1]),
822             "service_start_time": hour_to_text(route.metrics.service_start_times[idx - 1]),
823             "leave_time": hour_to_text(route.metrics.leave_times[idx - 1]),
824             "leg_distance": route.metrics.leg_distances[idx - 1],
825             "leg_energy": route.metrics.leg_energy[idx - 1],
826         })
827
828 pd.DataFrame(route_rows).to_csv(out_dir / "route_summary.csv", index=False, encoding="utf-8-sig")
829 pd.DataFrame(detail_rows).to_csv(out_dir / "route_details.csv", index=False, encoding="utf-8-sig")
830
831 plot_vehicle_usage(summary, out_dir / "vehicle_usage.png")
832 plot_cost_breakdown(summary, out_dir / "cost_breakdown.png")
833 plot_all_routes(solution, data, out_dir / "all_routes.png")
834
835
836 def plot_vehicle_usage(summary: Dict, save_path: Path) -> None:
837     df = pd.DataFrame([
838         {"type_id": tid, "vehicles_used": x["vehicle_count"]}
839         for tid, x in sorted(summary["type_summary"].items())
840     ])
841
842     plt.figure(figsize=(8, 5))
843     plt.bar(df["type_id"], df["vehicles_used"], color="#2563eb")
844     plt.title("Vehicle Usage by Type")
845     plt.xlabel("Vehicle Type")
846     plt.ylabel("Used Vehicles")
847     plt.grid(axis="y", alpha=0.25)
848     plt.tight_layout()
849     plt.savefig(save_path, dpi=180)

```

```

850     plt.close()
851
852
853 def plot_cost_breakdown(summary: Dict, save_path: Path) -> None:
854     labels = ["Fixed", "Energy", "Carbon", "TimePenalty"]
855     values = [
856         summary["fixed_cost"],
857         summary["energy_cost"],
858         summary["carbon_cost"],
859         summary["time_penalty_cost"],
860     ]
861     colors = ["#2563eb", "#0f766e", "#ea580c", "#dc2626"]
862
863     plt.figure(figsize=(8, 5))
864     plt.bar(labels, values, color=colors)
865     plt.title("Cost Breakdown")
866     plt.ylabel("Cost")
867     plt.grid(axis="y", alpha=0.25)
868     plt.tight_layout()
869     plt.savefig(save_path, dpi=180)
870     plt.close()
871
872
873 def plot_all_routes(solution: Solution, data: ProblemData, save_path: Path) -> None:
874     plt.figure(figsize=(12, 10))
875     colors = plt.cm.tab20(np.linspace(0, 1, max(20, len(solution.routes))))
876
877     for i, route in enumerate(solution.routes):
878         nodes = [data.depot_id] + [v.customer_id for v in route.visits] + [data.depot_id]
879         xs = []
880         ys = []
881         for cid in nodes:
882             if cid == data.depot_id:
883                 xs.append(data.depot_x)
884                 ys.append(data.depot_y)
885             else:
886                 c = data.customers[cid]
887                 xs.append(c.x)
888                 ys.append(c.y)
889             plt.plot(xs, ys, "-", color=colors[i % len(colors)], alpha=0.7, linewidth=1.5)
890
891     gx = [c.x for c in data.customers.values() if c.green_zone]
892     gy = [c.y for c in data.customers.values() if c.green_zone]
893     ox = [c.x for c in data.customers.values() if not c.green_zone]
894     oy = [c.y for c in data.customers.values() if not c.green_zone]
895
896     if ox:
897         plt.scatter(ox, oy, s=18, c="#334155", label="Customers")
898     if gx:
899         plt.scatter(gx, gy, s=26, c="#16a34a", label="Green Zone Customers")
900     plt.scatter([data.depot_x], [data.depot_y], s=90, c="#dc2626", marker="*", label="Depot")
901
902     plt.title("All Vehicle Routes")
903     plt.xlabel("X")
904     plt.ylabel("Y")
905     plt.legend()
906     plt.grid(alpha=0.2)
907     plt.tight_layout()
908     plt.savefig(save_path, dpi=220)
909     plt.close()
910
911
912
913
914 def main():
915     parser = argparse.ArgumentParser(description="Green logistics solver with pandas/numpy/matplotlib")
916     parser.add_argument("--config", default="config.json")
917     parser.add_argument("--iterations", type=int, default=600)
918     parser.add_argument("--restarts", type=int, default=6)
919     parser.add_argument("--destroy-min", type=int, default=3)
920     parser.add_argument("--destroy-max", type=int, default=8)
921     parser.add_argument("--output-dir", default="outputs")
922     args = parser.parse_args()

```

```

923
924     config_path = Path(args.config).resolve()
925     data = load_problem(config_path)
926     solver = GreenLogisticsSolver(data)
927     solution = solver.solve(
928         iterations=args.iterations,
929         restarts=args.restarts,
930         destroy_min=args.destroy_min,
931         destroy_max=args.destroy_max,
932     )
933
934     out_dir = Path(args.output_dir).resolve()
935     write_outputs(solution, data, out_dir)
936
937     summary = summarize_solution(solution)
938     print(f"Best total cost: {summary['total_cost']:.4f}")
939     print(f"Routes used: {summary['route_count']}")
940     print(f"Total distance: {summary['total_distance']:.2f}")
941     print("Vehicle usage by type:")
942     for tid, x in sorted(summary["type_summary"].items()):
943         print(f"    {tid}: {int(x['vehicle_count'])}")
944     print(f"Unserved customers: {summary['unserved_customer_count']}")
945
946
947 if __name__ == "__main__":
948     main()

```

Listing 2: problem1.py

A.3 问题一与问题二统一求解程序

```

1 from __future__ import annotations
2
3 import argparse
4 import copy
5 import math
6 import random
7 from dataclasses import dataclass
8 from pathlib import Path
9 from typing import Dict, Iterable, List, Optional, Tuple
10
11 import matplotlib
12
13 matplotlib.use("Agg")
14 import matplotlib.pyplot as plt
15 import numpy as np
16 import pandas as pd
17
18
19 # =====
20 # Config: edit here if needed
21 # =====
22 DATA_DIR = Path(r"C:/Users/15826/Desktop/华中杯/处理数据")
23 CUSTOMER_FILE = DATA_DIR / "客户需求汇总结果.xlsx"
24 VEHICLE_FILE = DATA_DIR / "车型参数.xlsx"
25 DISTANCE_FILE = DATA_DIR / "距离矩阵处理结果.xlsx"
26 OUTPUT_DIR = Path("outputs_green_logistics")
27 FIXED_COST_OVERRIDE: Optional[float] = None
28
29 RANDOM_SEED = 20260425
30 ITERATIONS = 450
31 REMOVE_RATE = 0.18
32 START_TIME = 8.0
33 EARLY_PENALTY = 20.0
34 SERVICE_TIME = 20.0 / 60.0 # hour/customer, required by the statement
35 LATE_PENALTY = 50.0
36 CARBON_PRICE = 0.65 # yuan/kg CO2, required by the statement
37 DETOUR_FACTOR = 1.25
38 GREEN_RADIUS = 10.0
39 DEPOT_X = 20.0

```

```

40 DEPOT_Y = 20.0
41 MULTI_TRIP_LIMIT = 1 # each physical vehicle can be used at most once
42
43 # Green-zone fuel-vehicle ban windows for Q2 strategy 2/3.
44 BAN_WINDOWS = [(8.0, 16.0)]
45
46
47 @dataclass(frozen=True)
48 class Vehicle:
49     vid: str
50     physical_id: str
51     trip_no: int
52     vehicle_type: str
53     energy_type: str
54     capacity_weight: float
55     capacity_volume: float
56     fixed_cost: float
57     load_energy_alpha: float
58     energy_price: float
59     carbon_factor: float
60
61
62 @dataclass(frozen=True)
63 class Job:
64     jid: int
65     customer_id: int
66     weight: float
67     volume: float
68     x: float
69     y: float
70     et: float
71     lt: float
72     in_green_zone: bool
73
74
75 @dataclass
76 class Route:
77     vehicle: Vehicle
78     jobs: List[Job]
79
80
81 def setup_matplotlib() -> None:
82     plt.rcParams["font.sans-serif"] = ["SimHei", "Microsoft YaHei", "Arial Unicode MS", "DejaVu Sans"]
83     plt.rcParams["axes.unicode_minus"] = False
84     plt.rcParams["figure.dpi"] = 140
85
86
87 def fuel_energy_per_100km(speed: float) -> float:
88     return 0.0025 * speed * speed - 0.2554 * speed + 31.75
89
90
91 def electric_energy_per_100km(speed: float) -> float:
92     return 0.0014 * speed * speed - 0.12 * speed + 36.19
93
94
95 def speed_at(t: float) -> float:
96     t = t % 24.0
97     if 8.0 <= t < 9.0 or 11.5 <= t < 13.0:
98         return 9.8
99     if 9.0 <= t < 10.0 or 13.0 <= t < 15.0:
100         return 55.3
101     if 10.0 <= t < 11.5 or 15.0 <= t < 17.0:
102         return 35.4
103     return 45.0
104
105
106 def next_speed_break(t: float) -> float:
107     breaks = [8.0, 9.0, 10.0, 11.5, 13.0, 15.0, 17.0, 24.0]
108     day_t = t % 24.0
109     for b in breaks:
110         if day_t < b - 1e-9:
111             return t + (b - day_t)
112     return t + (24.0 - day_t + 8.0)

```

```

113
114
115 def in_ban_window(t: float) -> bool:
116     day_t = t % 24.0
117     return any(a <= day_t < b for a, b in BAN_WINDOWS)
118
119
120 def next_allowed_time(t: float) -> float:
121     day_t = t % 24.0
122     for a, b in BAN_WINDOWS:
123         if a <= day_t < b:
124             return t + (b - day_t)
125     return t
126
127
128 def segment_crosses_green_zone(x1: float, y1: float, x2: float, y2: float) -> bool:
129     # Distance from origin to segment <= GREEN_RADIUS.
130     vx, vy = x2 - x1, y2 - y1
131     denom = vx * vx + vy * vy
132     if denom <= 1e-12:
133         return math.hypot(x1, y1) <= GREEN_RADIUS
134     u = max(0.0, min(1.0, -(x1 * vx + y1 * vy) / denom))
135     px, py = x1 + u * vx, y1 + u * vy
136     return math.hypot(px, py) <= GREEN_RADIUS
137
138
139 def load_data() -> Tuple[Dict[int, dict], Dict[Tuple[int, int], float], List[Vehicle], pd.DataFrame]:
140     customers_df = pd.read_excel(CUSTOMER_FILE)
141     vehicles_df = pd.read_excel(VEHICLE_FILE)
142     dist_raw = pd.read_excel(DISTANCE_FILE)
143
144     dist: Dict[Tuple[int, int], float] = {}
145     for _, row in dist_raw.iterrows():
146         i = int(row["客户"])
147         for col in dist_raw.columns[1:]:
148             dist[(i, int(col))] = float(row[col])
149
150     customers: Dict[int, dict] = {
151         int(r.customer_id): {
152             "weight": float(r.total_weight),
153             "volume": float(r.total_volume),
154             "x": float(r.x),
155             "y": float(r.y),
156             "et": float(r.ET),
157             "lt": float(r.LT),
158             "green": bool(r.in_green_zone),
159         }
160         for r in customers_df.itertuples(index=False)
161     }
162     customers[0] = {
163         "weight": 0.0,
164         "volume": 0.0,
165         "x": DEPOT_X,
166         "y": DEPOT_Y,
167         "et": 0.0,
168         "lt": 24.0,
169         "green": False,
170     }
171
172     vehicles: List[Vehicle] = []
173     for row in vehicles_df.itertuples(index=False):
174         for k in range(int(row.max_num)):
175             physical_id = f"{row.vehicle_type}-{k + 1}"
176             for trip_no in range(1, MULTI_TRIP_LIMIT + 1):
177                 vehicles.append(
178                     Vehicle(
179                         vid=f"{physical_id}-trip{trip_no}",
180                         physical_id=physical_id,
181                         trip_no=trip_no,
182                         vehicle_type=str(row.vehicle_type),
183                         energy_type=str(row.energy_type),
184                         capacity_weight=float(row.capacity_weight),
185                         capacity_volume=float(row.capacity_volume),

```

```

186         fixed_cost=float(row.fixed_cost) if FIXED_COST_OVERRIDE is None else float(
FIXED_COST_OVERRIDE),
187         load_energy_alpha=float(row.load_energy_alpha),
188         energy_price=float(row.energy_price),
189         carbon_factor=float(row.carbon_factor),
190     )
191 )
192 return customers, dist, vehicles, customers_df
193
194
195 def make_jobs(customers: Dict[int, dict], vehicles: List[Vehicle], strategy: str) -> List[Job]:
196     all_vehicles = vehicles
197     electric = [v for v in vehicles if v.energy_type == "electric"]
198     jobs: List[Job] = []
199     jid = 1
200     for cid, c in customers.items():
201         if cid == 0:
202             continue
203         allowed = electric if strategy == "green_priority" and c["green"] else all_vehicles
204         # Split demand at a medium-vehicle scale. This avoids two extremes:
205         # too-small chunks create too many routes, while max-capacity chunks
206         # can only be served by the largest trucks and may exhaust them.
207         unique_w = sorted({v.capacity_weight for v in allowed})
208         unique_v = sorted({v.capacity_volume for v in allowed})
209         split_w = unique_w[(len(unique_w) - 1) // 2]
210         split_v = unique_v[(len(unique_v) - 1) // 2]
211         parts = max(1, math.ceil(c["weight"] / split_w), math.ceil(c["volume"] / split_v))
212         for _ in range(parts):
213             jobs.append(
214                 Job(
215                     jid=jid,
216                     customer_id=cid,
217                     weight=c["weight"] / parts,
218                     volume=c["volume"] / parts,
219                     x=c["x"],
220                     y=c["y"],
221                     et=c["et"],
222                     lt=c["lt"],
223                     in_green_zone=c["green"],
224                 )
225             )
226             jid += 1
227     return jobs
228
229
230 def vehicle_allowed(vehicle: Vehicle, job: Job, strategy: str) -> bool:
231     if strategy == "green_priority" and job.in_green_zone and vehicle.energy_type == "fuel":
232         return False
233     return job.weight <= vehicle.capacity_weight + 1e-9 and job.volume <= vehicle.capacity_volume + 1e-9
234
235
236 def route_load(route: Route) -> Tuple[float, float]:
237     return sum(j.weight for j in route.jobs), sum(j.volume for j in route.jobs)
238
239
240 def distance_between(i: int, j: int, dist: Dict[Tuple[int, int], float]) -> float:
241     return float(dist.get((i, j), dist.get((j, i), 0.0)))
242
243
244 def travel_segments(
245     from_id: int,
246     to_job: Optional[Job],
247     depart: float,
248     vehicle: Vehicle,
249     customers: Dict[int, dict],
250     dist: Dict[Tuple[int, int], float],
251     strategy: str,
252 ) -> Tuple[float, float, List[Tuple[float, float, float]]]:
253     to_id = 0 if to_job is None else to_job.customer_id
254     base_dist = distance_between(from_id, to_id, dist)
255
256     c1, c2 = customers[from_id], customers[to_id]
257     actual_dist = base_dist

```

```

258     if strategy == "detour" and vehicle.energy_type == "fuel" and in_ban_window(depart):
259         if segment_crosses_green_zone(c1["x"], c1["y"], c2["x"], c2["y"]):
260             actual_dist *= DETOUR_FACTOR
261
262     remain = actual_dist
263     t = depart
264     segments: List[Tuple[float, float, float]] = []
265     while remain > 1e-9:
266         v = speed_at(t)
267         boundary = next_speed_break(t)
268         can_go = max(0.0, (boundary - t) * v)
269         d = min(remain, can_go if can_go > 1e-9 else remain)
270         dt = d / v
271         segments.append((d, v, t))
272         remain -= d
273         t += dt
274         if dt <= 1e-9:
275             t = boundary
276     return t - depart, actual_dist, segments
277
278
279 def energy_for_segments(vehicle: Vehicle, segments: Iterable[Tuple[float, float, float]], load_ratio: float
280 ) -> float:
281     total = 0.0
282     for d, v, _ in segments:
283         base = fuel_energy_per_100km(v) if vehicle.energy_type == "fuel" else electric_energy_per_100km(v)
284         total += d / 100.0 * base * (1.0 + vehicle.load_energy_alpha * load_ratio)
285     return total
286
287 def evaluate_solution(
288     routes: List[Route],
289     customers: Dict[int, dict],
290     dist: Dict[Tuple[int, int], float],
291     strategy: str,
292 ) -> dict:
293     fixed = energy_cost = carbon_cost = early_cost = late_cost = 0.0
294     total_distance = total_energy = total_co2 = total_late = satisfied = served_visits = 0.0
295     route_details = []
296
297     used_physical = {r.vehicle.physical_id for r in routes if r.jobs}
298     fixed = sum(next(r.vehicle.fixed_cost for r in routes if r.jobs and r.vehicle.physical_id == pid) for
299                 pid in used_physical)
300
301     for route in routes:
302         if not route.jobs:
303             continue
304         cur_id = 0
305         cur_t = START_TIME
306         load_w, _ = route_load(route)
307         max_load = max(route.vehicle.capacity_weight, 1e-9)
308         stops = []
309         route_distance = route_energy = route_co2 = 0.0
310         for job in route.jobs:
311             if strategy == "time_shift" and route.vehicle.energy_type == "fuel" and job.in_green_zone:
312                 cur_t = next_allowed_time(cur_t)
313                 load_ratio = max(0.0, min(1.0, load_w / max_load))
314                 travel_t, arc_d, segs = travel_segments(cur_id, job, cur_t, route.vehicle, customers, dist,
315                                                         strategy)
316                 arc_energy = energy_for_segments(route.vehicle, segs, load_ratio)
317                 arrive = cur_t + travel_t
318                 wait = max(0.0, job.et - arrive)
319                 late = max(0.0, arrive - job.lt)
320                 start_service = max(arrive, job.et)
321
322                 early_cost += EARLY_PENALTY * wait
323                 late_cost += LATE_PENALTY * late
324                 energy_cost += arc_energy * route.vehicle.energy_price
325                 co2 = arc_energy * route.vehicle.carbon_factor
326                 carbon_cost += co2 * CARBON_PRICE
327                 route_distance += arc_d
328                 route_energy += arc_energy
329                 route_co2 += co2

```

```

328         total_distance += arc_d
329         total_energy += arc_energy
330         total_co2 += co2
331         total_late += late
332         satisfied += 1 if late <= 1e-9 else 0
333         served_visits += 1
334
335         stops.append(
336             {
337                 "customer_id": job.customer_id,
338                 "job_id": job.jid,
339                 "arrival": arrive,
340                 "start_service": start_service,
341                 "late": late,
342                 "distance_from_prev": arc_d,
343             }
344         )
345         cur_t = start_service + SERVICE_TIME
346         load_w -= job.weight
347         cur_id = job.customer_id
348
349         travel_t, arc_d, segs = travel_segments(cur_id, None, cur_t, route.vehicle, customers, dist,
350 strategy)
351         arc_energy = energy_for_segments(route.vehicle, segs, 0.0)
352         energy_cost += arc_energy * route.vehicle.energy_price
353         co2 = arc_energy * route.vehicle.carbon_factor
354         carbon_cost += co2 * CARBON_PRICE
355         total_distance += arc_d
356         total_energy += arc_energy
357         total_co2 += co2
358         route_distance += arc_d
359         route_energy += arc_energy
360         route_co2 += co2
361
362         route_details.append(
363             {
364                 "vehicle_id": route.vehicle.vid,
365                 "physical_vehicle_id": route.vehicle.physical_id,
366                 "trip_no": route.vehicle.trip_no,
367                 "vehicle_type": route.vehicle.vehicle_type,
368                 "energy_type": route.vehicle.energy_type,
369                 "distance": route_distance,
370                 "energy": route_energy,
371                 "co2": route_co2,
372                 "weight": sum(j.weight for j in route.jobs),
373                 "volume": sum(j.volume for j in route.jobs),
374                 "stops": stops,
375             }
376         )
377
378         penalty = early_cost + late_cost
379         total = fixed + energy_cost + carbon_cost + penalty
380         return {
381             "total_cost": total,
382             "fixed_cost": fixed,
383             "energy_cost": energy_cost,
384             "carbon_cost": carbon_cost,
385             "time_penalty": penalty,
386             "total_distance": total_distance,
387             "total_energy": total_energy,
388             "total_co2": total_co2,
389             "avg_late": total_late / max(served_visits, 1.0),
390             "time_window_satisfaction": satisfied / max(served_visits, 1.0),
391             "route_count": len([r for r in routes if r.jobs]),
392             "used_vehicle_count": len(used_physical),
393             "electric_vehicle_count": len({r.vehicle.physical_id for r in routes if r.jobs and r.vehicle.
394 energy_type == "electric"}),
395             "fuel_vehicle_count": len({r.vehicle.physical_id for r in routes if r.jobs and r.vehicle.
396 energy_type == "fuel"}),
397             "electric_vehicle_rate": len({r.vehicle.physical_id for r in routes if r.jobs and r.vehicle.
398 energy_type == "electric"})
399 / max(len(used_physical), 1),
400             "route_details": route_details,

```

```

397     }
398
399
400 def feasible_insert(route: Route, job: Job, strategy: str) -> bool:
401     if not vehicle_allowed(route.vehicle, job, strategy):
402         return False
403     w, v = route_load(route)
404     return w + job.weight <= route.vehicle.capacity_weight + 1e-9 and v + job.volume <= route.vehicle.
        capacity_volume + 1e-9
405
406
407 def insert_cheapest(
408     routes: List[Route],
409     job: Job,
410     unused: List[Vehicle],
411     customers: Dict[int, dict],
412     dist: Dict[Tuple[int, int], float],
413     strategy: str,
414 ) -> None:
415     best = None
416
417     for ri, route in enumerate(routes):
418         if not feasible_insert(route, job, strategy):
419             continue
420         for pos in range(len(route.jobs) + 1):
421             prev_id = 0 if pos == 0 else route.jobs[pos - 1].customer_id
422             next_id = 0 if pos == len(route.jobs) else route.jobs[pos].customer_id
423             score = (
424                 distance_between(prev_id, job.customer_id, dist)
425                 + distance_between(job.customer_id, next_id, dist)
426                 - distance_between(prev_id, next_id, dist)
427             )
428             score += 20.0 * max(0.0, job.et - route.jobs[pos - 1].lt) if pos > 0 else 0.0
429             if strategy == "green_priority" and (not job.in_green_zone) and route.vehicle.energy_type == "
electric":
430                 score += 10000.0
431                 if best is None or score < best[0]:
432                     best = (score, "existing", ri, pos, None)
433
434     for vi, vehicle in enumerate(unused):
435         if not vehicle_allowed(vehicle, job, strategy):
436             continue
437         used_physical = {r.vehicle.physical_id for r in routes if r.jobs}
438         fixed_delta = 0.0 if vehicle.physical_id in used_physical else vehicle.fixed_cost
439         residual_w = (vehicle.capacity_weight - job.weight) / max(vehicle.capacity_weight, 1e-9)
440         residual_v = (vehicle.capacity_volume - job.volume) / max(vehicle.capacity_volume, 1e-9)
441         capacity_tightness = residual_w + residual_v
442         score = fixed_delta + 1000.0 * capacity_tightness + 0.15 * distance_between(0, job.customer_id,
dist)
443         if vehicle.physical_id in used_physical:
444             score -= 80.0
445         if strategy == "green_priority" and job.in_green_zone and vehicle.energy_type == "electric":
446             score -= 50.0
447         if strategy == "green_priority" and (not job.in_green_zone) and vehicle.energy_type == "electric":
448             score += 10000.0
449         if best is None or score < best[0]:
450             best = (score, "new", None, None, vi)
451
452     if best is None:
453         raise RuntimeError(f"No feasible vehicle for job {job.jid}, customer {job.customer_id}")
454     _, kind, ri, pos, vi = best
455     if kind == "existing":
456         routes[ri].jobs.insert(pos, job)
457     else:
458         vehicle = unused.pop(int(vi))
459         routes.append(Route(vehicle=vehicle, jobs=[job]))
460
461
462 def build_initial_solution(
463     jobs: List[Job],
464     vehicles: List[Vehicle],
465     customers: Dict[int, dict],
466     dist: Dict[Tuple[int, int], float],

```

```

467     strategy: str,
468 ) -> Tuple[List[Route], List[Vehicle]]:
469     vehicle_order = sorted(
470         vehicles,
471         key=lambda v: (
472             0 if v.energy_type == "electric" and strategy == "green_priority" else 1,
473             v.capacity_weight,
474             v.capacity_volume,
475             v.trip_no,
476         ),
477     )
478     unused = vehicle_order[:]
479     routes: List[Route] = []
480     if strategy == "green_priority":
481         job_order = sorted(jobs, key=lambda j: (not j.in_green_zone, -(j.weight / 3000.0 + j.volume / 15.0)
482         , j.et))
483     else:
484         job_order = sorted(jobs, key=lambda j: (-(j.weight / 3000.0 + j.volume / 15.0), j.et, not j.
485         in_green_zone))
486     for job in job_order:
487         insert_cheapest(routes, job, unused, customers, dist, strategy)
488     return routes, unused
489
490 def normalize_routes(routes: List[Route]) -> List[Route]:
491     return [r for r in routes if r.jobs]
492
493 def alns_solve(
494     jobs: List[Job],
495     vehicles: List[Vehicle],
496     customers: Dict[int, dict],
497     dist: Dict[Tuple[int, int], float],
498     strategy: str,
499     iterations: int,
500 ) -> List[Route]:
501     routes, unused = build_initial_solution(jobs, vehicles, customers, dist, strategy)
502     best_routes = copy.deepcopy(routes)
503     cur_routes = copy.deepcopy(routes)
504     cur_unused = copy.deepcopy(unused)
505     best_score = evaluate_solution(best_routes, customers, dist, strategy)["total_cost"]
506     cur_score = best_score
507     temp = max(best_score * 0.03, 1.0)
508
509     for it in range(iterations):
510         trial_routes = copy.deepcopy(cur_routes)
511         trial_unused = copy.deepcopy(cur_unused)
512         all_jobs = [(ri, ji, job) for ri, r in enumerate(trial_routes) for ji, job in enumerate(r.jobs)]
513         remove_n = max(1, int(len(all_jobs) * REMOVE_RATE))
514         if random.random() < 0.5:
515             seed_job = random.choice(all_jobs)[2]
516             all_jobs.sort(key=lambda x: distance_between(seed_job.customer_id, x[2].customer_id, dist))
517             removed_refs = all_jobs[:remove_n]
518         else:
519             removed_refs = random.sample(all_jobs, remove_n)
520
521         removed_refs.sort(key=lambda x: (x[0], x[1]), reverse=True)
522         removed_jobs: List[Job] = []
523         for ri, ji, _ in removed_refs:
524             if ri < len(trial_routes) and ji < len(trial_routes[ri].jobs):
525                 removed_jobs.append(trial_routes[ri].jobs.pop(ji))
526
527         empty = [r for r in trial_routes if not r.jobs]
528         trial_unused.extend([r.vehicle for r in empty])
529         trial_routes = normalize_routes(trial_routes)
530
531         random.shuffle(removed_jobs)
532         try:
533             for job in removed_jobs:
534                 insert_cheapest(trial_routes, job, trial_unused, customers, dist, strategy)
535         except RuntimeError:
536             continue
537

```

```

538     trial_score = evaluate_solution(trial_routes, customers, dist, strategy)["total_cost"]
539     accept = trial_score < cur_score or random.random() < math.exp(-(trial_score - cur_score) / max(
temp, 1e-9))
540     if accept:
541         cur_routes, cur_unused, cur_score = trial_routes, trial_unused, trial_score
542     if trial_score < best_score:
543         best_routes, best_score = copy.deepcopy(trial_routes), trial_score
544     temp *= 0.995
545     return normalize_routes(best_routes)
546
547
548 def save_results(
549     scenario_name: str,
550     routes: List[Route],
551     metrics: dict,
552     customers_df: pd.DataFrame,
553     customers: Dict[int, dict],
554     out_dir: Path,
555 ) -> None:
556     scenario_dir = out_dir / scenario_name
557     scenario_dir.mkdir(parents=True, exist_ok=True)
558
559     route_rows = []
560     stop_rows = []
561     for idx, rd in enumerate(metrics["route_details"], start=1):
562         route_rows.append({k: v for k, v in rd.items() if k != "stops"} | {"route_no": idx})
563         for seq, stop in enumerate(rd["stops"], start=1):
564             stop_rows.append(
565                 {
566                     "route_no": idx,
567                     "vehicle_id": rd["vehicle_id"],
568                     "vehicle_type": rd["vehicle_type"],
569                     "energy_type": rd["energy_type"],
570                     "seq": seq,
571                     **stop,
572                 }
573             )
574
575     pd.DataFrame(route_rows).to_csv(scenario_dir / "routes_summary.csv", index=False, encoding="utf-8-sig")
576     pd.DataFrame(stop_rows).to_csv(scenario_dir / "stops_detail.csv", index=False, encoding="utf-8-sig")
577     pd.DataFrame([{"k": v for k, v in metrics.items() if k != "route_details"}]).to_csv(
578         scenario_dir / "metrics.csv", index=False, encoding="utf-8-sig"
579     )
580
581     plot_routes(scenario_name, routes, customers, scenario_dir / "route_map.png")
582     plot_cost_breakdown(scenario_name, metrics, scenario_dir / "cost_breakdown.png")
583     plot_vehicle_structure(scenario_name, metrics, scenario_dir / "vehicle_structure.png")
584
585
586 def plot_routes(scenario_name: str, routes: List[Route], customers: Dict[int, dict], path: Path) -> None:
587     fig, ax = plt.subplots(figsize=(10, 8))
588     theta = np.linspace(0, 2 * np.pi, 300)
589     ax.plot(GREEN_RADIUS * np.cos(theta), GREEN_RADIUS * np.sin(theta), "--", color="#1f77b4", lw=1.6,
label="绿色配送区")
590     green_x, green_y, normal_x, normal_y = [], [], [], []
591     for cid, c in customers.items():
592         if cid == 0:
593             continue
594         if c["green"]:
595             green_x.append(c["x"])
596             green_y.append(c["y"])
597         else:
598             normal_x.append(c["x"])
599             normal_y.append(c["y"])
600     ax.scatter(normal_x, normal_y, s=24, c="#9ecae1", edgecolors="#2171b5", linewidths=0.35, label="普通客
户", zorder=3)
601     ax.scatter(green_x, green_y, s=34, c="#3182bd", edgecolors="#084594", linewidths=0.45, label="绿区客户"
, zorder=4)
602     ax.scatter([0], [0], marker="*", s=180, c="#08306b", label="配送中心", zorder=5)
603
604     colors = plt.cm.Blues(np.linspace(0.35, 0.95, min(len(routes), 20)))
605     for i, route in enumerate(routes[:35]):
606         ids = [0] + [j.customer_id for j in route.jobs] + [0]

```

```

607     xs = [customers[cid]["x"] for cid in ids]
608     ys = [customers[cid]["y"] for cid in ids]
609     ls = "-" if route.vehicle.energy_type == "electric" else ":"
610     ax.plot(xs, ys, lw=0.95, alpha=0.72, color=colors[i % len(colors)], linestyle=ls)
611 ax.set_title(f"{scenario_name} 路径空间分布")
612 ax.set_xlabel("x / km")
613 ax.set_ylabel("y / km")
614 ax.legend(loc="best", fontsize=8)
615 ax.grid(alpha=0.25)
616 ax.set_aspect("equal", adjustable="box")
617 fig.tight_layout()
618 fig.savefig(path)
619 plt.close(fig)
620
621
622 def plot_cost_breakdown(scenario_name: str, m: dict, path: Path) -> None:
623     labels = ["固定成本", "能源成本", "碳排放成本", "时间窗惩罚"]
624     values = [m["fixed_cost"], m["energy_cost"], m["carbon_cost"], m["time_penalty"]]
625     fig, ax = plt.subplots(figsize=(7.4, 4.8))
626     bars = ax.bar(labels, values, color=["#08306b", "#2171b5", "#6baed6", "#c6dbef"])
627     ax.set_title(f"{scenario_name} 成本构成")
628     ax.set_ylabel("成本 / 元")
629     for b in bars:
630         ax.text(b.get_x() + b.get_width() / 2, b.get_height(), f"{b.get_height():.0f}", ha="center", va="
        bottom", fontsize=9)
631     fig.tight_layout()
632     fig.savefig(path)
633     plt.close(fig)
634
635
636 def plot_vehicle_structure(scenario_name: str, m: dict, path: Path) -> None:
637     labels = ["燃油车", "新能源车"]
638     values = [m["fuel_vehicle_count"], m["electric_vehicle_count"]]
639     fig, ax = plt.subplots(figsize=(5.6, 4.6))
640     ax.pie(values, labels=labels, autopct="%1.1f%%", colors=["#6baed6", "#08306b"], startangle=90)
641     ax.set_title(f"{scenario_name} 车辆使用结构")
642     fig.tight_layout()
643     fig.savefig(path)
644     plt.close(fig)
645
646
647 def plot_scenario_comparison(summary: pd.DataFrame, out_dir: Path) -> None:
648     name_map = {
649         "q1_base": "问题一无政策",
650         "q2_green_priority": "策略一新能源优先",
651         "q2_time_shift": "策略二时空错位",
652         "q2_detour": "策略三路径绕行",
653     }
654     summary = summary.copy()
655     summary["label"] = summary["scenario"].map(name_map).fillna(summary["scenario"])
656
657     fig, axes = plt.subplots(2, 2, figsize=(12, 8))
658     items = [
659         ("total_cost", "总成本 / 元"),
660         ("total_co2", "碳排放总量 / kg"),
661         ("total_distance", "总里程 / km"),
662         ("electric_vehicle_rate", "新能源车使用率"),
663     ]
664     colors = ["#08306b", "#2171b5", "#6baed6", "#c6dbef"]
665     for ax, (col, title) in zip(axes.ravel(), items):
666         bars = ax.bar(summary["label"], summary[col], color=colors)
667         ax.set_title(title)
668         ax.tick_params(axis="x", rotation=18)
669         for b in bars:
670             txt = f"{b.get_height():.2f}" if col != "electric_vehicle_rate" else f"{b.get_height():.1%}"
671             ax.text(b.get_x() + b.get_width() / 2, b.get_height(), txt, ha="center", va="bottom", fontsize
        =8)
672     fig.tight_layout()
673     fig.savefig(out_dir / "scenario_metrics_compare.png")
674     plt.close(fig)
675
676     cost_cols = ["fixed_cost", "energy_cost", "carbon_cost", "time_penalty"]
677     cost_labels = ["固定", "能源", "碳排", "时间惩罚"]

```

```

678 fig, ax = plt.subplots(figsize=(10, 5.4))
679 bottom = np.zeros(len(summary))
680 stack_colors = ["#08306b", "#2171b5", "#6baed6", "#c6dbef"]
681 for col, label, color in zip(cost_cols, cost_labels, stack_colors):
682     ax.bar(summary[label], summary[col], bottom=bottom, label=label, color=color)
683     bottom += summary[col].to_numpy()
684 ax.set_title("各场景成本构成堆叠对比")
685 ax.set_ylabel("成本 / 元")
686 ax.tick_params(axis="x", rotation=18)
687 ax.legend(ncol=4)
688 fig.tight_layout()
689 fig.savefig(out_dir / "scenario_cost_stack.png")
690 plt.close(fig)
691
692
693 def run() -> None:
694     parser = argparse.ArgumentParser(description="Q1/Q2 green logistics routing heuristic and visualization")
695     parser.add_argument("--iterations", type=int, default=ITERATIONS)
696     parser.add_argument("--output", type=Path, default=OUTPUT_DIR)
697     parser.add_argument("--fixed-cost", type=float, default=None, help="Override vehicle startup fixed cost")
698     args = parser.parse_args()
699
700     global FIXED_COST_OVERRIDE
701     FIXED_COST_OVERRIDE = args.fixed_cost
702
703     random.seed(RANDOM_SEED)
704     np.random.seed(RANDOM_SEED)
705     setup_matplotlib()
706
707     out_dir = args.output
708     out_dir.mkdir(parents=True, exist_ok=True)
709
710     customers, dist, vehicles, customers_df = load_data()
711     scenarios = [
712         ("q1_base", "base"),
713         ("q2_green_priority", "green_priority"),
714         ("q2_time_shift", "time_shift"),
715         ("q2_detour", "detour"),
716     ]
717     rows = []
718     for scenario_name, strategy in scenarios:
719         print(f"Solving {scenario_name} ...")
720         jobs = make_jobs(customers, vehicles, strategy)
721         routes = alns_solve(jobs, vehicles, customers, dist, strategy, args.iterations)
722         metrics = evaluate_solution(routes, customers, dist, strategy)
723         save_results(scenario_name, routes, metrics, customers_df, customers, out_dir)
724         row = {k: v for k, v in metrics.items() if k != "route_details"}
725         row["scenario"] = scenario_name
726         row["strategy"] = strategy
727         rows.append(row)
728         print(
729             f"  cost={metrics['total_cost']:.2f}, co2={metrics['total_co2']:.2f}, "
730             f"distance={metrics['total_distance']:.2f}, EV rate={metrics['electric_vehicle_rate']:.2%}"
731         )
732
733     summary = pd.DataFrame(rows)
734     order_cols = ["scenario", "strategy"] + [c for c in summary.columns if c not in ("scenario", "strategy")]
735     summary = summary[order_cols]
736     summary.to_csv(out_dir / "scenario_summary.csv", index=False, encoding="utf-8-sig")
737     plot_scenario_comparison(summary, out_dir)
738     print(f"\nDone. Results saved to: {out_dir.resolve()}")
739
740
741 if __name__ == "__main__":
742     run()

```

Listing 3: green_logistics_q1_q2.py

A.4 问题一综合可视化程序

```
1 from __future__ import annotations
2
3 from pathlib import Path
4 import argparse
5
6 import matplotlib
7
8 matplotlib.use("Agg")
9 import matplotlib.pyplot as plt
10 import numpy as np
11 import pandas as pd
12
13
14 ROOT = Path(r"C:/Users/15826/Documents/Codex/2026-04-25/files-mentioned-by-the-user-a")
15 DEFAULT_RESULT_DIR = ROOT / "outputs_green_logistics_blue" / "q1_base"
16 CUSTOMER_FILE = Path(r"C:/Users/15826/Desktop/华中杯/处理数据/客户需求汇总结果.xlsx")
17 GREEN_RADIUS = 10.0
18 DEPOT_X = 20.0
19 DEPOT_Y = 20.0
20
21
22 def setup_matplotlib() -> None:
23     plt.rcParams["font.sans-serif"] = ["SimHei", "Microsoft YaHei", "Arial Unicode MS", "DejaVu Sans"]
24     plt.rcParams["axes.unicode_minus"] = False
25     plt.rcParams["figure.dpi"] = 160
26
27
28 def load(result_dir: Path) -> tuple[pd.DataFrame, pd.DataFrame, pd.Series, pd.DataFrame]:
29     routes = pd.read_csv(result_dir / "routes_summary.csv")
30     stops = pd.read_csv(result_dir / "stops_detail.csv")
31     metrics = pd.read_csv(result_dir / "metrics.csv").iloc[0]
32     customers = pd.read_excel(CUSTOMER_FILE)
33     return routes, stops, metrics, customers
34
35
36 def make_vehicle_scheme(routes: pd.DataFrame, vehicle_scheme_path: Path) -> pd.DataFrame:
37     scheme = (
38         routes.groupby(["vehicle_type", "energy_type"], as_index=False)
39         .agg(
40             启用物理车辆数=("physical_vehicle_id", "nunique"),
41             配送趟次=("route_no", "count"),
42             总里程_km=("distance", "sum"),
43             总载重_kg=("weight", "sum"),
44             总体积_m3=("volume", "sum"),
45             碳排放_kg=("co2", "sum"),
46         )
47         .sort_values(["energy_type", "vehicle_type"])
48     )
49     scheme.to_csv(vehicle_scheme_path, index=False, encoding="utf-8-sig")
50     return scheme
51
52
53 def plot_vehicle_scheme(ax: plt.Axes, scheme: pd.DataFrame) -> None:
54     labels = scheme["vehicle_type"].astype(str).to_list()
55     vehicle_counts = scheme["启用物理车辆数"].to_numpy()
56     trips = scheme["配送趟次"].to_numpy()
57     x = np.arange(len(labels))
58     width = 0.36
59     ax.bar(x - width / 2, vehicle_counts, width, color="#08306b", label="启用车辆数")
60     ax.bar(x + width / 2, trips, width, color="#6baed6", label="配送趟次")
61     ax.set_title("车辆使用方案")
62     ax.set_xticks(x)
63     ax.set_xticklabels(labels)
64     ax.set_ylabel("数量")
65     ax.legend(frameon=False, fontsize=9)
66     ax.grid(axis="y", alpha=0.25)
67     for i, (v, t) in enumerate(zip(vehicle_counts, trips)):
68         ax.text(i - width / 2, v, f"{int(v)}", ha="center", va="bottom", fontsize=8)
69         ax.text(i + width / 2, t, f"{int(t)}", ha="center", va="bottom", fontsize=8)
70
71
```

```

72 def plot_route_map(ax: plt.Axes, routes: pd.DataFrame, stops: pd.DataFrame, customers: pd.DataFrame) ->
    None:
73     cdict = {
74         int(r.customer_id): (float(r.x), float(r.y), bool(r.in_green_zone))
75         for r in customers.itertuples(index=False)
76     }
77     cdict[0] = (DEPOT_X, DEPOT_Y, False)
78     theta = np.linspace(0, 2 * np.pi, 320)
79     ax.plot(GREEN_RADIUS * np.cos(theta), GREEN_RADIUS * np.sin(theta), "--", color="#1f77b4", lw=1.5,
        label="绿色配送区")
80
81     normal = customers[~customers["in_green_zone"]]
82     green = customers[customers["in_green_zone"]]
83     ax.scatter(normal["x"], normal["y"], s=20, c="#c6dbef", edgecolors="#2171b5", linewidths=0.3, label="普
        通客户", zorder=3)
84     ax.scatter(green["x"], green["y"], s=30, c="#3182bd", edgecolors="#08306b", linewidths=0.4, label="绿区
        客户", zorder=4)
85     ax.scatter([DEPOT_X], [DEPOT_Y], marker="*", s=170, c="#08306b", label="配送中心", zorder=5)
86
87     colors = plt.cm.Blues(np.linspace(0.35, 0.95, 20))
88     for _, route in routes.sort_values("distance", ascending=False).head(35).iterrows():
89         path = stops[stops["route_no"] == route["route_no"]].sort_values("seq")["customer_id"].astype(int).
        to_list()
90         ids = [0] + path + [0]
91         xs = [cdict[i][0] for i in ids]
92         ys = [cdict[i][1] for i in ids]
93         linestyle = "-" if route["energy_type"] == "electric" else ":"
94         ax.plot(xs, ys, color=colors[int(route["route_no"]) % 20], lw=0.9, alpha=0.75, linestyle=linestyle)
95
96     ax.set_title("行驶路径空间分布 (前35条长路线)")
97     ax.set_xlabel("x / km")
98     ax.set_ylabel("y / km")
99     ax.set_aspect("equal", adjustable="box")
100    ax.grid(alpha=0.25)
101    ax.legend(loc="upper right", fontsize=8, frameon=False)
102
103
104 def plot_arrival_time(ax: plt.Axes, stops: pd.DataFrame) -> None:
105     s = stops.copy()
106     s["is_late"] = s["late"] > 1e-9
107     ax.scatter(
108         s.loc[~s["is_late"], "arrival"],
109         s.loc[~s["is_late"], "route_no"],
110         s=10,
111         c="#6baed6",
112         alpha=0.65,
113         label="准时/等待",
114     )
115     ax.scatter(
116         s.loc[s["is_late"], "arrival"],
117         s.loc[s["is_late"], "route_no"],
118         s=16,
119         c="#08306b",
120         alpha=0.8,
121         label="延误",
122     )
123     ax.set_title("客户到达时间分布")
124     ax.set_xlabel("到达时刻 / h")
125     ax.set_ylabel("配送趟次编号")
126     ax.set_xlim(8, max(22, s["arrival"].max() + 0.5))
127     ax.grid(alpha=0.25)
128     ax.legend(frameon=False, fontsize=9)
129
130
131 def plot_cost(ax: plt.Axes, metrics: pd.Series) -> None:
132     labels = ["固定成本", "能源成本", "碳排放成本", "时间窗惩罚"]
133     values = np.array([metrics["fixed_cost"], metrics["energy_cost"], metrics["carbon_cost"], metrics["
        time_penalty"]])
134     colors = ["#08306b", "#2171b5", "#6baed6", "#c6dbef"]
135     wedges, _ = ax.pie(values, colors=colors, startangle=90, wedgeprops={"width": 0.38, "edgecolor": "white
        "})
136     ax.set_title("成本构成")
137     total = metrics["total_cost"]

```

```

138 ax.text(0, 0.08, f"{total:,.0f}", ha="center", va="center", fontsize=17, color="#08306b", fontweight="
    bold")
139 ax.text(0, -0.12, "总成本/元", ha="center", va="center", fontsize=9, color="#08306b")
140 legend_labels = [f"{l} {v:,.0f}元 ({v / values.sum():.1%})" for l, v in zip(labels, values)]
141 ax.legend(wedges, legend_labels, loc="lower center", bbox_to_anchor=(0.5, -0.18), fontsize=8, frameon=
    False)
142
143
144 def add_metric_band(fig: plt.Figure, metrics: pd.Series) -> None:
145     text = (
146         f"总成本: {metrics['total_cost']:,.2f} 元 "
147         f"启用车辆: {int(metrics['used_vehicle_count'])} 辆 "
148         f"配送趟次: {int(metrics['route_count'])} 趟 "
149         f"总里程: {metrics['total_distance']:,.2f} km "
150         f"总碳排放: {metrics['total_co2']:,.2f} kg "
151         f"时间窗满足率: {metrics['time_window_satisfaction':.1%}"
152     )
153     fig.text(0.5, 0.955, text, ha="center", va="center", fontsize=11, color="#08306b")
154
155
156 def main() -> None:
157     parser = argparse.ArgumentParser()
158     parser.add_argument("--result-dir", type=Path, default=DEFAULT_RESULT_DIR)
159     args = parser.parse_args()
160
161     result_dir = args.result_dir
162     out_path = result_dir / "q1_integrated_visualization.png"
163     vehicle_scheme_path = result_dir / "q1_vehicle_use_scheme.csv"
164     separate_dir = result_dir / "q1_separate_figures"
165
166     setup_matplotlib()
167     routes, stops, metrics, customers = load(result_dir)
168     scheme = make_vehicle_scheme(routes, vehicle_scheme_path)
169     separate_dir.mkdir(parents=True, exist_ok=True)
170
171     fig = plt.figure(figsize=(15, 10))
172     gs = fig.add_gridspec(2, 2, height_ratios=[1, 1.1], width_ratios=[1, 1.25], hspace=0.28, wspace=0.22)
173     ax_vehicle = fig.add_subplot(gs[0, 0])
174     ax_cost = fig.add_subplot(gs[0, 1])
175     ax_route = fig.add_subplot(gs[1, 0])
176     ax_arrival = fig.add_subplot(gs[1, 1])
177
178     fig.suptitle("问题一: 无政策限制下城市物流配送调度结果可视化", fontsize=18, fontweight="bold", color="
        #08306b", y=0.99)
179     add_metric_band(fig, metrics)
180     plot_vehicle_scheme(ax_vehicle, scheme)
181     plot_cost(ax_cost, metrics)
182     plot_route_map(ax_route, routes, stops, customers)
183     plot_arrival_time(ax_arrival, stops)
184
185     fig.savefig(out_path, bbox_inches="tight")
186     plt.close(fig)
187
188     separate_specs = [
189         ("q1_vehicle_use_scheme.png", (8, 5), lambda ax: plot_vehicle_scheme(ax, scheme)),
190         ("q1_cost_breakdown.png", (7, 5.4), lambda ax: plot_cost(ax, metrics)),
191         ("q1_route_map.png", (9, 7.5), lambda ax: plot_route_map(ax, routes, stops, customers)),
192         ("q1_arrival_time.png", (9, 5.4), lambda ax: plot_arrival_time(ax, stops)),
193     ]
194     for filename, figsize, draw in separate_specs:
195         fig_single, ax_single = plt.subplots(figsize=figsize)
196         draw(ax_single)
197         fig_single.savefig(separate_dir / filename, bbox_inches="tight")
198         plt.close(fig_single)
199
200     print(f"saved: {out_path}")
201     print(f"vehicle scheme: {vehicle_scheme_path}")
202     print(f"separate figures: {separate_dir}")
203
204
205 if __name__ == "__main__":
206     main()

```

Listing 4: make_q1_integrated_visualization.py

A.5 问题二政策影响可视化程序

```
1 from __future__ import annotations
2
3 from pathlib import Path
4
5 import matplotlib
6
7 matplotlib.use("Agg")
8 import matplotlib.pyplot as plt
9 import numpy as np
10 import pandas as pd
11
12
13 ROOT = Path(r"C:/Users/15826/Documents/Codex/2026-04-25/files-mentioned-by-the-user-a")
14 RESULT_DIR = ROOT / "outputs_green_logistics_depot_20_20"
15 CUSTOMER_FILE = Path(r"C:/Users/15826/Desktop/华中杯/处理数据/客户需求汇总结果.xlsx")
16 OUT_DIR = RESULT_DIR / "q2_policy_figures"
17
18 GREEN_RADIUS = 10.0
19 DEPOT_X = 20.0
20 DEPOT_Y = 20.0
21
22 SCENARIO_LABELS = {
23     "q1_base": "无政策",
24     "q2_green_priority": "新能源优先",
25     "q2_time_shift": "时空错位",
26     "q2_detour": "路径绕行",
27 }
28
29
30 def setup_matplotlib() -> None:
31     plt.rcParams["font.sans-serif"] = ["SimHei", "Microsoft YaHei", "Arial Unicode MS", "DejaVu Sans"]
32     plt.rcParams["axes.unicode_minus"] = False
33     plt.rcParams["figure.dpi"] = 160
34
35
36 def load_summary() -> pd.DataFrame:
37     df = pd.read_csv(RESULT_DIR / "scenario_summary.csv")
38     df["label"] = df["scenario"].map(SCENARIO_LABELS)
39     return df
40
41
42 def make_policy_impact_table(summary: pd.DataFrame) -> pd.DataFrame:
43     base = summary[summary["scenario"] == "q1_base"].iloc[0]
44     rows = []
45     for row in summary.itertuples(index=False):
46         if row.scenario == "q1_base":
47             continue
48         total_cost_delta = row.total_cost - base.total_cost
49         co2_delta = row.total_co2 - base.total_co2
50         ev_delta = row.electric_vehicle_rate - base.electric_vehicle_rate
51         rows.append(
52             {
53                 "策略": row.label,
54                 "总成本_元": row.total_cost,
55                 "总成本较问题一变化_元": total_cost_delta,
56                 "总成本变化率": total_cost_delta / base.total_cost,
57                 "启用车辆数": row.used_vehicle_count,
58                 "新能源车车辆数": row.electric_vehicle_count,
59                 "燃油车辆数": row.fuel_vehicle_count,
60                 "新能源车使用率": row.electric_vehicle_rate,
61                 "新能源车使用率较问题一变化": ev_delta,
62                 "总碳排放_kg": row.total_co2,
63                 "碳排放较问题一变化_kg": co2_delta,
64                 "碳排放变化率": co2_delta / base.total_co2,
65                 "碳排放成本_元": row.carbon_cost,
```

```

66         "总里程_km": row.total_distance,
67     }
68 )
69 impact = pd.DataFrame(rows)
70 impact.to_csv(OUT_DIR / "q2_policy_impact_table.csv", index=False, encoding="utf-8-sig")
71 return impact
72
73
74 def autolabel(ax: plt.Axes, bars, fmt="{:.0f}", dy=2) -> None:
75     for bar in bars:
76         h = bar.get_height()
77         ax.text(bar.get_x() + bar.get_width() / 2, h + dy, fmt.format(h), ha="center", va="bottom",
78                 fontsize=8)
79
80 def plot_total_cost(summary: pd.DataFrame) -> None:
81     fig, ax = plt.subplots(figsize=(9, 5.2))
82     colors = ["#08306b", "#2171b5", "#6baed6", "#c6dbef"]
83     bars = ax.bar(summary["label"], summary["total_cost"], color=colors, edgecolor="#08306b", linewidth
84                  =0.5)
85     ax.set_title("绿色限行政策下总成本对比")
86     ax.set_ylabel("总成本 / 元")
87     ax.grid(axis="y", alpha=0.25)
88     autolabel(ax, bars, "{:.0f}", dy=900)
89     base = summary.loc[summary["scenario"] == "q1_base", "total_cost"].iloc[0]
90     ax.axhline(base, color="#08306b", linestyle="--", linewidth=1.0, alpha=0.7)
91     ax.text(3.5, base, "问题一基准", va="bottom", ha="right", fontsize=9, color="#08306b")
92     fig.tight_layout()
93     fig.savefig(OUT_DIR / "q2_total_cost_compare.png", bbox_inches="tight")
94     plt.close(fig)
95
96 def plot_cost_stack(summary: pd.DataFrame) -> None:
97     fig, ax = plt.subplots(figsize=(10, 5.5))
98     cost_cols = ["fixed_cost", "energy_cost", "carbon_cost", "time_penalty"]
99     labels = ["固定成本", "能源成本", "碳排放成本", "时间窗惩罚"]
100    colors = ["#08306b", "#2171b5", "#6baed6", "#c6dbef"]
101    bottom = np.zeros(len(summary))
102    for col, label, color in zip(cost_cols, labels, colors):
103        ax.bar(summary["label"], summary[col], bottom=bottom, label=label, color=color, edgecolor="white",
104              linewidth=0.4)
105        bottom += summary[col].to_numpy()
106    ax.set_title("绿色限行政策下成本构成对比")
107    ax.set_ylabel("成本 / 元")
108    ax.grid(axis="y", alpha=0.25)
109    ax.legend(ncol=4, frameon=False, loc="upper center", bbox_to_anchor=(0.5, -0.08))
110    fig.tight_layout()
111    fig.savefig(OUT_DIR / "q2_cost_structure_compare.png", bbox_inches="tight")
112    plt.close(fig)
113
114 def plot_vehicle_structure(summary: pd.DataFrame) -> None:
115     fig, ax = plt.subplots(figsize=(9.5, 5.3))
116     x = np.arange(len(summary))
117     fuel = summary["fuel_vehicle_count"].to_numpy()
118     electric = summary["electric_vehicle_count"].to_numpy()
119     ax.bar(x, fuel, color="#9ecae1", edgecolor="#2171b5", label="燃油车")
120     ax.bar(x, electric, bottom=fuel, color="#08306b", edgecolor="#08306b", label="新能源车")
121     ax.set_xticks(x)
122     ax.set_xticklabels(summary["label"])
123     ax.set_title("绿色限行政策下车辆使用结构对比")
124     ax.set_ylabel("启用车辆数 / 辆")
125     ax.grid(axis="y", alpha=0.25)
126     ax.legend(frameon=False)
127     for i, row in summary.iterrows():
128         ax.text(i, row.used_vehicle_count + 1.2, f"{int(row.used_vehicle_count)}辆", ha="center", va="
129         bottom", fontsize=8)
130         ax.text(i, row.fuel_vehicle_count / 2, f"燃油{int(row.fuel_vehicle_count)}", ha="center", va="
131         center", fontsize=8, color="#08306b")
132         ax.text(
133             i,
134             row.fuel_vehicle_count + max(row.electric_vehicle_count / 2, 1),
135             f"新能源{int(row.electric_vehicle_count)}",

```

```

134         ha="center",
135         va="center",
136         fontsize=8,
137         color="white",
138     )
139     fig.tight_layout()
140     fig.savefig(OUT_DIR / "q2_vehicle_structure_compare.png", bbox_inches="tight")
141     plt.close(fig)
142
143
144 def plot_carbon(summary: pd.DataFrame) -> None:
145     fig, ax1 = plt.subplots(figsize=(9.3, 5.2))
146     x = np.arange(len(summary))
147     bars = ax1.bar(x, summary["total_co2"], color=["#08306b", "#2171b5", "#6baed6", "#c6dbef"], edgecolor="#08306b", linewidth=0.5)
148     ax1.set_xticks(x)
149     ax1.set_xticklabels(summary["label"])
150     ax1.set_ylabel("总碳排放量 / kg")
151     ax1.set_title("绿色限行政策下碳排放总量与碳成本对比")
152     ax1.grid(axis="y", alpha=0.25)
153     autolabel(ax1, bars, "{:.0f}", dy=90)
154
155     ax2 = ax1.twinx()
156     ax2.plot(x, summary["carbon_cost"], color="#08306b", marker="o", linewidth=1.8, label="碳排放成本")
157     ax2.set_ylabel("碳排放成本 / 元")
158     ax2.legend(frameon=False, loc="upper right")
159     fig.tight_layout()
160     fig.savefig(OUT_DIR / "q2_carbon_compare.png", bbox_inches="tight")
161     plt.close(fig)
162
163
164 def load_route_data(scenario: str) -> tuple[pd.DataFrame, pd.DataFrame]:
165     base = RESULT_DIR / scenario
166     return pd.read_csv(base / "routes_summary.csv"), pd.read_csv(base / "stops_detail.csv")
167
168
169 def plot_route_maps() -> None:
170     customers = pd.read_excel(CUSTOMER_FILE)
171     cdict = {
172         int(r.customer_id): (float(r.x), float(r.y), bool(r.in_green_zone))
173         for r in customers.itertuples(index=False)
174     }
175     cdict[0] = (DEPOT_X, DEPOT_Y, False)
176     scenarios = ["q1_base", "q2_green_priority", "q2_time_shift", "q2_detour"]
177
178     fig, axes = plt.subplots(2, 2, figsize=(13.5, 11))
179     theta = np.linspace(0, 2 * np.pi, 320)
180     colors = plt.cm.Blues(np.linspace(0.35, 0.95, 20))
181     normal = customers[~customers["in_green_zone"]]
182     green = customers[customers["in_green_zone"]]
183
184     for ax, scenario in zip(axes.ravel(), scenarios):
185         routes, stops = load_route_data(scenario)
186         ax.plot(GREEN_RADIUS * np.cos(theta), GREEN_RADIUS * np.sin(theta), "--", color="#1f77b4", lw=1.4)
187         ax.scatter(normal["x"], normal["y"], s=17, c="#c6dbef", edgecolors="#2171b5", linewidths=0.25, zorder=3)
188         ax.scatter(green["x"], green["y"], s=26, c="#3182bd", edgecolors="#08306b", linewidths=0.35, zorder=4)
189         ax.scatter([DEPOT_X], [DEPOT_Y], marker="*", s=150, c="#08306b", zorder=5)
190
191         for _, route in routes.sort_values("distance", ascending=False).head(28).iterrows():
192             path = stops[stops["route_no"] == route["route_no"]].sort_values("seq")["customer_id"].astype(int).to_list()
193             ids = [0] + path + [0]
194             xs = [cdict[i][0] for i in ids]
195             ys = [cdict[i][1] for i in ids]
196             linestyle = "-" if route["energy_type"] == "electric" else ":"
197             ax.plot(xs, ys, color=colors[int(route["route_no"]) % 20], lw=0.85, alpha=0.72, linestyle=linestyle)
198
199         ax.set_title(SCENARIO_LABELS[scenario])
200         ax.set_xlabel("x / km")
201         ax.set_ylabel("y / km")

```

```

202     ax.grid(alpha=0.22)
203     ax.set_aspect("equal", adjustable="box")
204
205 fig.suptitle("问题一与绿色限行政策下典型路径空间分布对比", fontsize=16, fontweight="bold", color="#08306b")
206 fig.tight_layout()
207 fig.savefig(OUT_DIR / "q2_route_maps_compare.png", bbox_inches="tight")
208 plt.close(fig)
209
210
211 def plot_dashboard(summary: pd.DataFrame) -> None:
212     fig = plt.figure(figsize=(14.5, 9.2))
213     gs = fig.add_gridspec(2, 2, hspace=0.28, wspace=0.22)
214     ax_cost = fig.add_subplot(gs[0, 0])
215     ax_vehicle = fig.add_subplot(gs[0, 1])
216     ax_co2 = fig.add_subplot(gs[1, 0])
217     ax_delta = fig.add_subplot(gs[1, 1])
218
219     colors = ["#08306b", "#2171b5", "#6baed6", "#c6dbef"]
220     ax_cost.bar(summary["label"], summary["total_cost"], color=colors)
221     ax_cost.set_title("总成本")
222     ax_cost.set_ylabel("元")
223     ax_cost.grid(axis="y", alpha=0.25)
224
225     x = np.arange(len(summary))
226     fuel = summary["fuel_vehicle_count"].to_numpy()
227     electric = summary["electric_vehicle_count"].to_numpy()
228     ax_vehicle.bar(x, fuel, color="#9ecae1", label="燃油车")
229     ax_vehicle.bar(x, electric, bottom=fuel, color="#08306b", label="新能源车")
230     ax_vehicle.set_xticks(x)
231     ax_vehicle.set_xticklabels(summary["label"])
232     ax_vehicle.set_title("车辆结构")
233     ax_vehicle.set_ylabel("辆")
234     ax_vehicle.legend(frameon=False)
235     ax_vehicle.grid(axis="y", alpha=0.25)
236
237     ax_co2.bar(summary["label"], summary["total_co2"], color=colors)
238     ax_co2.set_title("总碳排放")
239     ax_co2.set_ylabel("kg")
240     ax_co2.grid(axis="y", alpha=0.25)
241
242     base = summary[summary["scenario"] == "q1_base"].iloc[0]
243     policy = summary[summary["scenario"] != "q1_base"].copy()
244     deltas = pd.DataFrame(
245         {
246             "label": policy["label"],
247             "总成本变化率": (policy["total_cost"] - base["total_cost"]) / base["total_cost"],
248             "碳排放变化率": (policy["total_co2"] - base["total_co2"]) / base["total_co2"],
249             "新能源率变化": policy["electric_vehicle_rate"] - base["electric_vehicle_rate"],
250         }
251     )
252     width = 0.25
253     xx = np.arange(len(deltas))
254     ax_delta.bar(xx - width, deltas["总成本变化率"], width, color="#08306b", label="总成本")
255     ax_delta.bar(xx, deltas["碳排放变化率"], width, color="#2171b5", label="碳排放")
256     ax_delta.bar(xx + width, deltas["新能源率变化"], width, color="#6baed6", label="新能源率")
257     ax_delta.axhline(0, color="#08306b", linewidth=0.8)
258     ax_delta.set_xticks(xx)
259     ax_delta.set_xticklabels(deltas["label"])
260     ax_delta.set_title("相对问题一变化率")
261     ax_delta.set_ylabel("变化比例")
262     ax_delta.yaxis.set_major_formatter(lambda x, pos: f"{x:.0%}")
263     ax_delta.legend(frameon=False)
264     ax_delta.grid(axis="y", alpha=0.25)
265
266     fig.suptitle("绿色限行政策影响综合对比", fontsize=18, fontweight="bold", color="#08306b")
267     fig.tight_layout()
268     fig.savefig(OUT_DIR / "q2_policy_impact_dashboard.png", bbox_inches="tight")
269     plt.close(fig)
270
271
272 def main() -> None:
273     setup_matplotlib()

```

```

274     OUT_DIR.mkdir(parents=True, exist_ok=True)
275     summary = load_summary()
276     make_policy_impact_table(summary)
277     plot_total_cost(summary)
278     plot_cost_stack(summary)
279     plot_vehicle_structure(summary)
280     plot_carbon(summary)
281     plot_route_maps()
282     plot_dashboard(summary)
283     print(f"saved figures to: {OUT_DIR}")
284
285
286 if __name__ == "__main__":
287     main()

```

Listing 5: make_q2_policy_visualization.py

A.6 问题三动态调度主程序

```

1  from __future__ import annotations
2
3  import copy
4  import math
5  from dataclasses import dataclass
6  from pathlib import Path
7  from typing import Dict, List, Optional, Tuple
8
9  import matplotlib
10
11 matplotlib.use("Agg")
12 import matplotlib.pyplot as plt
13 import numpy as np
14 import pandas as pd
15
16 import green_logistics_q1_q2 as gl
17
18
19 ROOT = Path(r"C:/Users/15826/Documents/Codex/2026-04-25/files-mentioned-by-the-user-a")
20 BASE_RESULT_DIR = ROOT / "outputs_green_logistics_depot_20_20"
21 BASE_SCENARIO = "q2_time_shift"
22 OUT_DIR = BASE_RESULT_DIR / "q3_event_driven_figures"
23
24 T_NOW = 12.0
25 FREEZE_HORIZON = 0.50
26 ROAD_FACTOR_FOR_NEW_NODE = 1.30
27 UNSERVED_PENALTY = 9000.0
28 STABILITY_ETA = 160.0
29
30 SCENARIO_LABELS = {
31     "baseline": "突变前",
32     "hold": "保持原方案",
33     "greedy": "局部插入",
34     "rolling": "滚动重优化",
35 }
36
37
38 @dataclass
39 class DynamicEvent:
40     event_type: str
41     event_time: float
42     target_job_id: Optional[int] = None
43     new_customer_id: Optional[int] = None
44     new_x: Optional[float] = None
45     new_y: Optional[float] = None
46     new_weight: Optional[float] = None
47     new_volume: Optional[float] = None
48     new_et: Optional[float] = None
49     new_lt: Optional[float] = None
50     priority: float = 1.0
51     description: str = ""

```

```

52
53
54 @dataclass
55 class RequestState:
56     request_id: int
57     job: gl.Job
58     status: str
59     priority: float = 1.0
60     source_event: str = ""
61
62
63 @dataclass
64 class VehicleState:
65     vehicle: gl.Vehicle
66     frozen_jobs: List[gl.Job]
67     flexible_jobs: List[gl.Job]
68     is_active: bool
69     can_accept_new_order: bool
70
71
72 def setup_matplotlib() -> None:
73     plt.rcParams["font.sans-serif"] = ["SimHei", "Microsoft YaHei", "Arial Unicode MS", "DejaVu Sans"]
74     plt.rcParams["axes.unicode_minus"] = False
75     plt.rcParams["figure.dpi"] = 160
76
77
78 def euclidean_distance(a: dict, b: dict) -> float:
79     return ROAD_FACTOR_FOR_NEW_NODE * math.hypot(a["x"] - b["x"], a["y"] - b["y"])
80
81
82 def add_or_update_dynamic_node(
83     customers: Dict[int, dict],
84     dist: Dict[Tuple[int, int], float],
85     node_id: int,
86     x: float,
87     y: float,
88     et: float,
89     lt: float,
90     weight: float,
91     volume: float,
92 ) -> None:
93     customers[node_id] = {
94         "weight": weight,
95         "volume": volume,
96         "x": x,
97         "y": y,
98         "et": et,
99         "lt": lt,
100         "green": math.hypot(x, y) <= gl.GREEN_RADIUS,
101     }
102     for other_id, other in list(customers.items()):
103         d = 0.0 if other_id == node_id else euclidean_distance(customers[node_id], other)
104         dist[(node_id, other_id)] = d
105         dist[(other_id, node_id)] = d
106
107
108 def load_base_routes(customers: Dict[int, dict], vehicles: List[gl.Vehicle]) -> Tuple[List[gl.Route], Dict[
109     int, gl.Job]]:
110     routes_df = pd.read_csv(BASE_RESULT_DIR / BASE_SCENARIO / "routes_summary.csv")
111     stops_df = pd.read_csv(BASE_RESULT_DIR / BASE_SCENARIO / "stops_detail.csv")
112     jobs = gl.make_jobs(customers, vehicles, "base")
113     job_by_id = {j.jid: j for j in jobs}
114     vehicle_by_id = {v.vid: v for v in vehicles}
115
116     routes: List[gl.Route] = []
117     for row in routes_df.sort_values("route_no").itertuples(index=False):
118         route_stops = stops_df[stops_df["route_no"] == row.route_no].sort_values("seq")
119         route_jobs = [job_by_id[int(s.job_id)] for s in route_stops.itertuples(index=False)]
120         routes.append(gl.Route(vehicle=vehicle_by_id[row.vehicle_id], jobs=route_jobs))
121     return routes, job_by_id
122
123 def route_stops(route: gl.Route, customers: Dict[int, dict], dist: Dict[Tuple[int, int], float]) -> List[

```

```

dict]:
124 metrics = gl.evaluate_solution([route], customers, dist, "time_shift")
125 return metrics["route_details"][0]["stops"] if metrics["route_details"] else []
126
127
128 def split_vehicle_states(routes: List[gl.Route], customers: Dict[int, dict], dist: Dict[Tuple[int, int],
float]) -> Dict[str, VehicleState]:
129 states: Dict[str, VehicleState] = {}
130 for route in routes:
131     stops = route_stops(route, customers, dist)
132     freeze_len = 0
133     for idx, stop in enumerate(stops):
134         if stop["start_service"] <= T_NOW + FREEZE_HORIZON:
135             freeze_len = idx + 1
136         else:
137             break
138     frozen = route.jobs[:freeze_len]
139     flexible = route.jobs[freeze_len:]
140     states[route.vehicle.vid] = VehicleState(
141         vehicle=route.vehicle,
142         frozen_jobs=frozen,
143         flexible_jobs=flexible,
144         is_active=len(frozen) > 0,
145         can_accept_new_order=len(frozen) == 0,
146     )
147 return states
148
149
150 def states_to_routes(states: Dict[str, VehicleState]) -> List[gl.Route]:
151 routes = []
152 for state in states.values():
153     jobs = state.frozen_jobs + state.flexible_jobs
154     if jobs:
155         routes.append(gl.Route(state.vehicle, jobs))
156 return routes
157
158
159 def route_signature(routes: List[gl.Route]) -> Dict[str, Tuple[int, ...]]:
160 return {r.vehicle.vid: tuple(j.customer_id for j in r.jobs) for r in routes if r.jobs}
161
162
163 def stability_change(base_sig: Dict[str, Tuple[int, ...]], routes: List[gl.Route]) -> float:
164 changes = 0
165 total = 0
166 for route in routes:
167     old = base_sig.get(route.vehicle.vid, tuple())
168     new = tuple(j.customer_id for j in route.jobs)
169     total += max(len(old), len(new), 1)
170     overlap = sum(1 for a, b in zip(old, new) if a == b)
171     changes += max(len(old), len(new)) - overlap
172 return changes / max(total, 1)
173
174
175 def choose_event_jobs(routes: List[gl.Route], states: Dict[str, VehicleState]) -> Tuple[gl.Job, gl.Job, gl.
Job]:
176 flexible = [j for s in states.values() for j in s.flexible_jobs]
177 flexible = sorted(flexible, key=lambda j: (j.in_green_zone, -j.weight, j.lt))
178 return flexible[0], flexible[len(flexible) // 3], flexible[2 * len(flexible) // 3]
179
180
181 def build_events(routes: List[gl.Route], states: Dict[str, VehicleState], customers: Dict[int, dict], dist:
Dict[Tuple[int, int], float]) -> List[DynamicEvent]:
182 cancel_job, relocate_job, tw_job = choose_event_jobs(routes, states)
183
184 add_or_update_dynamic_node(customers, dist, 99, 4.0, -6.0, 14.0, 17.0, 900.0, 3.2)
185 add_or_update_dynamic_node(
186     customers,
187     dist,
188     100,
189     5.5,
190     4.0,
191     relocate_job.et,
192     relocate_job.lt,

```

```

193         relocate_job.weight,
194         relocate_job.volume,
195     )
196     return [
197         DynamicEvent("cancel", T_NOW, target_job_id=cancel_job.jid, priority=1.0, description=f"取消客户{
cancel_job.customer_id}未服务任务"),
198         DynamicEvent(
199             "add",
200             T_NOW,
201             new_customer_id=99,
202             new_x=4.0,
203             new_y=-6.0,
204             new_weight=900.0,
205             new_volume=3.2,
206             new_et=14.0,
207             new_lt=17.0,
208             priority=1.3,
209             description="新增订单D1, 虚拟节点99",
210         ),
211         DynamicEvent(
212             "relocate",
213             T_NOW,
214             target_job_id=relocate_job.jid,
215             new_customer_id=100,
216             new_x=5.5,
217             new_y=4.0,
218             new_weight=relocate_job.weight,
219             new_volume=relocate_job.volume,
220             new_et=relocate_job.et,
221             new_lt=relocate_job.lt,
222             priority=1.2,
223             description=f"客户{relocate_job.customer_id}地址变更至虚拟节点100",
224         ),
225         DynamicEvent(
226             "time_window",
227             T_NOW,
228             target_job_id=tw_job.jid,
229             new_et=max(T_NOW + 0.5, tw_job.et),
230             new_lt=max(T_NOW + 1.2, min(tw_job.lt, T_NOW + 2.0)),
231             priority=1.5,
232             description=f"客户{tw_job.customer_id}时间窗提前",
233         ),
234     ]
235
236
237 def remove_flexible_job(states: Dict[str, VehicleState], job_id: int) -> Optional[gl.Job]:
238     for state in states.values():
239         kept = []
240         removed = None
241         for job in state.flexible_jobs:
242             if job.jid == job_id:
243                 removed = job
244             else:
245                 kept.append(job)
246         state.flexible_jobs = kept
247         if removed is not None:
248             return removed
249     return None
250
251
252 def apply_events(
253     states: Dict[str, VehicleState],
254     events: List[DynamicEvent],
255     job_by_id: Dict[int, gl.Job],
256     customers: Dict[int, dict],
257 ) -> List[RequestState]:
258     bank: List[RequestState] = []
259     for event in events:
260         if event.event_type == "cancel" and event.target_job_id is not None:
261             remove_flexible_job(states, event.target_job_id)
262         elif event.event_type == "add":
263             c = customers[int(event.new_customer_id)]
264             job = gl.Job(

```

```

265         jid=900000 + int(event.new_customer_id),
266         customer_id=int(event.new_customer_id),
267         weight=float(event.new_weight),
268         volume=float(event.new_volume),
269         x=float(event.new_x),
270         y=float(event.new_y),
271         et=float(event.new_et),
272         lt=float(event.new_lt),
273         in_green_zone=bool(c["green"]),
274     )
275     bank.append(RequestState(job.jid, job, "new", event.priority, event.event_type))
276     elif event.event_type == "relocate" and event.target_job_id is not None:
277         old = remove_flexible_job(states, event.target_job_id)
278         if old is not None:
279             c = customers[int(event.new_customer_id)]
280             job = gl.Job(
281                 jid=900000 + int(event.new_customer_id),
282                 customer_id=int(event.new_customer_id),
283                 weight=old.weight,
284                 volume=old.volume,
285                 x=float(event.new_x),
286                 y=float(event.new_y),
287                 et=old.et,
288                 lt=old.lt,
289                 in_green_zone=bool(c["green"]),
290             )
291             bank.append(RequestState(job.jid, job, "conflict", event.priority, event.event_type))
292     elif event.event_type == "time_window" and event.target_job_id is not None:
293         old = remove_flexible_job(states, event.target_job_id)
294         if old is not None:
295             job = gl.Job(
296                 jid=old.jid,
297                 customer_id=old.customer_id,
298                 weight=old.weight,
299                 volume=old.volume,
300                 x=old.x,
301                 y=old.y,
302                 et=float(event.new_et),
303                 lt=float(event.new_lt),
304                 in_green_zone=old.in_green_zone,
305             )
306             bank.append(RequestState(job.jid, job, "conflict", event.priority, event.event_type))
307     return bank
308
309
310 def unused_vehicles(routes: List[gl.Route], vehicles: List[gl.Vehicle]) -> List[gl.Vehicle]:
311     used = {r.vehicle.vid for r in routes if r.jobs}
312     return [v for v in vehicles if v.vid not in used]
313
314
315 def feasible_append(route: gl.Route, job: gl.Job) -> bool:
316     w = sum(j.weight for j in route.jobs)
317     v = sum(j.volume for j in route.jobs)
318     return (
319         w + job.weight <= route.vehicle.capacity_weight + 1e-9
320         and v + job.volume <= route.vehicle.capacity_volume + 1e-9
321         and gl.vehicle_allowed(route.vehicle, job, "time_shift")
322     )
323
324
325 def insertion_delta(route: gl.Route, pos: int, job: gl.Job, dist: Dict[Tuple[int, int], float]) -> float:
326     prev_id = 0 if pos == 0 else route.jobs[pos - 1].customer_id
327     next_id = 0 if pos == len(route.jobs) else route.jobs[pos].customer_id
328     return (
329         gl.distance_between(prev_id, job.customer_id, dist)
330         + gl.distance_between(job.customer_id, next_id, dist)
331         - gl.distance_between(prev_id, next_id, dist)
332     )
333
334
335 def insert_request(
336     routes: List[gl.Route],
337     unused: List[gl.Vehicle],

```

```

338     request: RequestState,
339     customers: Dict[int, dict],
340     dist: Dict[Tuple[int, int], float],
341     allow_active_for_new: bool,
342 ) -> bool:
343     best = None
344     job = request.job
345     for ri, route in enumerate(routes):
346         # New orders are not on active vehicles unless explicitly allowed.
347         if request.status == "new" and not allow_active_for_new:
348             continue
349         if not feasible_append(route, job):
350             continue
351         frozen_len = 0
352         for pos in range(frozen_len, len(route.jobs) + 1):
353             score = insertion_delta(route, pos, job, dist) / request.priority
354             if best is None or score < best[0]:
355                 best = (score, "existing", ri, pos, None)
356
357     for vi, vehicle in enumerate(unused):
358         if not gl.vehicle_allowed(vehicle, job, "time_shift"):
359             continue
360         slack_w = (vehicle.capacity_weight - job.weight) / vehicle.capacity_weight
361         slack_v = (vehicle.capacity_volume - job.volume) / vehicle.capacity_volume
362         score = (vehicle.fixed_cost + 850.0 * (slack_w + slack_v) + 0.15 * gl.distance_between(0, job.
customer_id, dist)) / request.priority
363         if best is None or score < best[0]:
364             best = (score, "new", None, None, vi)
365
366     if best is None:
367         return False
368     _, kind, ri, pos, vi = best
369     if kind == "existing":
370         routes[int(ri)].jobs.insert(int(pos), job)
371     else:
372         vehicle = unused.pop(int(vi))
373         routes.append(gl.Route(vehicle, [job]))
374     return True
375
376
377 def build_routes_from_states(states: Dict[str, VehicleState]) -> List[gl.Route]:
378     return [gl.Route(s.vehicle, s.frozen_jobs + s.flexible_jobs) for s in states.values() if s.frozen_jobs
or s.flexible_jobs]
379
380
381 def strategy_hold(base_states: Dict[str, VehicleState], events: List[DynamicEvent], job_by_id: Dict[int, gl
.Job], customers: Dict[int, dict]) -> Tuple[List[gl.Route], List[RequestState]]:
382     states = copy.deepcopy(base_states)
383     bank = apply_events(states, events, job_by_id, customers)
384     return build_routes_from_states(states), bank
385
386
387 def strategy_greedy(base_states: Dict[str, VehicleState], events: List[DynamicEvent], job_by_id: Dict[int,
gl.Job], customers: Dict[int, dict], dist: Dict[Tuple[int, int], float], vehicles: List[gl.Vehicle])
-> Tuple[List[gl.Route], List[RequestState]]:
388     states = copy.deepcopy(base_states)
389     bank = apply_events(states, events, job_by_id, customers)
390     routes = build_routes_from_states(states)
391     unused = unused_vehicles(routes, vehicles)
392     unserved: List[RequestState] = []
393     for request in sorted(bank, key=lambda r: (-r.priority, r.job.lt)):
394         if not insert_request(routes, unused, request, customers, dist, allow_active_for_new=False):
395             unserved.append(request)
396     return routes, unserved
397
398
399 def strategy_rolling(base_states: Dict[str, VehicleState], events: List[DynamicEvent], job_by_id: Dict[int,
gl.Job], customers: Dict[int, dict], dist: Dict[Tuple[int, int], float], vehicles: List[gl.Vehicle])
-> Tuple[List[gl.Route], List[RequestState]]:
400     states = copy.deepcopy(base_states)
401     bank = apply_events(states, events, job_by_id, customers)
402     for state in states.values():
403         for job in state.flexible_jobs:

```

```

404         bank.append(RequestState(job.jid, job, "open", 1.0, "release"))
405         state.flexible_jobs = []
406         routes = build_routes_from_states(states)
407         unused = unused_vehicles(routes, vehicles)
408         unserved: List[RequestState] = []
409         for request in sorted(bank, key=lambda r: (-r.priority, r.job.lt, -r.job.weight)):
410             if not insert_request(routes, unused, request, customers, dist, allow_active_for_new=False):
411                 unserved.append(request)
412         return routes, unserved
413
414
415 def evaluate_dynamic(
416     scenario: str,
417     routes: List[gl.Route],
418     unserved: List[RequestState],
419     customers: Dict[int, dict],
420     dist: Dict[Tuple[int, int], float],
421     base_sig: Dict[str, Tuple[int, ...]],
422 ) -> dict:
423     metrics = gl.evaluate_solution(routes, customers, dist, "time_shift")
424     stability = stability_change(base_sig, routes)
425     stability_cost = STABILITY_ETA * stability * max(metrics["used_vehicle_count"], 1)
426     unserved_penalty = sum(UNSERVED_PENALTY * r.priority for r in unserved)
427     metrics["scenario"] = scenario
428     metrics["label"] = SCENARIO_LABELS[scenario]
429     metrics["operating_cost"] = metrics["total_cost"]
430     metrics["stability_change"] = stability
431     metrics["stability_cost"] = stability_cost
432     metrics["unserved_count"] = len(unserved)
433     metrics["unserved_penalty"] = unserved_penalty
434     metrics["dynamic_total_cost"] = metrics["total_cost"] + stability_cost + unserved_penalty
435     metrics["response_rate"] = 1.0 if not unserved else 1.0 - len(unserved) / 3.0
436     return metrics
437
438
439 def save_route_files(name: str, routes: List[gl.Route], customers: Dict[int, dict], dist: Dict[Tuple[int,
440     int], float]) -> None:
441     metrics = gl.evaluate_solution(routes, customers, dist, "time_shift")
442     rows, stops = [], []
443     for route_no, detail in enumerate(metrics["route_details"], start=1):
444         rows.append({k: v for k, v in detail.items() if k != "stops"} | {"route_no": route_no})
445         for seq, stop in enumerate(detail["stops"], start=1):
446             stops.append({"route_no": route_no, "seq": seq, **stop})
447     pd.DataFrame(rows).to_csv(OUT_DIR / f"{name}_routes_summary.csv", index=False, encoding="utf-8-sig")
448     pd.DataFrame(stops).to_csv(OUT_DIR / f"{name}_stops_detail.csv", index=False, encoding="utf-8-sig")
449
450
451 def normalize(series: pd.Series, lower_is_better: bool) -> np.ndarray:
452     vals = series.to_numpy(dtype=float)
453     mn, mx = vals.min(), vals.max()
454     if abs(mx - mn) < 1e-9:
455         return np.ones_like(vals)
456     scaled = (vals - mn) / (mx - mn)
457     return 1.0 - scaled if lower_is_better else scaled
458
459
460 def plot_radar(summary: pd.DataFrame) -> None:
461     policy = summary[summary["scenario"] != "baseline"].copy()
462     metrics = [
463         ("dynamic_total_cost", "成本", True),
464         ("total_co2", "碳排放", True),
465         ("stability_change", "稳定", True),
466         ("response_rate", "响应", False),
467         ("used_vehicle_count", "车辆", True),
468     ]
469     labels = [m[1] for m in metrics]
470     angles = np.linspace(0, 2 * np.pi, len(metrics), endpoint=False).tolist()
471     angles += angles[:1]
472
473     fig, ax = plt.subplots(figsize=(7.8, 7.2), subplot_kw={"polar": True})
474     colors = ["#08306b", "#2171b5", "#6baed6"]
475     for color, (_, row) in zip(colors, policy.iterrows()):
476         values = []

```

```

476         for col, _, lower in metrics:
477             values.append(normalize(policy[col], lower)[list(policy.index).index(row.name)])
478             values += values[1:]
479             ax.plot(angles, values, color=color, linewidth=2, label=row["label"])
480             ax.fill(angles, values, color=color, alpha=0.13)
481         ax.set_xticks(angles[:-1])
482         ax.set_xticklabels(labels)
483         ax.set_yticklabels([])
484         ax.set_title("动态响应策略综合表现雷达图", color="#08306b", fontweight="bold", pad=18)
485         ax.legend(loc="lower center", bbox_to_anchor=(0.5, -0.16), ncol=3, frameon=False)
486         fig.savefig(OUT_DIR / "q3_event_radar.png", bbox_inches="tight")
487         plt.close(fig)
488
489
490 def plot_bubble(summary: pd.DataFrame) -> None:
491     fig, ax = plt.subplots(figsize=(8.5, 5.8))
492     policy = summary[summary["scenario"] != "baseline"].copy()
493     size = 260 + 3600 * policy["stability_change"].to_numpy()
494     colors = ["#08306b", "#2171b5", "#6baed6"]
495     ax.scatter(policy["total_co2"], policy["dynamic_total_cost"], s=size, c=colors, alpha=0.75, edgecolors=
496               "#08306b", linewidths=0.8)
497     for _, row in policy.iterrows():
498         ax.text(row["total_co2"], row["dynamic_total_cost"], row["label"], ha="center", va="center", color=
499               "white", fontsize=9)
500     ax.set_xlabel("总碳排放量 / kg")
501     ax.set_ylabel("动态总成本 / 元")
502     ax.set_title("成本-碳排-扰动气泡图")
503     ax.grid(alpha=0.25)
504     fig.savefig(OUT_DIR / "q3_event_cost_carbon_bubble.png", bbox_inches="tight")
505     plt.close(fig)
506
507
508 def plot_timeline(events: List[DynamicEvent]) -> None:
509     fig, ax = plt.subplots(figsize=(10, 3.4))
510     y = np.arange(len(events))[:-1]
511     colors = ["#08306b", "#2171b5", "#4292c6", "#9ecae1"]
512     for yi, event, color in zip(y, events, colors):
513         ax.scatter([event.event_time], [yi], s=180, color=color, edgecolors="#08306b", zorder=3)
514         ax.plot([event.event_time, event.event_time + 1.2], [yi, yi], color=color, linewidth=3, alpha=0.5)
515         ax.text(event.event_time + 1.3, yi, event.description, va="center", fontsize=9)
516     ax.set_yticks(y)
517     ax.set_yticklabels([e.event_type for e in events])
518     ax.set_xlim(T_NOW - 0.5, T_NOW + 4.2)
519     ax.set_xlabel("时间 / h")
520     ax.set_title("突发事件实时感知时间线")
521     ax.grid(axis="x", alpha=0.25)
522     fig.savefig(OUT_DIR / "q3_event_timeline.png", bbox_inches="tight")
523     plt.close(fig)
524
525
526 def plot_route_maps(customers: Dict[int, dict], scenario_routes: Dict[str, List[gl.Route]]) -> None:
527     fig, axes = plt.subplots(2, 2, figsize=(13.5, 11))
528     theta = np.linspace(0, 2 * np.pi, 320)
529     route_colors = plt.cm.Blues(np.linspace(0.35, 0.95, 20))
530     for ax, (name, routes) in zip(axes.ravel(), scenario_routes.items()):
531         normal_x, normal_y, green_x, green_y = [], [], [], []
532         for cid, c in customers.items():
533             if cid == 0:
534                 continue
535             if c["green"]:
536                 green_x.append(c["x"])
537                 green_y.append(c["y"])
538             else:
539                 normal_x.append(c["x"])
540                 normal_y.append(c["y"])
541         ax.plot(gl.GREEN_RADIUS * np.cos(theta), gl.GREEN_RADIUS * np.sin(theta), "--", color="#1f77b4", lw
542               =1.3)
543         ax.scatter(normal_x, normal_y, s=15, c="#c6dbef", edgecolors="#2171b5", linewidths=0.25)
544         ax.scatter(green_x, green_y, s=24, c="#3182bd", edgecolors="#08306b", linewidths=0.35)
545         ax.scatter([gl.DEPOT_X], [gl.DEPOT_Y], marker="*", s=150, c="#08306b")
546         for idx, route in enumerate(sorted(routes, key=lambda r: len(r.jobs), reverse=True)[:30]):
547             ids = [0] + [j.customer_id for j in route.jobs] + [0]
548             xs = [customers[i]["x"] for i in ids]

```

```

546         ys = [customers[i]["y"] for i in ids]
547         ax.plot(xs, ys, color=route_colors[idx % 20], lw=0.82, alpha=0.72, linestyle="-" if route.
vehicle.energy_type == "electric" else ":")
548         ax.set_title(SCENARIO_LABELS[name])
549         ax.set_xlabel("x / km")
550         ax.set_ylabel("y / km")
551         ax.grid(alpha=0.22)
552         ax.set_aspect("equal", adjustable="box")
553     fig.suptitle("事件驱动动态调度路径对比", fontsize=16, fontweight="bold", color="#08306b")
554     fig.tight_layout()
555     fig.savefig(OUT_DIR / "q3_event_route_maps.png", bbox_inches="tight")
556     plt.close(fig)
557
558
559 def write_event_table(events: List[DynamicEvent]) -> None:
560     rows = []
561     for event in events:
562         rows.append(
563             {
564                 "事件类型": event.event_type,
565                 "发生时刻": event.event_time,
566                 "目标任务": event.target_job_id,
567                 "虚拟节点": event.new_customer_id,
568                 "优先级": event.priority,
569                 "说明": event.description,
570             }
571         )
572     pd.DataFrame(rows).to_csv(OUT_DIR / "q3_event_list.csv", index=False, encoding="utf-8-sig")
573
574
575 def main() -> None:
576     setup_matplotlib()
577     OUT_DIR.mkdir(parents=True, exist_ok=True)
578     customers, dist, vehicles, _ = gl.load_data()
579     base_routes, job_by_id = load_base_routes(customers, vehicles)
580     base_states = split_vehicle_states(base_routes, customers, dist)
581     events = build_events(base_routes, base_states, customers, dist)
582     write_event_table(events)
583     base_sig = route_signature(base_routes)
584
585     hold_routes, hold_unserved = strategy_hold(base_states, events, job_by_id, customers)
586     greedy_routes, greedy_unserved = strategy_greedy(base_states, events, job_by_id, customers, dist,
vehicles)
587     rolling_routes, rolling_unserved = strategy_rolling(base_states, events, job_by_id, customers, dist,
vehicles)
588     scenario_routes = {
589         "baseline": base_routes,
590         "hold": hold_routes,
591         "greedy": greedy_routes,
592         "rolling": rolling_routes,
593     }
594     rows = [
595         evaluate_dynamic("baseline", base_routes, [], customers, dist, base_sig),
596         evaluate_dynamic("hold", hold_routes, hold_unserved, customers, dist, base_sig),
597         evaluate_dynamic("greedy", greedy_routes, greedy_unserved, customers, dist, base_sig),
598         evaluate_dynamic("rolling", rolling_routes, rolling_unserved, customers, dist, base_sig),
599     ]
600     summary = pd.DataFrame([k: v for k, v in row.items() if k != "route_details"] for row in rows])
601     summary.to_csv(OUT_DIR / "q3_event_strategy_summary.csv", index=False, encoding="utf-8-sig")
602     for name, routes in scenario_routes.items():
603         save_route_files(name, routes, customers, dist)
604     plot_radar(summary)
605     plot_bubble(summary)
606     plot_timeline(events)
607     plot_route_maps(customers, scenario_routes)
608     print(f"saved event-driven q3 outputs to: {OUT_DIR}")
609     print(summary[["label", "dynamic_total_cost", "unserved_count", "response_rate", "total_co2", "
stability_change", "used_vehicle_count"]].to_string(index=False))
610
611
612 if __name__ == "__main__":
613     main()

```

Listing 6: dynamic_dispatch_q3_event_driven.py

A.7 问题三变更路径可视化程序

```
1 from __future__ import annotations
2
3 from pathlib import Path
4
5 import matplotlib
6
7 matplotlib.use("Agg")
8 import matplotlib.pyplot as plt
9 import numpy as np
10 import pandas as pd
11
12
13 ROOT = Path(r"C:/Users/15826/Documents/Codex/2026-04-25/files-mentioned-by-the-user-a")
14 SOURCE_DIR = ROOT / "outputs_green_logistics_depot_20_20" / "q3_event_driven_figures"
15 OUT_DIR = ROOT / "outputs_green_logistics_depot_20_20" / "q3_optimal_local_insert_figures"
16 CUSTOMER_FILE = Path(r"C:/Users/15826/Desktop/华中杯/处理数据/客户需求汇总结果.xlsx")
17
18 DEPOT_X = 20.0
19 DEPOT_Y = 20.0
20 GREEN_RADIUS = 10.0
21
22
23 def setup_matplotlib() -> None:
24     plt.rcParams["font.sans-serif"] = ["SimHei", "Microsoft YaHei", "Arial Unicode MS", "DejaVu Sans"]
25     plt.rcParams["axes.unicode_minus"] = False
26     plt.rcParams["figure.dpi"] = 170
27
28
29 def coords() -> dict[int, dict]:
30     customers = pd.read_excel(CUSTOMER_FILE)
31     c = {
32         int(r.customer_id): {"x": float(r.x), "y": float(r.y), "green": bool(r.in_green_zone)}
33         for r in customers.itertuples(index=False)
34     }
35     c[0] = {"x": DEPOT_X, "y": DEPOT_Y, "green": False}
36     c[99] = {"x": 4.0, "y": -6.0, "green": True}
37     c[100] = {"x": 5.5, "y": 4.0, "green": True}
38     return c
39
40
41 def load_routes(name: str) -> tuple[pd.DataFrame, pd.DataFrame]:
42     return (
43         pd.read_csv(SOURCE_DIR / f"{name}_routes_summary.csv"),
44         pd.read_csv(SOURCE_DIR / f"{name}_stops_detail.csv"),
45     )
46
47
48 def route_sequences(routes: pd.DataFrame, stops: pd.DataFrame) -> pd.DataFrame:
49     seq = (
50         stops.sort_values(["route_no", "seq"])
51         .groupby("route_no")["customer_id"]
52         .apply(lambda s: tuple(int(x) for x in s))
53         .reset_index(name="seq_tuple")
54     )
55     out = routes.merge(seq, on="route_no", how="left")
56     out["seq_tuple"] = out["seq_tuple"].apply(lambda x: tuple() if pd.isna(x) else x)
57     return out
58
59
60 def identify_changes() -> pd.DataFrame:
61     base_routes, base_stops = load_routes("baseline")
62     greedy_routes, greedy_stops = load_routes("greedy")
63     base = route_sequences(base_routes, base_stops)
64     greedy = route_sequences(greedy_routes, greedy_stops)
65     merged = base[["vehicle_id", "vehicle_type", "energy_type", "seq_tuple", "distance", "co2"]].merge(
```

```

66     greedy[["vehicle_id", "vehicle_type", "energy_type", "seq_tuple", "distance", "co2"]],
67     on="vehicle_id",
68     how="outer",
69     suffixes=("_before", "_after"),
70 )
71 rows = []
72 for row in merged.itertuples(index=False):
73     before = tuple() if not isinstance(row.seq_tuple_before, tuple) else row.seq_tuple_before
74     after = tuple() if not isinstance(row.seq_tuple_after, tuple) else row.seq_tuple_after
75     if before == after:
76         continue
77     added = [x for x in after if x not in before]
78     removed = [x for x in before if x not in after]
79     rows.append(
80         {
81             "vehicle_id": row.vehicle_id,
82             "vehicle_type": row.vehicle_type_before if pd.notna(row.vehicle_type_before) else row.
vehicle_type_after,
83             "energy_type": row.energy_type_before if pd.notna(row.energy_type_before) else row.
energy_type_after,
84             "before_path": " -> ".join(map(str, before)) if before else "-",
85             "after_path": " -> ".join(map(str, after)) if after else "-",
86             "added_nodes": ", ".join(map(str, added)) if added else "-",
87             "removed_nodes": ", ".join(map(str, removed)) if removed else "-",
88             "distance_change_km": (row.distance_after if pd.notna(row.distance_after) else 0)
- (row.distance_before if pd.notna(row.distance_before) else 0),
89             "co2_change_kg": (row.co2_after if pd.notna(row.co2_after) else 0)
- (row.co2_before if pd.notna(row.co2_before) else 0),
90         }
91     )
92 )
93 changed = pd.DataFrame(rows)
94 changed.to_csv(OUT_DIR / "q3_changed_routes_detail.csv", index=False, encoding="utf-8-sig")
95 return changed
96
97
98
99 def path_xy(path: list[int], c: dict[int, dict]) -> tuple[list[float], list[float]]:
100     ids = [0] + [int(x) for x in path if int(x) in c] + [0]
101     return [c[i]["x"] for i in ids], [c[i]["y"] for i in ids]
102
103
104 def parse_path(s: str) -> list[int]:
105     if s == "-" or not isinstance(s, str):
106         return []
107     return [int(x.strip()) for x in s.split("->")]
108
109
110 def draw_background(ax, c: dict[int, dict]) -> None:
111     theta = np.linspace(0, 2 * np.pi, 240)
112     ax.plot(GREEN_RADIUS * np.cos(theta), GREEN_RADIUS * np.sin(theta), "--", color="#1f77b4", lw=1.1,
alpha=0.8)
113     normal_x, normal_y, green_x, green_y = [], [], [], []
114     for cid, node in c.items():
115         if cid in (0, 99, 100):
116             continue
117         if node["green"]:
118             green_x.append(node["x"])
119             green_y.append(node["y"])
120         else:
121             normal_x.append(node["x"])
122             normal_y.append(node["y"])
123     ax.scatter(normal_x, normal_y, s=10, c="#d9ecf7", edgecolors="#9ecae1", linewidths=0.2, zorder=1)
124     ax.scatter(green_x, green_y, s=14, c="#9ecae1", edgecolors="#3182bd", linewidths=0.25, zorder=2)
125     ax.scatter([DEPOT_X], [DEPOT_Y], marker="*", s=130, c="#08306b", zorder=5)
126     ax.scatter([c[99]["x"], c[100]["x"]], [c[99]["y"], c[100]["y"]], marker="D", s=72, c="#f28e2b",
edgecolors="#8a4b08", zorder=6)
127     ax.text(c[99]["x"] + 0.7, c[99]["y"], "D1(99)", fontsize=8, color="#8a4b08")
128     ax.text(c[100]["x"] + 0.7, c[100]["y"], "新址(100)", fontsize=8, color="#8a4b08")
129
130
131 def plot_changed_routes(changed: pd.DataFrame) -> None:
132     c = coords()
133     if changed.empty:
134         return

```

```

135 # Show routes with dynamic nodes first, then largest distance changes.
136 changed = changed.copy()
137 changed["priority"] = changed["after_path"].str.contains("99|100", regex=True).astype(int)
138 changed = changed.sort_values(["priority", "distance_change_km"], ascending=[False, False]).head(6)
139
140 fig, axes = plt.subplots(2, 3, figsize=(15, 9.2))
141 for ax, row in zip(axes.ravel(), changed.itertuples(index=False)):
142     draw_background(ax, c)
143     before = parse_path(row.before_path)
144     after = parse_path(row.after_path)
145     if before:
146         xs, ys = path_xy(before, c)
147         ax.plot(xs, ys, color="#8da0b6", linestyle="--", linewidth=1.5, alpha=0.9, label="调整前")
148         ax.scatter(xs[1:-1], ys[1:-1], s=26, c="#b8c7d9", edgecolors="#5d738b", zorder=4)
149     if after:
150         xs, ys = path_xy(after, c)
151         ax.plot(xs, ys, color="#08306b", linestyle="-", linewidth=2.2, alpha=0.95, label="调整后")
152         ax.scatter(xs[1:-1], ys[1:-1], s=32, c="#2171b5", edgecolors="#08306b", zorder=5)
153     for node in set(after) - set(before):
154         if node in c:
155             ax.scatter([c[node]["x"]], [c[node]["y"]], s=110, marker="o", c="#f28e2b", edgecolors="#8
a4b08", zorder=7)
156             ax.text(c[node]["x"] + 0.6, c[node]["y"] + 0.4, f"+{node}", color="#8a4b08", fontsize=9,
fontweight="bold")
157         for node in set(before) - set(after):
158             if node in c:
159                 ax.scatter([c[node]["x"]], [c[node]["y"]], s=110, marker="x", c="#d62728", linewidths=2.0,
zorder=7)
160                 ax.text(c[node]["x"] + 0.6, c[node]["y"] + 0.4, f"--{node}", color="#d62728", fontsize=9,
fontweight="bold")
161         ax.set_title(f"{row.vehicle_id}: {row.before_path} → {row.after_path}", fontsize=9)
162         ax.set_xlabel("x / km")
163         ax.set_ylabel("y / km")
164         ax.grid(alpha=0.22)
165         ax.set_aspect("equal", adjustable="box")
166         ax.legend(loc="upper right", fontsize=8, frameon=False)
167     for ax in axes.ravel()[len(changed) :]:
168         ax.axis("off")
169 fig.suptitle("突发事件触发的重点调整路径", fontsize=16, color="#08306b", fontweight="bold")
170 fig.tight_layout()
171 fig.savefig(OUT_DIR / "q3_changed_routes_focus.png", bbox_inches="tight")
172 plt.close(fig)
173
174
175 def plot_strategy_cards(changed: pd.DataFrame) -> None:
176     fig, ax = plt.subplots(figsize=(11, 5.2))
177     ax.axis("off")
178     cards = [
179         ("订单取消", "客户49未服务任务取消", "从原路径删除该任务, 释放车辆容量; 后续节点顺延重算到达时间。"),
180         ("新增订单", "新增订单D1, 虚拟节点99", "因订单未预装, 优先分配给尚未出发或备用车辆; 若可行则按最小边际成
本插入。"),
181         ("地址变更", "客户55迁移至虚拟节点100", "删除原地址任务, 将新地址作为冲突任务重新插入; 若位于绿区则校验
燃油车限行。"),
182         ("时间窗调整", "客户27时间窗提前", "若原到达时间不满足新时间窗, 则移入任务池并优先插入可准时服务路径。")
183     ]
184     x0s = [0.03, 0.52, 0.03, 0.52]
185     y0s = [0.58, 0.58, 0.12, 0.12]
186     for (title, subtitle, body), x, y in zip(cards, x0s, y0s):
187         rect = plt.Rectangle((x, y), 0.45, 0.32, transform=ax.transAxes, facecolor="#eef6fc", edgecolor="
#2171b5", linewidth=1.2)
188         ax.add_patch(rect)
189         ax.text(x + 0.025, y + 0.235, title, transform=ax.transAxes, fontsize=14, color="#08306b",
fontweight="bold")
190         ax.text(x + 0.025, y + 0.17, subtitle, transform=ax.transAxes, fontsize=10, color="#2171b5")
191         ax.text(x + 0.025, y + 0.07, body, transform=ax.transAxes, fontsize=9, color="#08306b", wrap=True)
192     ax.set_title("突发事件下的车辆调度策略", fontsize=17, color="#08306b", fontweight="bold", pad=14)
193     fig.savefig(OUT_DIR / "q3_event_dispatch_strategy_cards.png", bbox_inches="tight")
194     plt.close(fig)
195
196
197 def main() -> None:
198     setup_matplotlib()

```

```

199     OUT_DIR.mkdir(parents=True, exist_ok=True)
200     changed = identify_changes()
201     plot_changed_routes(changed)
202     plot_strategy_cards(changed)
203     print(f"saved changed-route focus figures to: {OUT_DIR}")
204     print(changed.to_string(index=False))
205
206
207 if __name__ == "__main__":
208     main()

```

Listing 7: make_q3_changed_routes_focus.py

A.8 问题三最优调整可视化程序

```

1  from __future__ import annotations
2
3  from pathlib import Path
4
5  import matplotlib
6
7  matplotlib.use("Agg")
8  import matplotlib.pyplot as plt
9  import numpy as np
10 import pandas as pd
11
12
13 ROOT = Path(r"C:/Users/15826/Documents/Codex/2026-04-25/files-mentioned-by-the-user-a")
14 SOURCE_DIR = ROOT / "outputs_green_logistics_depot_20_20" / "q3_event_driven_figures"
15 OUT_DIR = ROOT / "outputs_green_logistics_depot_20_20" / "q3_optimal_local_insert_figures"
16 CUSTOMER_FILE = Path(r"C:/Users/15826/Desktop/华中杯/处理数据/客户需求汇总结果.xlsx")
17
18 GREEN_RADIUS = 10.0
19 DEPOT_X = 20.0
20 DEPOT_Y = 20.0
21
22
23 def setup_matplotlib() -> None:
24     plt.rcParams["font.sans-serif"] = ["SimHei", "Microsoft YaHei", "Arial Unicode MS", "DejaVu Sans"]
25     plt.rcParams["axes.unicode_minus"] = False
26     plt.rcParams["figure.dpi"] = 160
27
28
29 def load_customer_coords() -> dict[int, dict]:
30     customers = pd.read_excel(CUSTOMER_FILE)
31     coords = {
32         int(r.customer_id): {
33             "x": float(r.x),
34             "y": float(r.y),
35             "green": bool(r.in_green_zone),
36         }
37         for r in customers.itertuples(index=False)
38     }
39     coords[0] = {"x": DEPOT_X, "y": DEPOT_Y, "green": False}
40     coords[99] = {"x": 4.0, "y": -6.0, "green": True}
41     coords[100] = {"x": 5.5, "y": 4.0, "green": True}
42     return coords
43
44
45 def plot_event_timeline() -> None:
46     events = pd.read_csv(SOURCE_DIR / "q3_event_list.csv")
47     fig, ax = plt.subplots(figsize=(10, 3.4))
48     y = np.arange(len(events))[:-1]
49     colors = ["#08306b", "#2171b5", "#4292c6", "#9ecae1"]
50     labels = {
51         "cancel": "订单取消",
52         "add": "新增订单",
53         "relocate": "地址变更",
54         "time_window": "时间窗调整",
55     }

```

```

56     for yi, row, color in zip(y, events.itertuples(index=False), colors):
57         ax.scatter([row.发生时刻], [yi], s=180, color=color, edgecolors="#08306b", zorder=3)
58         ax.plot([row.发生时刻, row.发生时刻 + 1.2], [yi, yi], color=color, linewidth=3, alpha=0.5)
59         ax.text(row.发生时刻 + 1.3, yi, row.说明, va="center", fontsize=9)
60     ax.set_yticks(y)
61     ax.set_yticklabels([labels.get(t, t) for t in events["事件类型"]])
62     ax.set_xlim(11.5, 16.2)
63     ax.set_xlabel("时间 / h")
64     ax.set_title("事件驱动局部插入策略的突发事件感知过程")
65     ax.grid(axis="x", alpha=0.25)
66     fig.savefig(OUT_DIR / "q3_optimal_event_timeline.png", bbox_inches="tight")
67     plt.close(fig)
68
69
70 def draw_routes(ax: plt.Axes, title: str, route_file: Path, stop_file: Path, coords: dict[int, dict]) ->
    None:
71     routes = pd.read_csv(route_file)
72     stops = pd.read_csv(stop_file)
73     theta = np.linspace(0, 2 * np.pi, 320)
74     ax.plot(GREEN_RADIUS * np.cos(theta), GREEN_RADIUS * np.sin(theta), "--", color="#1f77b4", lw=1.3)
75
76     normal_x, normal_y, green_x, green_y = [], [], [], []
77     for cid, c in coords.items():
78         if cid == 0:
79             continue
80         if c["green"]:
81             green_x.append(c["x"])
82             green_y.append(c["y"])
83         else:
84             normal_x.append(c["x"])
85             normal_y.append(c["y"])
86     ax.scatter(normal_x, normal_y, s=15, c="#c6dbef", edgecolors="#2171b5", linewidths=0.25)
87     ax.scatter(green_x, green_y, s=25, c="#3182bd", edgecolors="#08306b", linewidths=0.35)
88     ax.scatter([DEPOT_X], [DEPOT_Y], marker="*", s=150, c="#08306b")
89
90     colors = plt.cm.Blues(np.linspace(0.35, 0.95, 20))
91     for idx, route in enumerate(routes.sort_values("distance", ascending=False).head(30).itertuples(index=False)):
92         path = stops[stops["route_no"] == route.route_no].sort_values("seq")["customer_id"].astype(int).to_list()
93         ids = [0] + path + [0]
94         xs = [coords[i]["x"] for i in ids if i in coords]
95         ys = [coords[i]["y"] for i in ids if i in coords]
96         linestyle = "-" if route.energy_type == "electric" else ":"
97         ax.plot(xs, ys, color=colors[idx % 20], lw=0.82, alpha=0.72, linestyle=linestyle)
98     ax.set_title(title)
99     ax.set_xlabel("x / km")
100    ax.set_ylabel("y / km")
101    ax.grid(alpha=0.22)
102    ax.set_aspect("equal", adjustable="box")
103
104
105 def plot_route_adjustment() -> None:
106     coords = load_customer_coords()
107     fig, axes = plt.subplots(1, 2, figsize=(14, 6.5))
108     draw_routes(
109         axes[0],
110         "突变前路径",
111         SOURCE_DIR / "baseline_routes_summary.csv",
112         SOURCE_DIR / "baseline_stops_detail.csv",
113         coords,
114     )
115     draw_routes(
116         axes[1],
117         "局部插入调整后路径",
118         SOURCE_DIR / "greedy_routes_summary.csv",
119         SOURCE_DIR / "greedy_stops_detail.csv",
120         coords,
121     )
122     fig.suptitle("事件驱动局部插入策略路径调整结果", fontsize=16, fontweight="bold", color="#08306b")
123     fig.tight_layout()
124     fig.savefig(OUT_DIR / "q3_optimal_route_adjustment.png", bbox_inches="tight")
125     plt.close(fig)

```

```

126
127
128 def draw_progress_ring(ax: plt.Axes, value: float, label: str, center_text: str, color: str) -> None:
129     ax.pie(
130         [value, max(0.0, 1 - value)],
131         startangle=90,
132         counterclock=False,
133         colors=[color, "#eaf2fb"],
134         wedgeprops={"width": 0.24, "edgecolor": "white"},
135     )
136     ax.text(0, 0.06, center_text, ha="center", va="center", fontsize=15, color="#08306b", fontweight="bold")
137     ax.text(0, -0.18, label, ha="center", va="center", fontsize=9, color="#08306b")
138     ax.set_aspect("equal")
139
140
141 def plot_optimal_summary() -> None:
142     summary = pd.read_csv(SOURCE_DIR / "q3_event_strategy_summary.csv")
143     row = summary[summary["scenario"] == "greedy"].iloc[0]
144     baseline = summary[summary["scenario"] == "baseline"].iloc[0]
145
146     fig = plt.figure(figsize=(12, 7))
147     gs = fig.add_gridspec(2, 3, height_ratios=[1.05, 0.95], hspace=0.34, wspace=0.28)
148
149     ax_cost = fig.add_subplot(gs[0, 0])
150     ax_response = fig.add_subplot(gs[0, 1])
151     ax_stability = fig.add_subplot(gs[0, 2])
152     draw_progress_ring(ax_cost, min(row["dynamic_total_cost"] / 170000, 1), "动态总成本", f"{row['dynamic_total_cost']:.0f}元", "#2171b5")
153     draw_progress_ring(ax_response, row["response_rate"], "任务响应率", f"{row['response_rate']:.0%}", "#08306b")
154     draw_progress_ring(ax_stability, 1 - row["stability_change"], "路径稳定度", f"{1-row['stability_change']:.1%}", "#6baed6")
155
156     ax_text = fig.add_subplot(gs[1, :])
157     ax_text.axis("off")
158     lines = [
159         ("未响应任务数", f"{int(row['unserved_count'])}"),
160         ("总碳排放量", f"{row['total_co2']:.2f} kg"),
161         ("启用车辆数", f"{int(row['used_vehicle_count'])} 辆"),
162         ("路径扰动比例", f"{row['stability_change']:.2%}"),
163         ("相对突变前成本变化", f"{row['dynamic_total_cost'] - baseline['dynamic_total_cost']:.2f} 元"),
164         ("相对突变前碳排变化", f"{row['total_co2'] - baseline['total_co2']:.2f} kg"),
165     ]
166     x_positions = [0.05, 0.38, 0.70, 0.05, 0.38, 0.70]
167     y_positions = [0.72, 0.72, 0.72, 0.32, 0.32, 0.32]
168     for (label, value), x, y in zip(lines, x_positions, y_positions):
169         ax_text.text(x, y + 0.12, label, transform=ax_text.transAxes, fontsize=10, color="#2171b5")
170         ax_text.text(x, y, value, transform=ax_text.transAxes, fontsize=18, color="#08306b", fontweight="bold")
171     ax_text.set_title("事件驱动局部插入策略关键指标", fontsize=14, color="#08306b", fontweight="bold", pad=14)
172
173     fig.suptitle("问题三最优动态调度方案", fontsize=18, fontweight="bold", color="#08306b")
174     fig.savefig(OUT_DIR / "q3_optimal_summary.png", bbox_inches="tight")
175     plt.close(fig)
176
177
178 def main() -> None:
179     setup_matplotlib()
180     OUT_DIR.mkdir(parents=True, exist_ok=True)
181     plot_event_timeline()
182     plot_route_adjustment()
183     plot_optimal_summary()
184     print(f"saved optimal-only q3 figures to: {OUT_DIR}")
185
186
187 if __name__ == "__main__":
188     main()

```

Listing 8: make_q3_optimal_only_visuals.py

A.9 车速灵敏度分析程序

```
1 from __future__ import annotations
2
3 import argparse
4 import copy
5 import importlib.util
6 import json
7 import math
8 import sys
9 from pathlib import Path
10 from typing import Dict, Iterable, List, Optional
11
12 import matplotlib.pyplot as plt
13 import numpy as np
14 import pandas as pd
15
16
17 DEFAULT_PROJECT_DIR = Path(r"C:\Users\15826\Desktop\华中杯")
18 DEFAULT_CONFIG = DEFAULT_PROJECT_DIR / "config.json"
19 DEFAULT_Q1_CODE = DEFAULT_PROJECT_DIR / "problem1.py"
20 DEFAULT_ROUTE_DETAILS = DEFAULT_PROJECT_DIR / "outputs" / "route_details.csv"
21
22 SPEED_STD_BY_MEAN = {
23     55.3: 0.1, # smooth traffic:  $N(55.3, 0.1^2)$ 
24     35.4: 5.2, # normal traffic:  $N(35.4, 5.2^2)$ 
25     9.8: 4.7, # congested traffic:  $N(9.8, 4.7^2)$ 
26 }
27
28
29 def import_q1_module(q1_code: Path):
30     spec = importlib.util.spec_from_file_location("q1_problem1", q1_code)
31     if spec is None or spec.loader is None:
32         raise RuntimeError(f"Cannot import first-question code from {q1_code}")
33     module = importlib.util.module_from_spec(spec)
34     sys.modules[spec.name] = module
35     spec.loader.exec_module(module)
36     return module
37
38
39 def speed_std(speed: float) -> float:
40     nearest_mean = min(SPEED_STD_BY_MEAN, key=lambda x: abs(x - speed))
41     if abs(nearest_mean - speed) <= 1.0:
42         return SPEED_STD_BY_MEAN[nearest_mean]
43     return max(0.05 * speed, 0.1)
44
45
46 def clone_data_with_segments(data, time_segments: List[Dict[str, float]]):
47     cloned = copy.copy(data)
48     cloned.time_segments = [dict(seg) for seg in time_segments]
49     return cloned
50
51
52 def reconstruct_solution_from_details(q1, data, route_details_path: Path):
53     details = pd.read_csv(route_details_path, encoding="utf-8-sig")
54     vehicle_lookup = {v.vehicle_id: v for v in data.vehicles}
55     routes = []
56
57     order_col = "sequence" if "sequence" in details.columns else "stop_order"
58     for vehicle_id, group in details.groupby("vehicle_id", sort=False):
59         if vehicle_id not in vehicle_lookup:
60             raise KeyError(f"Vehicle {vehicle_id} in route details is absent from vehicle list.")
61         group = group.sort_values(order_col)
62         visits = [
63             q1.Visit(
64                 customer_id=str(row["customer_id"]),
65                 deliver_weight=float(row["deliver_weight"]),
66                 deliver_volume=float(row["deliver_volume"]),
67             )
68             for _, row in group.iterrows()
69         ]
70         vt = vehicle_lookup[vehicle_id].vehicle_type
71         weight_over = sum(v.deliver_weight for v in visits) - vt.max_weight
```

```

72     volume_over = sum(v.deliver_volume for v in visits) - vt.max_volume
73     # CSV outputs are rounded to four decimals; trim tiny round-off excesses.
74     if 0 < weight_over < 1e-2 and visits:
75         max(visits, key=lambda v: v.deliver_weight).deliver_weight -= weight_over
76     if 0 < volume_over < 1e-4 and visits:
77         max(visits, key=lambda v: v.deliver_volume).deliver_volume -= volume_over
78     routes.append(q1.Route(vehicle=vehicle_lookup[vehicle_id], visits=visits))
79
80     solution = q1.Solution(
81         routes=routes,
82         unserved_weight={cid: 0.0 for cid in data.customers},
83         unserved_volume={cid: 0.0 for cid in data.customers},
84     )
85     return solution
86
87
88 def solve_or_load_base_solution(q1, data, route_details_path: Optional[Path], args):
89     if route_details_path and route_details_path.exists():
90         return reconstruct_solution_from_details(q1, data, route_details_path)
91
92     solver = q1.GreenLogisticsSolver(data)
93     return solver.solve(
94         iterations=args.iterations,
95         restarts=args.restarts,
96         destroy_min=args.destroy_min,
97         destroy_max=args.destroy_max,
98     )
99
100
101 def reevaluate_solution(q1, data, base_solution):
102     solver = q1.GreenLogisticsSolver(data)
103     routes = []
104     for route in base_solution.routes:
105         new_route = route.copy()
106         metrics = solver._evaluate_route(new_route)
107         if metrics is None:
108             raise RuntimeError(f"Route for vehicle {route.vehicle.vehicle_id} became infeasible.")
109         new_route.metrics = metrics
110         routes.append(new_route)
111     solution = q1.Solution(
112         routes=routes,
113         unserved_weight=dict(base_solution.unserved_weight),
114         unserved_volume=dict(base_solution.unserved_volume),
115     )
116     solution.total_cost = sum(route.metrics.total_cost for route in routes)
117     return solution
118
119
120 def optimize_solution(q1, data, args):
121     solver = q1.GreenLogisticsSolver(data)
122     return solver.solve(
123         iterations=args.iterations,
124         restarts=args.restarts,
125         destroy_min=args.destroy_min,
126         destroy_max=args.destroy_max,
127     )
128
129
130 def summarize_row(q1, solution, data, scenario: str, scenario_type: str, value: float, speeds: List[float])
131 :
132     summary = q1.summarize_solution(solution)
133     total_carbon_kg = (
134         summary["carbon_cost"] / data.carbon_cost_per_unit
135         if data.carbon_cost_per_unit > 0
136         else math.nan
137     )
138     total_travel_time = sum(r.metrics.travel_time for r in solution.routes)
139     total_waiting_time = sum(r.metrics.waiting_time for r in solution.routes)
140     total_late_time = sum(r.metrics.late_time for r in solution.routes)
141     total_energy_amount = sum(r.metrics.energy_amount for r in solution.routes)
142
143     return {
144         "scenario": scenario,

```

```

144         "scenario_type": scenario_type,
145         "value": value,
146         "avg_speed": float(np.mean(speeds)),
147         "min_speed": float(np.min(speeds)),
148         "max_speed": float(np.max(speeds)),
149         "total_cost": summary["total_cost"],
150         "fixed_cost": summary["fixed_cost"],
151         "energy_cost": summary["energy_cost"],
152         "carbon_cost": summary["carbon_cost"],
153         "time_penalty_cost": summary["time_penalty_cost"],
154         "carbon_kg": total_carbon_kg,
155         "energy_amount": total_energy_amount,
156         "total_distance": summary["total_distance"],
157         "route_count": summary["route_count"],
158         "travel_time": total_travel_time,
159         "waiting_time": total_waiting_time,
160         "late_time": total_late_time,
161         "unserved_customer_count": summary["unserved_customer_count"],
162     }
163
164
165 def deterministic_segments(base_segments: List[Dict[str, float]], scale: float):
166     ans = []
167     for seg in base_segments:
168         item = dict(seg)
169         item["speed"] = max(3.0, float(seg["speed"])) * scale
170         ans.append(item)
171     return ans
172
173
174 def sampled_segments(base_segments: List[Dict[str, float]], rng: np.random.Generator):
175     ans = []
176     for seg in base_segments:
177         item = dict(seg)
178         mu = float(seg["speed"])
179         sigma = speed_std(mu)
180         item["speed"] = float(max(3.0, rng.normal(mu, sigma)))
181         ans.append(item)
182     return ans
183
184
185 def run_sensitivity(q1, data, base_solution, args):
186     rows = []
187     base_segments = [dict(seg) for seg in data.time_segments]
188
189     for pct in args.percent_levels:
190         scale = 1.0 + pct / 100.0
191         segs = deterministic_segments(base_segments, scale)
192         scenario_data = clone_data_with_segments(data, segs)
193         solution = optimize_solution(q1, scenario_data, args) if args.reoptimize else reevaluate_solution(
194             q1, scenario_data, base_solution)
195         rows.append(
196             summarize_row(
197                 q1,
198                 solution,
199                 scenario_data,
200                 scenario=f"{pct:.0f}%",
201                 scenario_type="deterministic",
202                 value=float(pct),
203                 speeds=[seg["speed"] for seg in segs],
204             )
205         )
206
207     rng = np.random.default_rng(args.seed)
208     for i in range(args.mc_samples):
209         segs = sampled_segments(base_segments, rng)
210         scenario_data = clone_data_with_segments(data, segs)
211         solution = optimize_solution(q1, scenario_data, args) if args.reoptimize else reevaluate_solution(
212             q1, scenario_data, base_solution)
213         rows.append(
214             summarize_row(
215                 q1,
216                 solution,

```

```

215         scenario_data,
216         scenario=f"MC-{i + 1:03d}",
217         scenario_type="monte_carlo",
218         value=float(i + 1),
219         speeds=[seg["speed"] for seg in segs],
220     )
221 )
222
223 return pd.DataFrame(rows)
224
225
226 def setup_matplotlib():
227     plt.rcParams["axes.unicode_minus"] = False
228     plt.rcParams["font.sans-serif"] = ["SimHei", "Microsoft YaHei", "Arial Unicode MS", "DejaVu Sans"]
229     plt.rcParams["figure.dpi"] = 160
230
231
232 def style_axis(ax: plt.Axes, grid_axis: str = "both") -> None:
233     ax.grid(axis=grid_axis, alpha=0.22, color="#9ecae1", linewidth=0.8)
234     ax.spines["top"].set_visible(False)
235     ax.spines["right"].set_visible(False)
236     ax.spines["left"].set_color("#9ecae1")
237     ax.spines["bottom"].set_color("#9ecae1")
238     ax.tick_params(colors="#08306b", labels=9)
239     ax.xaxis.label.set_color("#08306b")
240     ax.yaxis.label.set_color("#08306b")
241     ax.title.set_color("#08306b")
242
243
244 def plot_results(df: pd.DataFrame, out_dir: Path):
245     setup_matplotlib()
246     det = df[df["scenario_type"] == "deterministic"].sort_values("value")
247     mc = df[df["scenario_type"] == "monte_carlo"].copy()
248     base = det.loc[det["value"].abs().idxmin()]
249
250     fig, ax = plt.subplots(figsize=(9, 5.2))
251     ax.plot(det["value"], det["total_cost"], marker="o", linewidth=2.3, color="#08306b", label="总成本")
252     ax.plot(det["value"], det["energy_cost"], marker="s", linewidth=1.8, color="#2171b5", label="能源成本")
253     ax.plot(det["value"], det["time_penalty_cost"], marker="^", linewidth=1.8, color="#6baed6", label="时间窗惩罚")
254     ax.set_xlabel("车速扰动比例/%")
255     ax.set_ylabel("成本/元")
256     ax.set_title("车速确定性扰动下的成本变化", fontsize=13, fontweight="bold")
257     style_axis(ax)
258     ax.legend(frameon=False)
259     fig.tight_layout()
260     fig.savefig(out_dir / "q1_speed_cost_curves.png", dpi=220, bbox_inches="tight")
261     plt.close(fig)
262
263     fig, ax = plt.subplots(figsize=(8.6, 5.1))
264     relative = 100 * (det["total_cost"] / base["total_cost"] - 1)
265     ax.plot(det["value"], relative, marker="o", color="#08306b", linewidth=2.2)
266     ax.fill_between(det["value"], 0, relative, color="#c6dbef", alpha=0.55)
267     ax.axhline(0, color="#08306b", linewidth=1.0, linestyle="--", alpha=0.8)
268     ax.set_xlabel("车速扰动比例/%")
269     ax.set_ylabel("总成本变化率/%")
270     ax.set_title("相对基准方案的总成本变化", fontsize=13, fontweight="bold")
271     for x, y in zip(det["value"], relative):
272         if x in [-20, 0, 20]:
273             ax.text(x, y, f"{y:.2f}%", ha="center", va="bottom" if y >= 0 else "top", fontsize=8, color="#08306b")
274     style_axis(ax)
275     fig.tight_layout()
276     fig.savefig(out_dir / "q1_speed_relative_cost_change.png", dpi=220, bbox_inches="tight")
277     plt.close(fig)
278
279     fig, ax = plt.subplots(figsize=(9, 5.2))
280     labels = ["total_cost", "carbon_kg", "travel_time", "late_time"]
281     display = ["总成本", "碳排放", "行驶时间", "迟到时间"]
282     data = [mc[col].to_numpy() for col in labels]
283     box = ax.boxplot(data, tick_labels=display, patch_artist=True, showfliers=False)
284     for patch, color in zip(box["boxes"], ["#08306b", "#2171b5", "#6baed6", "#c6dbef"]):
285         patch.set(facecolor=color, alpha=0.72, edgecolor="#08306b", linewidth=0.8)

```

```

286 for median in box["medians"]:
287     median.set(color="white", linewidth=1.5)
288 ax.set_title("原始速度分布抽样下的指标波动", fontsize=13, fontweight="bold")
289 ax.set_ylabel("指标数值")
290 style_axis(ax, grid_axis="y")
291 fig.tight_layout()
292 fig.savefig(out_dir / "q1_speed_monte_carlo_boxplot.png", dpi=220, bbox_inches="tight")
293 plt.close(fig)
294
295 fig, ax = plt.subplots(figsize=(8.8, 5.1))
296 changes = {
297     "总成本": 100 * (det.iloc[-1]["total_cost"] - det.iloc[0]["total_cost"]) / base["total_cost"],
298     "能源成本": 100 * (det.iloc[-1]["energy_cost"] - det.iloc[0]["energy_cost"]) / base["energy_cost"],
299     "碳排放": 100 * (det.iloc[-1]["carbon_kg"] - det.iloc[0]["carbon_kg"]) / base["carbon_kg"],
300     "时间窗惩罚": 100 * (det.iloc[-1]["time_penalty_cost"] - det.iloc[0]["time_penalty_cost"]) / max(
301         base["time_penalty_cost"], 1e-9),
302 }
303 names = list(changes)
304 vals = [changes[k] for k in names]
305 bars = ax.barh(names, vals, color=["#08306b", "#2171b5", "#6baed6", "#c6dbef"], edgecolor="#08306b",
306     linewidth=0.5)
307 ax.axvline(0, color="#08306b", linewidth=1.0)
308 for bar, val in zip(bars, vals):
309     ax.text(val, bar.get_y() + bar.get_height() / 2, f"{val:.2f}%", va="center", ha="left", fontsize=8,
310         color="#08306b")
311 ax.set_xlabel("由 -20% 到 +20% 速度扰动变化幅度/%")
312 ax.set_title("主要指标对车速扰动的敏感程度", fontsize=13, fontweight="bold")
313 style_axis(ax, grid_axis="x")
314 fig.tight_layout()
315 fig.savefig(out_dir / "q1_speed_sensitivity_strength.png", dpi=220, bbox_inches="tight")
316 plt.close(fig)
317
318 fig, ax = plt.subplots(figsize=(8, 5))
319 ax.hist(mc["total_cost"], bins=18, color="#2171b5", alpha=0.86, edgecolor="white")
320 ax.axvline(base["total_cost"], color="#08306b", linestyle="--", linewidth=2, label="均值速度基准")
321 ax.set_xlabel("总成本/元")
322 ax.set_ylabel("频数")
323 ax.set_title("原始速度分布下总成本的蒙特卡洛分布", fontsize=13, fontweight="bold")
324 style_axis(ax, grid_axis="y")
325 ax.legend(frameon=False)
326 fig.tight_layout()
327 fig.savefig(out_dir / "q1_speed_sensitivity_mc_cost.png", dpi=220, bbox_inches="tight")
328 plt.close(fig)
329
330 def latex_table_rows(det: pd.DataFrame) -> str:
331     selected = det[det["value"].isin([-20, -10, 0, 10, 20])].sort_values("value")
332     rows = []
333     base_cost = float(selected[selected["value"] == 0]["total_cost"].iloc[0])
334     for _, row in selected.iterrows():
335         change = 100 * (row["total_cost"] / base_cost - 1)
336         rows.append(
337             f"{row['scenario']} & {row['total_cost']:.2f} & {change:.2f}%% & "
338             f"{row['carbon_kg']:.2f} & {row['time_penalty_cost']:.2f} \\\\"
339         )
340     return "\n".join(rows)
341
342 def write_latex(df: pd.DataFrame, out_dir: Path):
343     det = df[df["scenario_type"] == "deterministic"].sort_values("value")
344     mc = df[df["scenario_type"] == "monte_carlo"]
345     base = det.loc[det["value"].abs().idxmin()]
346     plus20 = det[det["value"] == 20].iloc[0]
347     minus20 = det[det["value"] == -20].iloc[0]
348     mc_mean = mc["total_cost"].mean()
349     mc_std = mc["total_cost"].std(ddof=1)
350     mc_p05 = mc["total_cost"].quantile(0.05)
351     mc_p95 = mc["total_cost"].quantile(0.95)
352
353     latex = rf"""
354 \subsection{{车速扰动灵敏度分析}}
355 在问题一中，分时段车速服从题目给定的正态分布。基准模型为降低求解复杂度，采用各时段速度均值进行确定性计算。为检验

```

该简化处理对调度结果的影响, 本文以问题一得到的基准配送路线为对象, 在保持客户分配、车辆类型与访问顺序不变的条件下, 对车速参数进行扰动并重新计算各项成本。三类交通状态的速度分布设定为

```

\[\begin{aligned}v_{\text{畅通}} &\sim N(55.3, 0.1^2), \\v_{\text{一般}} &\sim N(35.4, 5.2^2), \\v_{\text{拥堵}} &\sim N(9.8, 4.7^2).\end{aligned}

```

首先进行确定性扰动实验, 将所有时段速度均值分别调整 -20% , -15% , $\dots$, 20% , 得到表~\ref{tab:q1_speed_sensitivity}。可以看出, 当速度整体降低 20% 时, 总成本由基准的 $\{base['total_cost']:.2f\}$ 元变化为 $\{minus20['total_cost']:.2f\}$ 元; 当速度整体提高 20% 时, 总成本变化为 $\{plus20['total_cost']:.2f\}$ 元。车速扰动主要通过两条路径影响目标函数: 一是改变行驶时间, 从而影响早到等待和晚到惩罚; 二是通过 U 型能耗函数改变单位里程能耗, 进而影响能源成本和碳排放成本。

```

\begin{table}[H]
\centering
\caption{问题一车速扰动灵敏度结果}
\label{tab:q1_speed_sensitivity}
\begin{tabular}{c}
\toprule
速度扰动 & 总成本/元 & 成本变化率 & 碳排放/kg & 时间窗惩罚/元 \\
\midrule
\{latex\_table\_rows(det)\}
\bottomrule
\end{tabular}
\end{table}

```

进一步地, 按照原始正态分布对每个时段车速进行蒙特卡洛抽样, 重复计算 $\{len(mc)\}$ 次。结果显示, 总成本均值为 $\{mc_mean:.2f\}$ 元, 标准差为 $\{mc_std:.2f\}$ 元, 90% 样本区间为 $[\{mc_p05:.2f\}, \{mc_p95:.2f\}]$ 元。该结果表明, 虽然拥堵时段速度波动较大, 但问题一方案在随机车速扰动下的总成本分布仍集中在基准值附近, 说明基于均值速度得到的调度方案具有一定稳健性。

```

\begin{figure}[H]
\centering
\includegraphics[width=0.78\textwidth]{q1_speed_cost_curves.png}
\caption{车速确定性扰动下的成本变化}
\label{fig:q1_speed_cost_curves}
\end{figure}

\begin{figure}[H]
\centering
\includegraphics[width=0.72\textwidth]{q1_speed_relative_cost_change.png}
\caption{相对基准方案的总成本变化率}
\label{fig:q1_speed_relative_cost_change}
\end{figure}

\begin{figure}[H]
\centering
\includegraphics[width=0.78\textwidth]{q1_speed_monte_carlo_boxplot.png}
\caption{原始速度分布抽样下主要指标的波动}
\label{fig:q1_speed_monte_carlo_boxplot}
\end{figure}

\begin{figure}[H]
\centering
\includegraphics[width=0.72\textwidth]{q1_speed_sensitivity_strength.png}
\caption{主要指标对车速扰动的敏感程度}
\label{fig:q1_speed_sensitivity_strength}
\end{figure}

\begin{figure}[H]
\centering
\includegraphics[width=0.75\textwidth]{q1_speed_sensitivity_mc_cost.png}
\caption{原始速度分布下总成本的蒙特卡洛分布}
\label{fig:q1_speed_sensitivity_mc_cost}
\end{figure}
"""
.strip()

(out_dir / "q1_speed_sensitivity_latex.tex").write_text(latex, encoding="utf-8")

def write_run_config(args, out_dir: Path):
    payload = {

```

```

420     "config": str(args.config),
421     "q1_code": str(args.q1_code),
422     "route_details": str(args.route_details) if args.route_details else None,
423     "mc_samples": args.mc_samples,
424     "percent_levels": args.percent_levels,
425     "reoptimize": args.reoptimize,
426     "speed_std_by_mean": SPEED_STD_BY_MEAN,
427     "seed": args.seed,
428 }
429 (out_dir / "run_config.json").write_text(json.dumps(payload, ensure_ascii=False, indent=2), encoding="
utf-8")
430
431
432 def parse_args():
433     parser = argparse.ArgumentParser(description="Question 1 speed sensitivity analysis.")
434     parser.add_argument("--config", type=Path, default=DEFAULT_CONFIG)
435     parser.add_argument("--q1-code", type=Path, default=DEFAULT_Q1_CODE)
436     parser.add_argument("--route-details", type=Path, default=DEFAULT_ROUTE_DETAILS)
437     parser.add_argument("--output-dir", type=Path, default=Path("q1_sensitivity_results"))
438     parser.add_argument("--mc-samples", type=int, default=100)
439     parser.add_argument("--seed", type=int, default=20260425)
440     parser.add_argument("--percent-levels", type=float, nargs="+", default=[-20, -15, -10, -5, 0, 5, 10,
15, 20])
441     parser.add_argument("--reoptimize", action="store_true", help="Re-run the first-question optimizer for
every scenario.")
442     parser.add_argument("--iterations", type=int, default=600)
443     parser.add_argument("--restarts", type=int, default=6)
444     parser.add_argument("--destroy-min", type=int, default=3)
445     parser.add_argument("--destroy-max", type=int, default=8)
446     return parser.parse_args()
447
448
449 def main():
450     args = parse_args()
451     args.output_dir.mkdir(parents=True, exist_ok=True)
452
453     q1 = import_q1_module(args.q1_code)
454     data = q1.load_problem(args.config)
455     base_solution = solve_or_load_base_solution(q1, data, args.route_details, args)
456     base_solution = reevaluate_solution(q1, data, base_solution)
457
458     df = run_sensitivity(q1, data, base_solution, args)
459     df.to_csv(args.output_dir / "q1_speed_sensitivity_results.csv", index=False, encoding="utf-8-sig")
460     plot_results(df, args.output_dir)
461     write_latex(df, args.output_dir)
462     write_run_config(args, args.output_dir)
463
464     det = df[df["scenario_type"] == "deterministic"].sort_values("value")
465     mc = df[df["scenario_type"] == "monte_carlo"]
466     base = det.loc[det["value"].abs().idxmin()]
467     print(f"Baseline cost: {base['total_cost']:.2f}")
468     print(f"Monte Carlo mean cost: {mc['total_cost'].mean():.2f}")
469     print(f"Monte Carlo std cost: {mc['total_cost'].std(ddof=1):.2f}")
470     print(f"Results written to: {args.output_dir.resolve()}")
471
472
473 if __name__ == "__main__":
474     main()

```

Listing 9: q1_speed_sensitivity_analysis.py